

# **DETECTORNX**

## **EVALUATING SPIKING NEURAL NETWORKS FOR HIGH-DEFINITION AUTOMOTIVE PERCEPTION**

A REPORT SUBMITTED TO THE UNIVERSITY OF MANCHESTER  
FOR THE DEGREE OF BACHELOR OF SCIENCE  
IN THE FACULTY OF SCIENCE AND ENGINEERING

2026

Student id: 11372282

Department of Computer Science

# Contents

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                  | <b>8</b>  |
| <b>Declaration</b>                                               | <b>9</b>  |
| <b>Copyright</b>                                                 | <b>10</b> |
| <b>Acknowledgements</b>                                          | <b>11</b> |
| <b>1 Introduction</b>                                            | <b>14</b> |
| 1.1 Overview . . . . .                                           | 14        |
| 1.2 Motivation . . . . .                                         | 15        |
| 1.3 Project Aims and Objectives . . . . .                        | 15        |
| 1.4 Success Criteria . . . . .                                   | 16        |
| 1.5 Evaluation Approach . . . . .                                | 17        |
| 1.6 Report Structure . . . . .                                   | 18        |
| <b>2 Background</b>                                              | <b>19</b> |
| 2.1 Artificial Neural Networks and Differentiability . . . . .   | 19        |
| 2.2 Convolutional Neural Networks for Object Detection . . . . . | 19        |
| 2.2.1 Convolutional Feature Extraction . . . . .                 | 19        |
| 2.2.2 Residual Learning . . . . .                                | 20        |
| 2.2.3 Object Detection and Bounding Box Regression . . . . .     | 20        |
| 2.3 Spiking Neural Networks . . . . .                            | 21        |
| 2.3.1 The Leaky Integrate-and-Fire Neuron . . . . .              | 21        |
| 2.3.2 Surrogate Gradients . . . . .                              | 23        |
| 2.3.3 Temporal Dynamics and Rapid Categorisation . . . . .       | 23        |
| 2.4 Event-Based Vision . . . . .                                 | 24        |
| 2.4.1 Event Cameras . . . . .                                    | 24        |
| 2.4.2 The Natural SNN–Event Camera Pairing . . . . .             | 25        |
| 2.5 ETraM Dataset . . . . .                                      | 26        |
| 2.5.1 Dataset Overview and Acquisition Setup . . . . .           | 26        |
| 2.5.2 Class Taxonomy, Scale and Annotation . . . . .             | 26        |

|          |                                                                              |           |
|----------|------------------------------------------------------------------------------|-----------|
| 2.5.3    | Suitability for This Project . . . . .                                       | 27        |
| 2.6      | Neuromorphic Hardware and Energy Modelling . . . . .                         | 27        |
| 2.6.1    | Neuromorphic Hardware Platforms . . . . .                                    | 27        |
| 2.6.2    | The MAC vs AC Energy Distinction . . . . .                                   | 27        |
| 2.7      | Related Work . . . . .                                                       | 28        |
| 2.7.1    | The Progression of Spiking Architectures . . . . .                           | 28        |
| 2.7.2    | Event-Based Object Detection . . . . .                                       | 28        |
| 2.7.3    | The Gap This Project Addresses . . . . .                                     | 28        |
| <b>3</b> | <b>Technical Quality and Methodology</b>                                     | <b>29</b> |
| 3.1      | Requirements Analysis . . . . .                                              | 29        |
| 3.1.1    | Functional Requirements . . . . .                                            | 29        |
| 3.1.2    | Non-Functional Requirements and Constraints . . . . .                        | 30        |
| 3.2      | System Architecture Overview . . . . .                                       | 31        |
| 3.3      | Data Preprocessing: The Box-aligned Offline Label-aware Tensorisation (BOLT) |           |
|          | Pipeline . . . . .                                                           | 33        |
| 3.3.1    | Raw Event Ingestion and Windowing . . . . .                                  | 34        |
| 3.3.2    | Signal Conditioning . . . . .                                                | 35        |
| 3.3.3    | Spatiotemporal voxelisation . . . . .                                        | 35        |
| 3.3.4    | Downsampling and Normalisation . . . . .                                     | 36        |
| 3.3.5    | Multi-Object Label Alignment . . . . .                                       | 36        |
| 3.3.6    | Chunking for High-Throughput I/O . . . . .                                   | 38        |
| 3.3.7    | BOLT Pipeline: Visualised . . . . .                                          | 38        |
| 3.4      | Spiking Neural Network (SNN) Architecture . . . . .                          | 40        |
| 3.4.1    | Spatiotemporal Input and Multi-Step Execution . . . . .                      | 40        |
| 3.4.2    | Neuron Dynamics: Leaky Integrate-and-Fire (LIF) . . . . .                    | 40        |
| 3.4.3    | Backbone: SEW-ResNet and Attention . . . . .                                 | 41        |
| 3.4.4    | Firing Rate Regularisation (FRR) . . . . .                                   | 43        |
| 3.4.5    | Multi-Box Regression Head . . . . .                                          | 44        |
| 3.5      | Convolutional Neural Network (CNN) Baseline . . . . .                        | 47        |
| 3.5.1    | Structural Configuration and Layer-Wise Mapping . . . . .                    | 47        |
| 3.5.2    | Input Strategy: Channel-Wise Temporal Integration . . . . .                  | 47        |
| 3.5.3    | Backbone: Identity Mapping and Residual Connections . . . . .                | 48        |
| 3.5.4    | SEWResBlock vs Standard ResBlock . . . . .                                   | 48        |
| 3.5.5    | Attention Mechanism: SE-Block . . . . .                                      | 49        |
| 3.5.6    | The “Gold Standard” Upper Bound . . . . .                                    | 49        |
| 3.6      | Architectural Evolution . . . . .                                            | 51        |
| 3.6.1    | Generation 0: The Spatial Cropping Pitfall (Early Prototyping) . . . . .     | 51        |
| 3.6.2    | Generation 1 (G1): The Multi-Box Paradigm Shift . . . . .                    | 52        |

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| 3.6.3    | Generation 2 (G2): The Expanded Spatial Head (7.00M Parameters) . . | 53        |
| 3.6.4    | Generation 3 (G3): The High-Resolution Backbone (11.72M Parameters) | 54        |
| 3.6.5    | Generations: Summarised . . . . .                                   | 56        |
| 3.7      | Training Structure . . . . .                                        | 56        |
| 3.7.1    | Objective Function . . . . .                                        | 57        |
| 3.7.2    | Optimisation . . . . .                                              | 57        |
| 3.7.3    | Validation and Checkpointing . . . . .                              | 58        |
| 3.7.4    | Hyperparameter Summary . . . . .                                    | 58        |
| 3.8      | Experimental Setup and Benchmarking . . . . .                       | 59        |
| 3.8.1    | Dynamic Power Estimation . . . . .                                  | 59        |
| 3.8.2    | Quantisation Error Analysis (Accuracy vs. Object Size) . . . . .    | 61        |
| 3.8.3    | Confidence Calibration . . . . .                                    | 62        |
| 3.8.4    | Temporal Evidence Accumulation . . . . .                            | 63        |
| 3.8.5    | Real-Time Latency Benchmark . . . . .                               | 64        |
| 3.8.6    | Theoretical Energy Consumption . . . . .                            | 64        |
| 3.8.7    | Global Performance Evaluator . . . . .                              | 66        |
| <b>4</b> | <b>Results and Evaluation</b>                                       | <b>68</b> |
| 4.1      | Runtime Environment . . . . .                                       | 68        |
| 4.2      | Architectural Progression: Validation Evidence . . . . .            | 69        |
| 4.3      | Global Evaluator . . . . .                                          | 70        |
| 4.4      | Dynamic Power . . . . .                                             | 73        |
| 4.5      | Theoretical Energy Consumption . . . . .                            | 77        |
| 4.6      | Quantisation Error . . . . .                                        | 79        |
| 4.7      | Confidence Calibration . . . . .                                    | 81        |
| 4.8      | Temporal Evidence Accumulation . . . . .                            | 83        |
| 4.9      | Real-Time Latency Benchmark . . . . .                               | 85        |
| <b>5</b> | <b>Summary and Conclusions</b>                                      | <b>87</b> |
| 5.1      | Summary of Outcomes . . . . .                                       | 87        |
| 5.2      | Critical Reflection and Future Work . . . . .                       | 88        |
| 5.2.1    | Hardware-Paradigm Inequity: The LIF Simulation Penalty . . . . .    | 88        |
| 5.2.2    | The Temporal Quantisation Ceiling . . . . .                         | 89        |
| 5.2.3    | The Detection Head Mismatch . . . . .                               | 89        |
| 5.2.4    | Dataset Domain Gap and Strategic Selection . . . . .                | 90        |
| <b>A</b> | <b>Implementation Specifications</b>                                | <b>97</b> |
| A.1      | DetectorNX-G3-SNN Hyperparameters . . . . .                         | 97        |
| A.2      | DetectorNX-G3-CNN Hyperparameters . . . . .                         | 99        |
| A.3      | Qualitative Audit Samples . . . . .                                 | 100       |

**Word Count: 17888**

# List of Tables

|     |                                                                                                                                                                                                                                                                                                                                                                                       |    |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | Success criteria traceability table. . . . .                                                                                                                                                                                                                                                                                                                                          | 33 |
| 3.2 | Network Architecture Stages . . . . .                                                                                                                                                                                                                                                                                                                                                 | 41 |
| 3.3 | 1:1 Architectural Mapping (SNN vs. CNN) demonstrating strict structural parity. Both models utilise an identical parameter budget (11.7M) and channel progression, differing only in the temporal dimension and activation functions (LIF vs. ReLU). . . . .                                                                                                                          | 47 |
| 3.4 | Identity Mapping Comparison . . . . .                                                                                                                                                                                                                                                                                                                                                 | 48 |
| 3.5 | Generational evolution of the <code>DetectorNX</code> architecture. . . . .                                                                                                                                                                                                                                                                                                           | 56 |
| 3.6 | Training configuration for <code>DetectorNX-G3-SNN</code> . . . . .                                                                                                                                                                                                                                                                                                                   | 58 |
| 3.7 | Energy constants per operation ( $E_{chip}$ ) for each architecture and platform. . . .                                                                                                                                                                                                                                                                                               | 65 |
| 4.1 | Architectural-progression validation evidence. . . . .                                                                                                                                                                                                                                                                                                                                | 69 |
| 4.2 | Global Performance Benchmark results for <code>DetectorNX-G3-SNN</code> and <code>DetectorNX-G3-CNN</code> , evaluated across the full 200-sample ETraM validation set [42]. Ground truth average detections per frame is 2.02 for both models by definition. . . . .                                                                                                                 | 70 |
| 4.3 | Layer-Wise Spike Activation Rates . . . . .                                                                                                                                                                                                                                                                                                                                           | 74 |
| 4.4 | Energy Statistics per Frame . . . . .                                                                                                                                                                                                                                                                                                                                                 | 74 |
| 4.5 | Layer-Wise Operations Comparison . . . . .                                                                                                                                                                                                                                                                                                                                            | 77 |
| 4.6 | Neuromorphic Hardware Energy Extrapolation . . . . .                                                                                                                                                                                                                                                                                                                                  | 77 |
| 4.7 | Comparative efficiency landscape of related SNN detectors. . . . .                                                                                                                                                                                                                                                                                                                    | 79 |
| 4.8 | Quantisation Error Analysis showing the absolute and percentage drop in Spiking Neural Network (SNN) Mean Intersection Over Union (mIoU) performance against the Convolutional Neural Network (CNN) baseline. The validation set is partitioned into Small, Medium, and Large objects based on pixel area to evaluate the scale-dependency of the 1-bit quantisation penalty. . . . . | 80 |
| 4.9 | Inference Latency Benchmark . . . . .                                                                                                                                                                                                                                                                                                                                                 | 85 |
| 5.1 | Success criteria validation outcomes. . . . .                                                                                                                                                                                                                                                                                                                                         | 88 |
| A.1 | <code>DetectorNX-G3-SNN</code> hyperparameters. . . . .                                                                                                                                                                                                                                                                                                                               | 97 |
| A.2 | <code>DetectorNX-G3-CNN</code> hyperparameters. . . . .                                                                                                                                                                                                                                                                                                                               | 99 |

# List of Figures

|     |                                                                                                                                                                                                                                                                                                                              |    |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Hierarchical feature extraction in a CNN . . . . .                                                                                                                                                                                                                                                                           | 20 |
| 2.2 | The residual learning building block. . . . .                                                                                                                                                                                                                                                                                | 21 |
| 2.3 | Leaky integrate-and-fire neuron dynamics. . . . .                                                                                                                                                                                                                                                                            | 22 |
| 2.4 | The surrogate gradient method. . . . .                                                                                                                                                                                                                                                                                       | 24 |
| 2.5 | Frame-based vs event-based output on the same urban scene. . . . .                                                                                                                                                                                                                                                           | 25 |
| 2.6 | Sample frames from the ETraM dataset. . . . .                                                                                                                                                                                                                                                                                | 26 |
| 3.1 | High-level system architecture pipeline. . . . .                                                                                                                                                                                                                                                                             | 32 |
| 3.2 | Overview of the spatiotemporal preprocessing pipeline applied to the ETraM dataset. Raw events are voxelised into a 4D tensor, spatially downsampled, and aligned with multi-object ground truth labels to produce a final sample of shape $[10, 2, 180, 320]$ with a corresponding label tensor of shape $[5, 5]$ . . . . . | 39 |
| 3.3 | The Spike-Element-Wise (SEW) residual block (SEWResBlock) as proposed by Fang et al. [12]. . . . .                                                                                                                                                                                                                           | 42 |
| 3.4 | The Squeeze-and-Excitation (SE) block [20] as applied in Stage 3 of the <code>DetectorNX-G3-SNN</code> backbone ( $22 \times 40 \times 256$ ). . . . .                                                                                                                                                                       | 44 |
| 3.5 | End-to-end <code>DetectorNX-G3-SNN</code> architecture. . . . .                                                                                                                                                                                                                                                              | 46 |
| 3.6 | End-to-end <code>DetectorNX-G3-CNN</code> architecture. . . . .                                                                                                                                                                                                                                                              | 50 |
| 3.7 | Constant Target Bias failure mode . . . . .                                                                                                                                                                                                                                                                                  | 52 |
| 3.8 | Generation 1 Super-Box Failure . . . . .                                                                                                                                                                                                                                                                                     | 53 |
| 4.1 | Validation mIoU progression across architecture generations. . . . .                                                                                                                                                                                                                                                         | 69 |
| 4.2 | Qualitative Inference Audit . . . . .                                                                                                                                                                                                                                                                                        | 72 |
| 4.3 | Dynamic Power Consumption vs. Scene Activity . . . . .                                                                                                                                                                                                                                                                       | 76 |
| 4.4 | Quantisation Error Analysis (Accuracy vs. Object Size) of SNN (Blue) and CNN (Orange). The results invalidate the expected scale-dependent penalty, showing instead that the $T = 10$ temporal resolution imposes a global geometric drag across all object sizes. . . . .                                                   | 80 |
| 4.5 | Confidence Calibration Reliability Diagrams . . . . .                                                                                                                                                                                                                                                                        | 81 |
| 4.6 | Temporal Evidence Accumulation Sequences . . . . .                                                                                                                                                                                                                                                                           | 84 |

# Abstract

Legislation is pushing automotive manufacturers to adopt increasingly complex autonomous driving systems, on a trajectory that will ultimately require full, real-time spatial understanding of the surrounding environment. As a result, detection pipelines have grown substantially in complexity and compute demand, while vehicles remain subject to tight on-board power and thermal budgets. These trends suggest that scaling conventional dense DNNs on GPU-class accelerators is increasingly unsustainable, motivating ultra-efficient architectures such as spiking neural networks for tasks in resource-constrained environments.

We investigate the viability of spiking networks for automotive-grade perception, and measure the cost of the spiking paradigm against a continuous equivalent. We propose DetectorNX: a directly-trained spiking detector (DetectorNX-G3-SNN) designed around activation sparsity to minimise energy consumption while preserving detection accuracy, paired with a structurally equivalent continuous baseline (DetectorNX-G3-CNN) at identical parameter count, trained on the ETraM event-camera dataset to isolate the spiking mechanism as the sole variable. To the best of our knowledge, this is the first directly-trained SNN developed and evaluated on ETraM.

DetectorNX-G3-SNN achieves 94.9% mIoU parity and 91.3%  $mAP_{50}$  parity against the continuous baseline, with 100% recall at IoU  $> 0.5$ . Per-frame theoretical energy consumption drops by 94.33%, scaling linearly with scene activity ( $R^2 = 0.7514$ ). The SNN maintains 259.80 FPS on GPU hardware ( $8.6\times$  the automotive real-time threshold) with superior confidence calibration at low-confidence regions.

This project establishes directly-trained SNNs as a viable route to energy-efficient automotive-grade vehicle detection, at a cost of only  $\sim 5\%$  mIoU precision under identical-model SNN–CNN parity. The 1-bit quantisation penalty underlying this gap acts as a scale-independent global ceiling, invalidating the initial scale-dependent hypothesis. Superior low-confidence calibration further positions the SNN as an implicit uncertainty sensor, safety-relevant for autonomous driving. DetectorNX thereby serves as an ETraM baseline, and proposes areas for future work to close the gap with published broader-class spiking detectors.



# Declaration

No portion of the work referred to in this report has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

# Copyright

- i. The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the “Copyright”) and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.
- ii. Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made **only** in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.
- iii. The ownership of certain Copyright, patents, designs, trade marks and other intellectual property (the “Intellectual Property”) and any reproductions of copyright works in the thesis, for example graphs and tables (“Reproductions”), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.
- iv. Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see <http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420>), in any relevant Thesis restriction declarations deposited in the University Library, The University Library’s regulations (see <http://www.library.manchester.ac.uk/about/regulations/>) and in The University’s policy on presentation of Theses

# Acknowledgements

Firstly, I would like to give thanks to my supervisor, Dr Oliver Rhodes, for his guidance and critical feedback throughout the duration of this project, but also for supporting and motivating me to reach the conclusion of this project and degree to the best of my ability. His advice has proved crucial to aspects of this project and report.

Secondly, I want to express my gratitude to my family — namely my parents, Jane and Kanthan, for their endless support and ultimately leading me to this position.

Finally, I would like to thank my friends: Denitta, Gajen, Aden, Neil, Felix, Toby, Tom, and Prav for their support and advice throughout this degree — it has been invaluable.

# List of Abbreviations

**AC** Conditional Addition.

**ADAS** Advanced Driver Assistance Systems.

**ADP** Average Detections Per Frame.

**AMP** Automatic Mixed Precision.

**ANN** Artificial Neural Network.

**ATan** ArcTangent.

**BAF** Background Activity Filtering.

**BOLT** Box-aligned Offline Label-aware Tensorisation.

**BPTT** Backpropagation Through Time.

**CMOS** Complementary Metal-Oxide-Semiconductor.

**CNN** Convolutional Neural Network.

**CPU** Central Processing Unit.

**CUDA** Compute Unified Device Architecture.

**ECE** Expected Calibration Error.

**ETraM** Event-based Traffic Monitoring Dataset.

**FLOP** Floating-Point Operation.

**FPS** Frames Per Second.

**FRR** Firing Rate Regularisation.

**GIoU** Generalised Intersection over Union.

**GPU** Graphics Processing Unit.

**HD** High Definition.

**HPC** High Performance Computing.

**IoU** Intersection over Union.

**LIF** Leaky Integrate-and-Fire.

**LRU** Least Recently Used.

**MAC** Multiply-Accumulate.

**mAP<sub>50</sub>** mean Average Precision at 0.5 IoU threshold.

**mIoU** Mean Intersection Over Union.

**ReLU** Rectified Linear Unit.

**ResNet** Residual Neural Network.

**ROC** Rank Order Coding.

**SE** Squeeze-and-Excitation.

**SEW** Spike-Element-Wise.

**SNN** Spiking Neural Network.

**SNR** Signal-to-Noise Ratio.

**SOP** Synaptic Operation.

**TTFS** Time-to-First-Spike.

**WCET** Worst-Case Execution Time.

# Chapter 1

## Introduction

### 1.1 Overview

Autonomous perception is one of the most critical frontiers in automotive engineering today, driven by both legislation and consumer demand to adopt increasingly complex autonomous driving systems, which will no doubt lead to systems requiring real-time spatial understanding of the surrounding environment. Euro NCAP (Europe’s vehicle safety rating programme) has published 2026 protocols, for example, that require advanced Autonomous Emergency Braking (AEB) to handle increasingly complex scenarios, such as blind junctions, and a concept shift from reactive to predictive intervention [9]. It also introduces an all-new requirement for these more complex Advanced Driver Assistance Systems (ADAS) to operate at speeds up to 130 km/h, which will require lower inference latencies [10].

Modern vehicles rely on dense Artificial Neural Networks (ANNs) to process high-definition video streams in real-time, executing billions of Floating-Point Operations (FLOPs) every second. As a result, scaling such systems to handle more complex scenarios has caused both energy and thermal demands to grow substantially. Lin et al. [26] documented CPU-based CNN inference exceeding the thermal and power constraints of ADAS hardware, motivating the industry’s shift to Graphics Processing Unit (GPU)-class accelerators; continued growth in perception complexity now places dense Artificial Neural Network (ANN) workloads on GPUs on a similar trajectory, motivating a paradigm shift towards fundamentally more efficient architectures.

This project investigates utilising a cutting-edge neuromorphic paradigm, Spiking Neural Networks (SNNs), and their feasibility in this domain. To achieve this, the project develops `DetectorNX-G3-SNN`, a high-definition spiking architecture designed for multi-object automotive perception. While recent architectures such as SpikeDet [11], SpikeYOLO [27], and EMS-YOLO [37] have advanced SNN detection on general-purpose benchmarks, their applicability and efficiency trade-offs in high-definition automotive environments remain critically underexplored. By rigorously benchmarking this system against a strictly equivalent continuous CNN on the event-based Event-based Traffic Monitoring Dataset (ETraM) dataset [42],

this research empirically deconstructs the performance-efficiency frontier—measuring the precise trade-offs between the energy savings of event-driven dynamics and the spatial precision constraints of 1-bit temporal quantisation.

## 1.2 Motivation

Legislation is pushing automotive manufacturers to adopt increasingly complex autonomous driving systems, on a trajectory that will ultimately require full, real-time spatial understanding of the surrounding environment. Beyond this scaling pressure, the dense Artificial Neural Networks (ANNs) currently powering automotive vision suffer a paradigm-specific weakness: their static, frame-based nature results in billions of dense Multiply-Accumulate (MAC) operations per frame regardless of scene content, generating substantial thermal, latency and power pressure even on visually redundant scenes.

To resolve these constraints, this research looks toward the most power-efficient perception engine known: the biological brain. Unlike frame-based cameras, biological retinas and neuromorphic event sensors transmit only asynchronous luminance changes, allowing biological neural networks to consume metabolic energy only when processing new information [30].

Spiking Neural Networks (SNNs) are built around this same principle of activation sparsity. By replacing continuous MAC operations with discrete, event-driven Conditional Addition (AC) additions, they offer a theoretical pathway to sub-milliwatt, real-time object detection at the edge. Since spiking neurons remain dormant until triggered, they naturally sidestep the thermal and power pressure that dense CNN inference places on resource-constrained automotive hardware [26].

However, the transition from 32-bit floating-point activations to 1-bit discrete spikes introduces quantisation errors and temporal precision limits. Historically, this has restricted SNNs to low-resolution classification tasks, leaving a significant gap in their deployment for high-definition automotive perception.

The `DetectorNX` project is motivated by the need to measure and mitigate these trade-offs on modern, high-definition event datasets. By establishing strict architectural parity between an SNN and a CNN baseline, this project determines whether the neuromorphic paradigm’s thermal and power savings can be achieved without fatally compromising the spatial precision required for autonomous vehicle safety.

## 1.3 Project Aims and Objectives

To address these challenges, this project pursues a single, well-defined research question: can a SNN, constrained to the same architectural capacity as a continuous CNN, achieve competitive object detection accuracy on high-definition event-based data while delivering meaningful energy

efficiency gains? To answer this, the following objectives were established:

1. **Develop a high-definition SNN for automotive multi-object detection:** Design and train `DetectorNX-G3-SNN`, a spiking architecture capable of performing bounding box regression on the High Definition (HD) Event-based Traffic Monitoring Dataset (ETraM) event-camera dataset.
2. **Develop a structurally equivalent CNN baseline:** Construct `DetectorNX-G3-CNN`, a continuous-valued counterpart that mirrors the SNN architecture layer-for-layer with an identical parameter budget, isolating the spiking mechanism as the sole independent variable in all subsequent experiments and evaluations.
3. **Train and evaluate both models under identical conditions:** Apply the same pre-processed dataset, loss function, optimiser, and training duration to both models, ensuring that any observed performance differences are attributable solely to the computational paradigm.
4. **Benchmark inference latency on GPU hardware:** Measure the real-time throughput of both architectures to assess their viability for deployment in automotive perception pipelines.
5. **Estimate theoretical energy consumption:** Derive energy usage from layer-wise spike activity using established neuromorphic hardware constants, and project consumption figures onto representative neuromorphic platforms such as Intel Loihi 2 and IBM TrueNorth.
6. **Conduct a rigorous, multi-dimensional comparison:** Synthesise the accuracy, latency, and energy results to empirically characterise the performance-efficiency frontier of the spiking paradigm on a real-world, high-definition detection task.

## 1.4 Success Criteria

To provide a concrete evaluation framework, the following measurable success criteria were defined prior to experimentation. Each criterion is revisited explicitly in Chapter 5.

1. **Detection accuracy parity:** On both mIoU and mean Average Precision at 0.5 IoU threshold ( $\text{mAP}_{50}$ ), the SNN must score at least 90% of the CNN baseline on the ETraM validation set. This 90% threshold is established as a rigorous but realistic target, as achieving exact parity remains an open challenge in deep spiking vision [21]. Achieving this will validate that our architecture succeeds in mitigating the compromises expected from the 1-bit temporal quantisation error, across both the geometric-precision metric (mIoU) and the stricter COCO-convention detection-quality metric ( $\text{mAP}_{50}$ ).



2. **Measurable theoretical energy savings:** The SNN must demonstrate a reduction in theoretical energy consumption per inference relative to the CNN baseline, computed from layer-wise Synaptic Operation (SOP) counts and published neuromorphic hardware constants.
3. **Real-time viability:** Both models must sustain an inference throughput of at least 30 frames per second on GPU hardware, meeting the established minimum threshold for real-time automotive perception [23].
4. **Architectural parity:** The SNN and CNN must share an equivalent parameter count (within 1%), ensuring that any observed performance gap reflects the spiking mechanism rather than differences in model capacity.
5. **Detection completeness:** Both models must achieve a recall of 100% for objects with an Intersection over Union (IoU) exceeding 0.5 on the validation set, confirming that the multi-box regression head reliably detects all ground-truth vehicles in the scene.
6. **Confidence calibration:** The SNN must exhibit confidence calibration equivalent to, or superior to, that of the CNN baseline on the ETraM validation set. This criterion is safety-critical: in ADAS applications, a model that systematically over-states its detection confidence may suppress downstream safety interventions. A neuromorphic architecture that delivers energy savings at the cost of increased over-confidence would therefore be unsuitable for deployment, regardless of its accuracy or efficiency gains.

## 1.5 Evaluation Approach

Evaluation is designed to isolate the effect of the spiking mechanism as precisely as possible. Because `DetectorNX-G3-SNN` and `DetectorNX-G3-CNN` are parameter-matched and trained under strictly identical conditions, any observed differences can be attributed to the computational paradigm rather than to capacity or training asymmetries. All experiments are conducted on a validation split of the ETraM dataset, built using the stratified sampling strategy described in Section 4.1, to ensure that day, night, rain, and clear-weather scenes are proportionally represented.

Two complementary dimensions are measured. *Prediction Performance* captures how accurately each model localises vehicles in the event stream, using IoU,  $mAP_{50}$ , recall at a fixed overlap threshold, and Expected Calibration Error (ECE)-based confidence calibration. *Model Performance* captures deployment efficiency, using inference latency on GPU hardware against the 30 Frames Per Second (FPS) real-time threshold [23], and a theoretical energy model projected onto neuromorphic targets. Detailed formulations, hypotheses, and the analytical energy framework are presented with their experiments in Section 3.8.

## 1.6 Report Structure

The report is structured to first establish the necessary theoretical foundation, before detailing the architecture and its empirical evaluation:

**Background:** Chapter 2 provides the prerequisite theory for the `DetectorNX` project. It establishes CNN and SNN fundamentals, event-based vision, and neuromorphic energy economics, before concluding with related work to define the critical gap in high-definition automotive spiking detection that this project addresses.

**Methodology and Evaluation:** Chapter 3 details the technical implementation and its results. By integrating the architectural design (`DetectorNX-G3-SNN` and `DetectorNX-G3-CNN`) with the experimental setup and evaluation within a single chapter, the report directly links architectural choices—such as the shared multi-box head and BOLT pipeline—to their empirical outcomes across accuracy, latency, and energy dimensions.

**Conclusions:** Chapter 5 closes the report by evaluating the overall project against the success criteria defined in Section 1.4. It synthesises the principal findings, reflects critically on experimental limitations, and outlines future work, such as physical neuromorphic deployment and alternative spike encodings.

# Chapter 2

## Background

### 2.1 Artificial Neural Networks and Differentiability

ANNs are a family of computational models that learn hierarchical feature representations by progressively mapping input data through layers of weighted, non-linear transformations [25]. Training an ANN relies entirely on minimising a loss function via gradient descent, where error signals are propagated backward through the network using the chain rule of calculus (backpropagation).

A fundamental, architectural constraint of this process is that every operation in the computational graph must be continuous and differentiable; if a non-differentiable operation is encountered, the chain rule breaks down and the gradient signal is zeroed. This constraint is easily satisfied in standard ANNs by using continuous activation functions like Rectified Linear Unit (ReLU), which are differentiable almost everywhere. However, as will be established in Section 2.3, this continuous requirement becomes the primary mathematical obstacle when transitioning to the discrete, event-driven paradigm of SNNs.

### 2.2 Convolutional Neural Networks for Object Detection

#### 2.2.1 Convolutional Feature Extraction

A Convolutional Neural Network (CNN) extends the standard ANN architecture by replacing dense linear layers with convolutional operations, in which a bank of learnt filters is applied across the spatial extent of an input. Each filter performs a localised weighted sum at every spatial position, producing a feature map that encodes the presence and magnitude of a particular learnt pattern at each location. Stacking these layers yields a representational hierarchy, with early layers capturing low-level edges and motion boundaries and deeper layers composing them into object-level features LeCun, Bengio and Hinton [25].

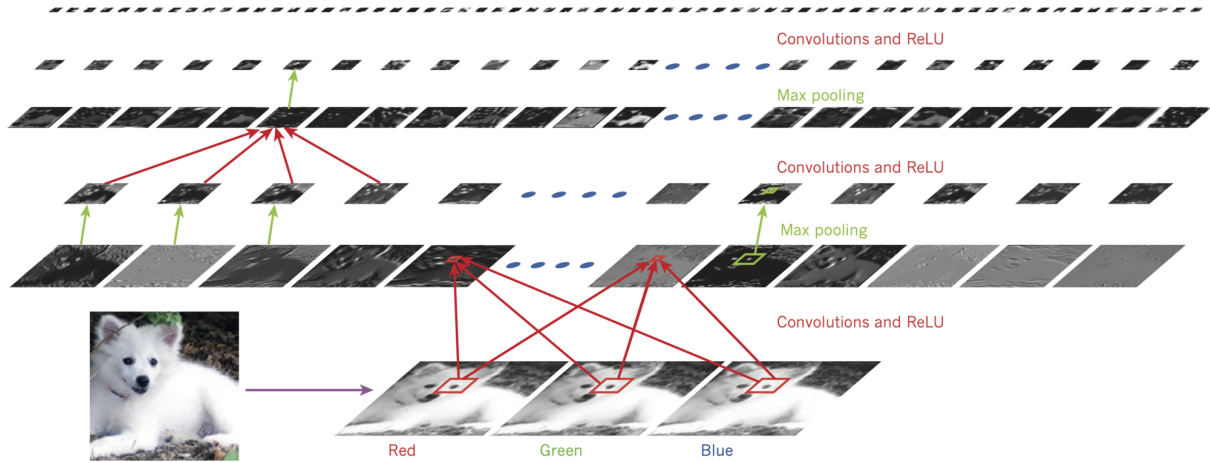


Figure 2.1: Hierarchical feature extraction in a CNN. Filters are convolved across the input’s spatial extent, producing feature maps that are progressively pooled and recombined. Early layers respond to low-level structure such as edges, while deeper layers compose these into increasingly abstract, object-level representations. Reproduced from LeCun, Bengio and Hinton [25].

### 2.2.2 Residual Learning

A well-documented failure mode of deep CNNs is the *vanishing gradient* problem: as back-propagation proceeds through many successive layers, gradients are repeatedly scaled by values less than one, causing them to decay exponentially and preventing effective learning in early layers [18]. He et al. [18] resolved this through the introduction of *residual learning*: rather than tasking each layer with learning a direct mapping  $y = \mathcal{H}(x)$ , the network instead learns the *residual*  $\mathcal{F}(x) = \mathcal{H}(x) - x$ , recasting the target as:

$$y = \mathcal{F}(x) + x \quad (2.1)$$

The additive shortcut connection, otherwise known as a skip connection, bypasses one or more convolutional layers and adds the input directly to the output, providing an unobstructed gradient path back to the earliest layers. Within a Standard Residual Block, this shortcut carries the raw input features forward through the identity path, and the convolutional branch learns only the incremental refinement needed to improve the representation. This mechanism forms the backbone of the CNN baseline employed in this project. In a spiking network, however, the standard residual formulation fails, since naively summing two binary spikes produces a non-binary value; a variant that preserves binary output therefore underpins the `DetectorNX-G3-SNN` architecture, as detailed in Section 3.4.3.

### 2.2.3 Object Detection and Bounding Box Regression

Object detection requires a network to simultaneously localise and identify one or more targets within a scene, and determine what their coordinates and size in the space are. In the bounding

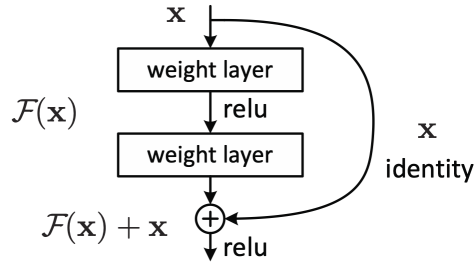


Figure 2.2: The standard residual block. The convolutional branch learns the residual mapping  $\mathcal{F}(x)$ , while an identity shortcut adds the unmodified input  $x$  back at the summation node, producing  $\mathcal{F}(x) + x$ . The shortcut provides an unobstructed gradient path through the block, mitigating vanishing gradients in deep networks. Reproduced from He et al. [18].

box regression paradigm popularised by Redmon et al. [31], this is framed as a direct regression problem: the network predicts a set of candidate boxes, each parameterised by a feature tuple  $\mathcal{F} = \{x, y, w, h, c\}$ , where  $(x, y)$  and  $(w, h)$  are the normalised centre coordinates and dimensions of the box, and  $c$  is an objectness confidence score reflecting the probability that a genuine target is present within that region.

By predicting multiple candidate boxes simultaneously in a single forward pass, this multi-box formulation enables real-time detection across the full spatial extent of the input – a necessity for automotive perception, where multiple vehicles and artefacts may coexist in the same scene. Each predicted box is evaluated against the ground-truth annotations using an Intersection over Union (IoU) metric, which quantifies the ratio of the intersection to the union of the two boxes, providing a scale-invariant, directly interpretable measure of spatial overlap. The multi-box regression paradigm, including the  $(x, y, w, h, c)$  output format and the use of IoU as the evaluation criterion, forms the direct foundation of the detection head employed in this project, as detailed in Section 3.4.5.

## 2.3 Spiking Neural Networks

SNNs represent a fundamentally different computational paradigm to the standard ANN architectures described in the preceding sections. Rather than propagating continuous-valued activations between layers, SNNs communicate exclusively through discrete binary events, spikes, mimicking the all-or-nothing firing behaviour of biological neurons [33]. By prioritising event-driven binary communication, this paradigm aims to drastically reduce both dynamic power consumption and computational complexity compared to traditional deep learning models, making it inherently suited to power and latency-constrained environments.

### 2.3.1 The Leaky Integrate-and-Fire Neuron

At the foundation of most practical SNNs lies the Leaky Integrate-and-Fire (LIF) neuron model, which governs the temporal dynamics and binary spike generation of the network. Unlike

standard ANNs that utilise continuous activation functions such as ReLU, the LIF node maintains an internal state: the membrane potential  $V[t]$ , which updates iteratively across discrete timesteps  $t \in [1, T]$ .

The dynamics of the LIF neuron at a given spatial position and timestep  $t$  are governed by the integration of incoming pre-synaptic spatial features  $X[t]$  and an inherent leak factor  $\tau$ , which dictates how quickly  $V[t]$  returns to its resting state in absence of incoming input. The hidden membrane potential  $H[t]$  prior to spiking is mathematically defined as:

$$H[t] = V[t-1] + \frac{1}{\tau} (X[t] - (V[t-1] - V_{reset})) \quad (2.2)$$

When  $H[t]$  exceeds a predefined threshold  $V_{th}$ , the neuron emits a binary spike  $S[t] \in \{0, 1\}$  to the subsequent layer, governed by the Heaviside step function  $\Theta(x)$ :

$$S[t] = \Theta(H[t] - V_{th}) \quad (2.3)$$

Following a spike ( $S[t] = 1$ ), the neuron undergoes a hard reset, instantly dropping its potential  $V[t]$  to  $V_{reset}$ , mimicking the refractory period of biological neurons, preventing infinite spiking and overstimulation [34]; the complete integrate-fire-reset cycle is illustrated in Figure 2.3.

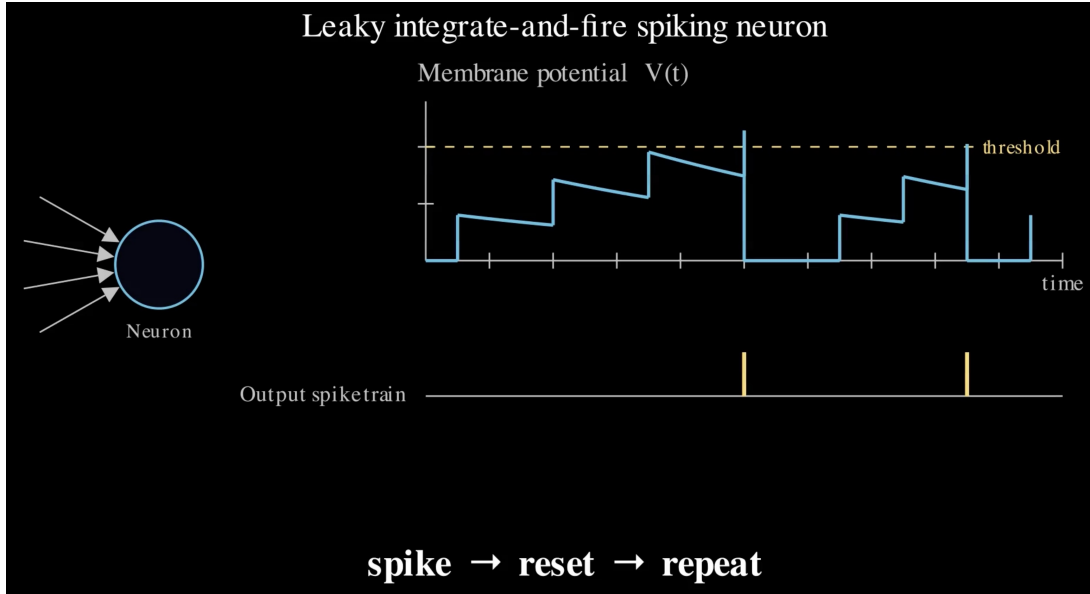


Figure 2.3: The leaky integrate-and-fire neuron over time. *Left*: the neuron receives pre-synaptic spikes through multiple input synapses. *Right (upper)*: the membrane potential trace  $V(t)$  accumulates each input contribution and decays toward its resting state at a rate set by the leak constant  $\tau$  (Equation 2.2). When  $V(t)$  crosses the firing threshold (dashed yellow), the neuron emits a spike (Equation 2.3) and the potential is hard-reset toward  $V_{reset}$  before the cycle repeats. *Right (lower)*: the output spike train, with each yellow tick marking the discrete moments at which the threshold was crossed. The neuron consumes computation only at these firing moments, illustrating the activation-sparsity property responsible for the SNN’s energy efficiency.

If no spike is emitted ( $S[t] = 0$ ), the potential naturally decays, ensuring that subsequent layers only perform computations upon receiving active events [35]. This *activation sparsity*, the property that computation is triggered only by genuine signal, not by silence, is the primary driver of the SNN’s exceptional energy efficiency [33].

### 2.3.2 Surrogate Gradients

Training deep SNNs via backpropagation presents a fundamental mathematical challenge, since the Heaviside step function governing spike generation is non-differentiable. As established in Section 2.1, the chain rule requires every function in the computational graph to be differentiable; the Heaviside function violates this directly, since its derivative is zero everywhere except at the threshold  $V_{th}$ , where it is infinite. Using conventional backpropagation therefore results in zeroed gradients that immediately halt the learning process, a phenomenon known as the “dead neuron” problem.

To resolve this, SNNs employ a *surrogate gradient* during the backward pass [29]. While the forward pass remains strictly binary, the backward pass approximates the step function using a smooth, differentiable curve. A common choice is the ArcTangent (ATan) function:

$$\frac{\partial S}{\partial H} \approx \frac{\alpha}{2 \left( 1 + \left( \frac{\pi}{2} \alpha (H - V_{th}) \right)^2 \right)} \quad (2.4)$$

where  $\alpha$  is a hyperparameter controlling the steepness of the surrogate curve, as visualised in Figure 2.4.

This mathematical substitution allows the network to learn complex spatiotemporal representations from scratch while maintaining a purely binary forward execution, with gradients propagated through the temporal dimension via Backpropagation Through Time (BPTT).

### 2.3.3 Temporal Dynamics and Rapid Categorisation

A defining property of SNNs is their natural ability for spatiotemporal integration, combining timing and location when processing information. By processing an event stream iteratively across  $T$  discrete timesteps, an SNN is capable of producing meaningful predictions before the full observation window has elapsed [30].

This progressive refinement reflects the *rapid categorisation* phenomenon documented in the human visual system, where early feedforward spikes drive coarse semantic decisions that sharpen as further neural evidence accumulates [39]. While Thorpe, Fize and Marlot [39] address categorical recognition rather than coordinate regression, the underlying principle that sparse early spike activity produces meaningful but imprecise representations which sharpen over time is directly consistent with the anticipated behaviour of a spiking detection backbone.

Rate coding over a fixed window of  $T$  timesteps carries an inherent representational limit

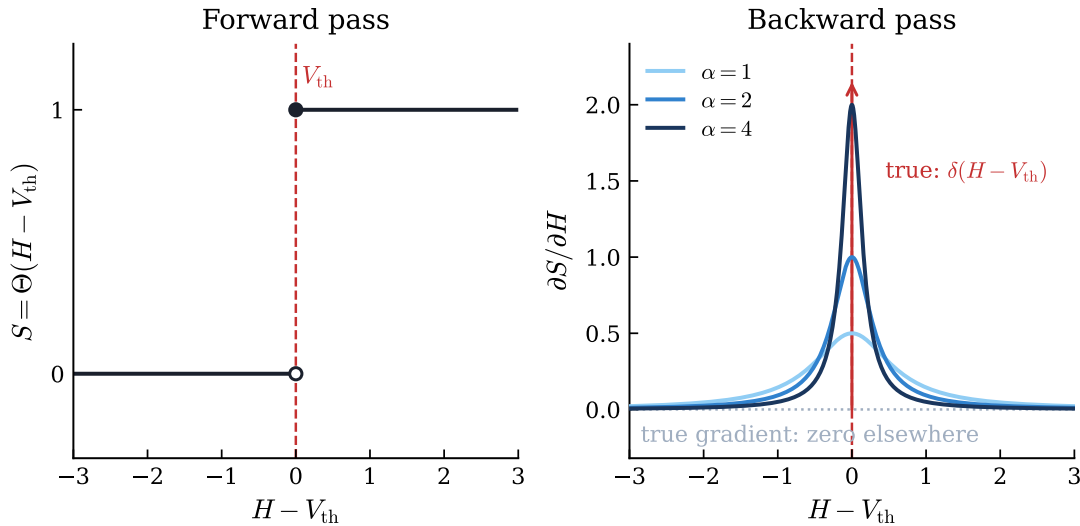


Figure 2.4: The surrogate gradient method. *Left*: the Heaviside step function  $S = \Theta(H - V_{th})$  governing spike emission in the forward pass. *Right*: its true derivative is the Dirac delta  $\delta(H - V_{th})$ , which is zero everywhere except at the threshold and therefore unusable for backpropagation. During the backward pass, the true derivative is replaced by a smooth atan surrogate (Equation 2.4); the steepness hyperparameter  $\alpha$  controls how closely the surrogate approximates the true delta. This substitution allows gradients to flow through the otherwise-blocking spike non-linearity while preserving the strictly binary forward execution.

known as the “quantisation error”; the precision of the encoded signal is bounded by a  $1/T$  quantisation floor, since information is encoded as spike counts that can take only  $T + 1$  discrete values [38].

## 2.4 Event-Based Vision

### 2.4.1 Event Cameras

#### Frame-Based vs. Event-Based Sensing

Conventional frame-based cameras operate synchronously, where every pixel is read out at a fixed rate, producing a dense intensity matrix regardless of whether anything in the scene has changed since the last frame. This design introduces several well-documented limitations: motion blur at high speeds, a fixed dynamic range bounded by the exposure window, and substantial data redundancy during static scenes — all of which are particularly problematic for high-speed automotive perception [14].

An event camera fundamentally departs from this paradigm; rather than reading out all pixels simultaneously, each pixel operates independently, firing an event the moment its log-intensity changes beyond a threshold. The result is a continuous, asynchronous stream of discrete events rather than a sequence of synchronous frames, registering only what has changed between scenes instead of the full scenes themselves (Figure 2.5).



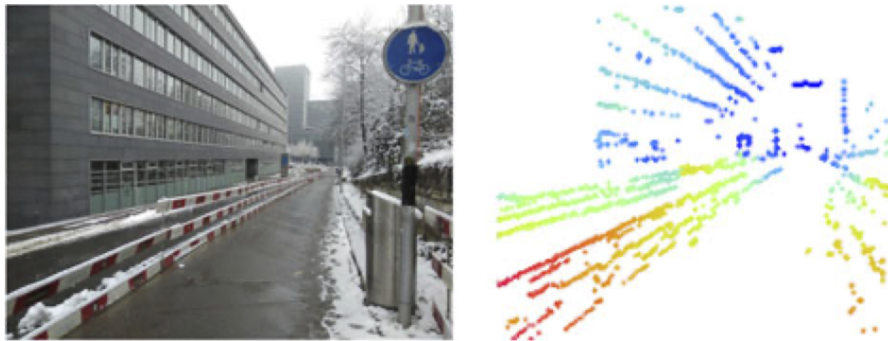


Figure 2.5: Comparison of (a) a frame-based RGB image (left) and (b) the corresponding event-camera output of the same urban scene (right), rendered as a spatio-temporal point cloud. The event camera only registers the locations of intensity changes, producing a sparse representation that ignores the static building visible in (a). Reproduced from Gallego et al. [14].

### Advantages and the Compatibility Problem

This per-pixel, threshold-driven design gives event cameras a compelling profile for autonomous perception: high temporal resolution, high dynamic range, low latency, and low data redundancy [14]. However, a sparse, unordered point cloud of  $(x, y, t, p)$  tuples is fundamentally incompatible with conventional deep learning frameworks, which expect dense, structured tensors of fixed spatial dimensions. Correcting this to adapt to deep learning approaches requires a preprocessing step to convert the asynchronous stream into a format suitable for neural network ingestion, the specific design of which is detailed in Section 3.3.

## 2.4.2 The Natural SNN–Event Camera Pairing

### An Event-Driven Architecture for an Event-Driven Sensor

Both systems are fundamentally event-driven: both remain silent in the absence of stimuli and emit discrete binary events only in response to genuine activity.

A CNN must first collapse the event stream into a single dense synchronous tensor before it can be processed, discarding all temporal structure. In contrast, an SNN processes the preprocessed spatiotemporal tensor iteratively across  $T$  timesteps through its LIF dynamics, preserving the event-driven dynamics [14, 30]. An SNN can therefore perform early-exit inference, emitting predictions before the full  $T$ -step window has elapsed, whereas a CNN must wait for its complete aggregated tensor.

### Implications for Neuromorphic Deployment

On neuromorphic hardware such as Intel Loihi 2 [16] or IBM TrueNorth [28], this alignment extends to the physical level. An event camera connected directly to a neuromorphic chip can route events to spiking neurons without any voxelisation or dense tensor conversion, since the raw events can be routed directly to spiking neurons without any intermediate tensor representation,

enabling a fully asynchronous end-to-end inference pipeline and eliminating the preprocessing bottleneck entirely.

## 2.5 ETraM Dataset

### 2.5.1 Dataset Overview and Acquisition Setup

ETraM is a first-of-its-kind, fully event-based traffic monitoring dataset introduced by Verma et al. [42] at CVPR 2024. Data was captured using the Prophesee EVK4 HD event camera at  $1280 \times 720$  spatial resolution with a temporal resolution exceeding 10,000 fps, mounted statically at approximately 6 m height with a  $35^\circ$  pitch angle. Unlike ego-motion datasets, where the camera itself moves through the scene, this dataset provides a static perspective; a fixed vantage point ensures that events are generated predominantly by moving traffic participants rather than background infrastructure. Recordings span intersections, roadways, and local streets around Arizona State University, Tempe campus, totalling ten hours of annotated event data.

### 2.5.2 Class Taxonomy, Scale and Annotation

ETraM covers eight distinct object classes organised into three broad categories: Vehicles (Car, Truck, Bus, Tram), Pedestrians, and Micro-mobility (Bicycle, Bike, Wheelchair). Over 2M bounding boxes were manually annotated using CVAT [42], with each annotation additionally carrying an object ID to support multi-object tracking. Raw data is distributed in multiple formats (RAW, DAT, and H5), with ground-truth annotations provided in NumPy format. For this project, the dataset was abstracted to the Vehicle category exclusively, allowing the SNN–CNN comparison to be isolated on a single, well-defined object class. Figure 2.6 shows representative sample frames demonstrating both the visual character of the event-camera output and the multi-class bounding-box annotation scheme.

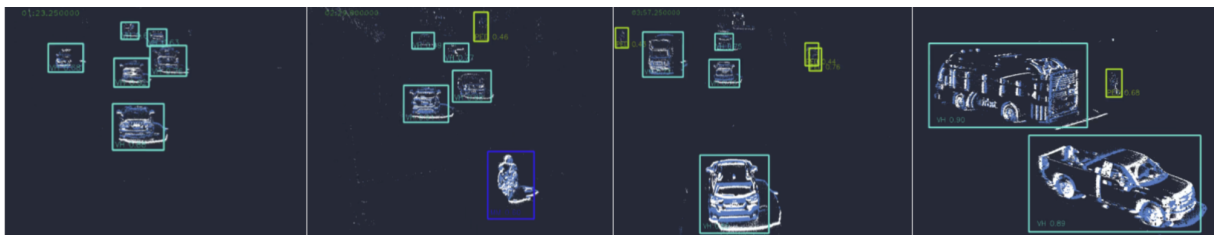


Figure 2.6: Sample frames from the ETraM dataset, showing the sparse white-on-dark visual character of event-camera output, with only changing events being registered by the sensor. Objects are annotated using boxes and colours to separate classes, with confidence scores from the original authors’ CNN detector. Reproduced from Verma et al. [42].

### 2.5.3 Suitability for This Project

Two properties motivated the selection of ETraM for this project. First, critically for an ADAS and autonomous driving context, recordings span the full spectrum of real-world operating conditions: daytime, nighttime, and twilight, across sunny, overcast, and rainy weather - precisely the range of environments an autonomous perception system must handle reliably, and one that deliberately exercises the conditions under which frame-based sensors are known to degrade. Second, the ETraM study provides a dataset-specific spatiotemporal filtering and denoising pipeline tuned to the noise characteristics of the EVK4 sensor, which was adopted directly as the signal conditioning stage of the BOLT preprocessing pipeline. This eliminated the need to develop bespoke noise suppression from first principles, and significantly accelerated the transition to model development. The full BOLT pipeline is detailed in Section 3.3.

## 2.6 Neuromorphic Hardware and Energy Modelling

### 2.6.1 Neuromorphic Hardware Platforms

Intel Loihi 2 [16] and IBM TrueNorth [28] represent the two principal reference platforms for neuromorphic computing. Unlike conventional GPUs, which execute computation synchronously across all neurons at every clock cycle, both architectures operate asynchronously and event-driven. A neuron consumes energy only when it receives a spike and crosses its firing threshold, remaining entirely silent, drawing no dynamic power in the absence of input activity. This mirrors the SNN's spike-based dynamics precisely, making neuromorphic hardware the natural target deployment environment for event-driven architectures and completing the end-to-end alignment with the event camera described in Section 2.4.2. The per-SOP energy costs of these platforms, 16.1 pJ for Intel Loihi 2 [16] and 26.0 pJ for IBM TrueNorth [28], form the hardware extrapolation targets used in the energy evaluation of this project.

### 2.6.2 The MAC vs AC Energy Distinction

The foundational energy analysis underlying this project rests on the silicon-level operation costs documented by Horowitz [19]. A standard CNN performs a full MAC at each synaptic connection - a floating-point multiplication followed by an accumulation, costing  $E_{\text{MAC}} = 4.6$  pJ per 32-bit operation. An SNN, by contrast, communicates exclusively via binary spikes; since a spike is either 0 or 1, the multiplication is eliminated entirely, reducing each synaptic event to a Conditional Addition (AC) operation at a cost of  $E_{\text{AC}} = 0.9$  pJ - a  $5\times$  reduction in per-operation energy at the silicon level [19]. This asymmetry is the first of two complementary mechanisms underpinning the SNN's energy advantage over its CNN counterpart; the second, input-dependent activation sparsity, is introduced and materialised alongside the complete per-frame energy model in Sections 3.8.1 and 3.8.6.

## 2.7 Related Work

### 2.7.1 The Progression of Spiking Architectures

Early efforts to close the accuracy gap between SNNs and CNNs focused on classification tasks on low-resolution benchmarks. Pfeiffer and Pfeil [30] provided foundational evidence that SNNs could exploit temporal sparsity for energy-efficient inference, while Sengupta et al. [35] demonstrated that deep SNNs could approach CNN accuracy on ImageNet via ANN-to-SNN conversion. The architectural frontier advanced further with the SEW ResNet introduced by Fang et al. [12], which resolved the vanishing gradient problem in directly-trained deep SNNs through element-wise spike residuals. Despite this depth progression, a systematic review of 151 studies by Iaboni and Abichandani [21] confirms that SNN evaluation remains predominantly anchored to low-resolution datasets such as MNIST ( $28 \times 28$ ) and CIFAR-10 ( $32 \times 32$ ). More recent efforts have extended SNNs to multi-object detection at higher resolutions [11, 27, 37, 44], but these architectures target general-purpose benchmarks such as COCO; SNN-based multi-object detection in the automotive domain remains a critically underexplored area.

### 2.7.2 Event-Based Object Detection

The landscape of event-based object detection has been comprehensively surveyed by Gallego et al. [14], who established that tensor-based representations, voxel grids and event surfaces, dominate the field, typically consumed by continuous-valued detectors. The introduction of ETraM by Verma et al. [42] reinforced this pattern; despite providing the first HD event-based traffic dataset, all published baselines, RVT, RED, and YOLOv8, are continuous architectures, and no spiking detector has been evaluated against them. Furthermore, no prior work has systematically compared a spiking and a non-spiking detector on a shared HD event dataset under strict architectural parity.

### 2.7.3 The Gap This Project Addresses

Lin et al. [26] established that CPU-based CNN inference had exceeded the thermal and power constraints of early ADAS systems, motivating the industry’s shift to GPU-class accelerators. Since 2018, legislative pressure has driven further growth in perception complexity, placing dense ANN inference on GPUs on a similar trajectory to become unsustainable. Despite this, as Pfeiffer and Pfeil [30] and the event-vision survey of Gallego et al. [14] collectively establish, the neuromorphic alternative has not been rigorously validated at the spatial resolution and domain complexity of real-world automotive perception. The `DetectorNX` project directly addresses this gap: by enforcing strict architectural parity between an SNN and a CNN baseline on the HD ETraM dataset, it provides the first controlled empirical quantification of the precision–efficiency trade-off at the spatial resolution demanded by ADAS systems.

# Chapter 3

## Technical Quality and Methodology

### 3.1 Requirements Analysis

The primary objective of this project was the development and rigorous evaluation of a high-performance Spiking Neural Network (SNN) for multi-object detection using high-definition event-camera data (the ETraM dataset). To effectively quantify the accuracy versus energy efficiency tradeoff between spiking architectures and traditional Convolutional Neural Networks (CNNs), the system had to be designed against a stringent set of functional and non-functional requirements. These requirements guided the iterative evolution of the neural architectures and the underlying computational pipeline.

#### 3.1.1 Functional Requirements

To ensure a fair empirical comparison and to solve the complexities of multi-object detection on neuromorphic data, the following functional criteria were established:

- **Multi-Object Regression Capabilities:** Unlike simplified classification tasks, the system was required to perform multi-object detection capable of identifying multiple vehicles simultaneously within a single scene. This necessitated a regression head capable of outputting multiple bounding boxes per frame (defined by spatial coordinates and confidence scores) while actively suppressing “ghost detections” and resolving spatial ambiguities.
- **Architectural Parity (The “Gold Standard” Baseline):** To strictly isolate the impact of the computational paradigm (event-driven spiking versus continuous-valued activations), the baseline CNN was required to be a 1:1 non-spiking counterpart to the SNN. Both models were constrained to equivalent structures, depths and parameter counts ( $\sim 11.7\text{M}$  parameters). Crucially, to eliminate data variance as a confounding factor, both the SNN and the CNN were trained on the exact same pre-processed neuromorphic dataset (the BOLT pipeline output). This ensured that any observed discrepancies in performance or

power consumption were solely attributable to the underlying network architectures, rather than differing input modalities or varied model capacity.

- **High-Definition Spatiotemporal Ingestion:** The system was required to process raw, high-definition ( $1280 \times 720$ ) event streams. Furthermore, the pipeline had to preserve the temporal dynamics of the data by segregating it into discrete time bins (e.g.,  $T = 10$ ) to allow the SNN to demonstrate temporal evidence accumulation.
- **Comprehensive Evaluation Metrics:** The evaluation suite was required to output granular metrics beyond top-line Mean IoU and mAP<sub>50</sub>. This included real-time dynamic power consumption mapping, confidence calibration analysis (reliability diagrams), and quantisation error tracking across varying object sizes.

### 3.1.2 Non-Functional Requirements and Constraints

The processing of high-definition event data at scales of up to 150 training epochs imposed severe computational bottlenecks. Consequently, the system had to satisfy critical performance and stability constraints:

- **Computational Scalability and Speed:** The SNN simulation had to be optimised for execution on local accelerator hardware (GPUs). This required the integration of Automatic Mixed Precision (AMP) to utilise Tensor Cores, memory-efficient vectorised loss functions to eliminate sequential batch loops, and the utilisation of optimised C++/CUDA kernels (e.g., SpikingJelly’s multi-step modes) over native Python loops.
- **Numerical Stability:** Given the reliance on 16-bit precision through AMP and the deep accumulation of membrane potentials in the SNN, the architecture required specific safeguards against gradient explosion. This constrained the design of the detection head, mandating the use of raw logits and numerically stable loss functions (e.g., BCEWithLogitsLoss).
- **Hardware Extrapolation:** While constrained to local GPU execution, the benchmark outputs (e.g., dynamic power tracking) were required to be translatable into theoretical energy consumption metrics (in mJ) based on established hardware constants, facilitating projections for specialised neuromorphic chips like Intel Loihi 2 or IBM TrueNorth.

These requirements formed the foundation for the three-generation evolution of the `DetectorNX-G3-SNN` architecture detailed in Section 3.4, and drove the design of the rigorous benchmarking suite evaluated in Section 3.8.

## 3.2 System Architecture Overview

To satisfy the success criteria established in Section 1.4 and the functional requirements in Section 3.1, the overall system was designed as an end-to-end, highly modular pipeline. Decoupling the preprocessing, neural backbone, and regression stages enabled the isolation and empirical study of individual architectural modifications, ensuring that any performance gap between the spiking and continuous paradigms is attributable to the paradigm swap itself rather than to differences in preprocessing or head design. The high-level data flow is formalised in Figure 3.1.

As illustrated, the architecture is divided into four primary sequential stages, where the central neural backbone acts as an interchangeable module:

1. **Data Ingestion and The BOLT Pipeline (Fixed):** Raw, asynchronous event data at a high spatial resolution of  $1280 \times 720$  is ingested from the ETraM dataset. Due to the sparse and noisy nature of neuromorphic hardware outputs, the data enters the BOLT preprocessing pipeline, which is heavily adapted from the methodology proposed in the original ETraM study [42]. While the core spatiotemporal noise filtering and bilinear temporal interpolation (to  $T = 10$  bins) remain faithful to the original study, this project included an additional spatial downsampling stage ( $ds = 4 : 1280 \times 720 \rightarrow 320 \times 180$ ). This modification was strategically implemented to ensure training feasibility on local accelerator hardware and reduce input sparsity, thereby improving the stability of neuronal feature extraction. This stage is detailed thoroughly in Section 3.3.
2. **The Neural Backbone (Interchangeable):** The preprocessed voxel tensor serves as the input to the primary feature extraction engine. This stage houses either the spiking architecture `DetectorNX-G3-SNN` or its parameter-matched continuous counterpart `DetectorNX-G3-CNN`. Both backbones comprise four residual blocks — `SEWResBlock` units in the SNN variant, `Standard ResBlocks` in the CNN baseline — with Squeeze-and-Excitation attention at the Stage 3 bottleneck, and together perform an aggressive  $8 \times$  total spatial downsampling. The evolution and formal design of these backbones are detailed in Sections 3.4 and 3.5.
3. **Multi-Box Regression Head (Fixed):** The final abstracted feature maps from either backbone are fed into a unified detection head. To resolve the spatial ambiguities and “ghost detections” observed in earlier architectural iterations, the head utilises an  $8 \times 8$  Adaptive Pooling mechanism to aggregate features from 64 spatial receptive blocks. This abstracted information is then mapped to a final output tensor of shape  $[B, 5, 5]$ , representing the top 5 predicted bounding boxes per image. Each detection is defined by the feature set  $\mathcal{F} = \{x, y, w, h, c\}$ , where  $(x, y, w, h)$  denote the normalised bounding box coordinates and  $c$  represents the object confidence logit. The regression output is evaluated via a fully vectorised Multi-Box Loss function during training.

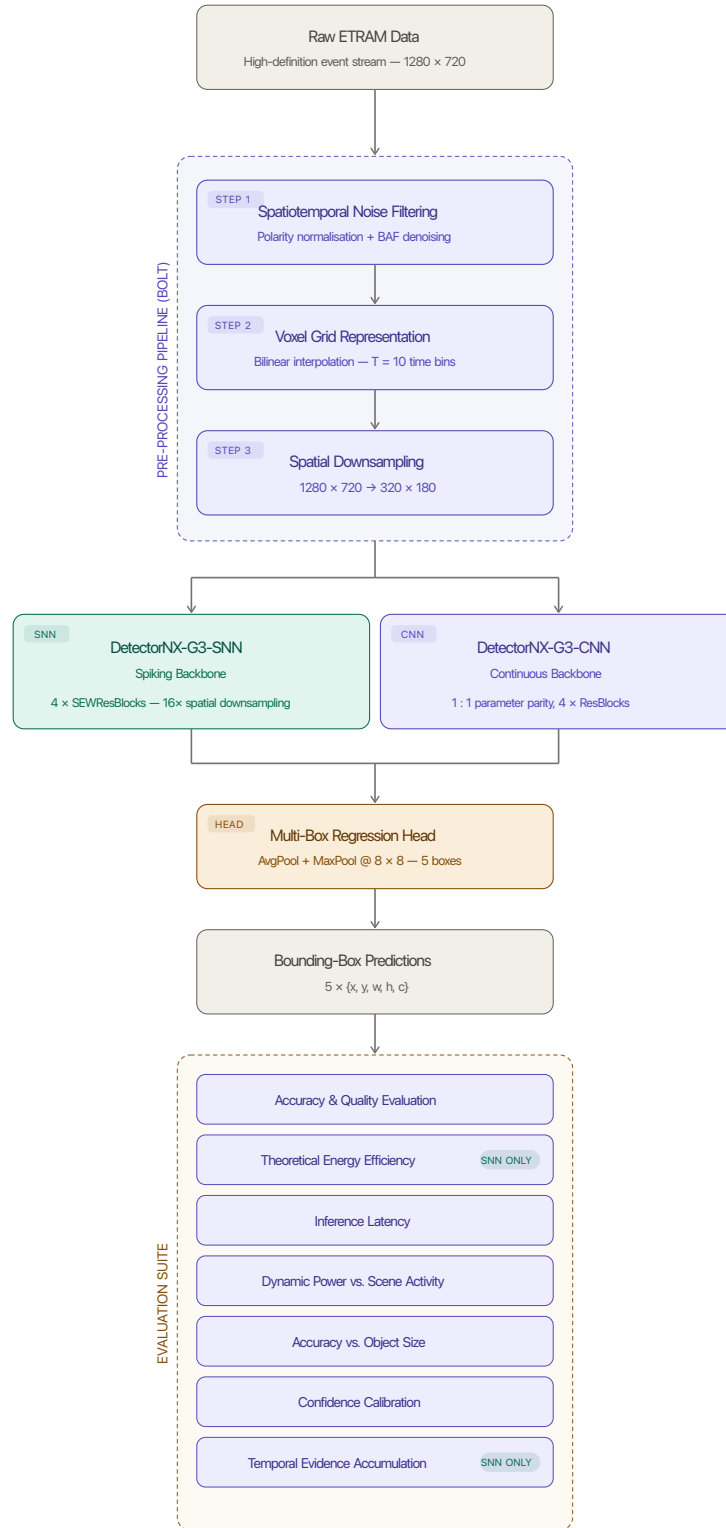


Figure 3.1: The high-level system architecture pipeline, illustrating the flow from raw ETRaM high-definition event data, through the fixed BOLT preprocessing stages, into the interchangeable neural backbone (SNN or CNN), and concluding with the multi-box regression head and the comprehensive evaluation suite.



4. **Evaluation and Benchmarking Suite (Fixed):** The predictions and internal neuronal activity from both backbones are passed into a dedicated benchmarking suite that pairs each of the six success criteria with a matched experiment, as summarised in Table 3.1. Diagnostic analyses of quantisation error (Section 3.8.2) [3] and temporal evidence accumulation (Section 3.8.4) complement the primary accuracy result by explaining *how* the SNN’s rate-coded regression converges, rather than validating a distinct criterion. The full benchmarking suite is detailed in Section 3.8.

| SC  | Success Criterion                                | Architectural Mechanism                                                                 | Validating Experiment                                              |
|-----|--------------------------------------------------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| SC1 | Detection accuracy parity ( $\geq 90\%$ mIoU)    | Multi-Box regression head, SEWResBlock backbone, SE attention at Stage 3 bottleneck     | Global Evaluator (Section 3.8.7)                                   |
| SC2 | Measurable theoretical energy savings            | Binary spike encoding, Firing Rate Regularisation (FRR) metabolic penalty               | Dynamic Power (Section 3.8.1) & Theoretical Energy (Section 3.8.6) |
| SC3 | Real-time viability ( $\geq 30$ FPS)             | Fused CUDA kernels (step_mode='m'), Automatic Mixed Precision (AMP) 16-bit tensor cores | Real-Time Latency (Section 3.8.5)                                  |
| SC4 | Architectural parity (within 1% parameters)      | 1:1 block-for-block backbone matching, shared regression head, shared BOLT input        | Parameter count audit (Tables A.1 and A.2)                         |
| SC5 | Detection completeness (100% recall @ IoU > 0.5) | Vectorised Multi-Box Loss with greedy assignment matching                               | Global Evaluator (Section 3.8.7)                                   |
| SC6 | Confidence calibration equivalence               | Objectness logits + BCEWithLogitsLoss on unmatched boxes                                | Confidence Calibration (Section 3.8.3)                             |

Table 3.1: Mapping of each success criterion (Section 1.4) to the architectural mechanism that addresses it and the dedicated experiment that validates it in Chapter 5.

This Fixed/Interchangeable partition is the mechanism by which SC4 (Architectural Parity) is enforced: all paradigm-level variance between the two evaluated models is confined to Stage 2, so any discrepancy observed in Chapter 5 is attributable to the spike-vs-continuous swap alone.

### 3.3 Data Preprocessing: The BOLT Pipeline

As established in Section 2.4, an event camera outputs a sparse, asynchronous stream of events  $e = (x, y, t, p)$ , where  $(x, y)$  are the spatial coordinates,  $t$  is the microsecond timestep, and  $p \in$

$\{-1, 1\}$  is the polarity indicating an increase or decrease in brightness. The BOLT preprocessing pipeline takes the raw ETraM data at  $1280 \times 720$  spatial resolution and converts it into the dense, structured tensor format required by the SNN and CNN backbones.

### 3.3.1 Raw Event Ingestion and Windowing

The BOLT pipeline begins by reading the raw h5 event files  $(x, y, t, p)$  and the partnering npy files containing the bounding box annotations from the ETraM dataset. Once both files have been parsed, the first step taken is to apply a  $100\text{ms}$  sliding window across the entire duration of the event file. “Windowing” is the process of slicing the continuous event stream into chunks that represent a single instance of time for the model, thereby anchoring the asynchronous event stream into discrete, structured training instances.

To maximise training efficiency and ensure a high signal-to-noise ratio in the training distribution, the BOLT pipeline adopts a Box-Central sampling strategy rather than more conventional uniform temporal partitioning; training windows are anchored deterministically around the centroids of existing ground truth annotations. This approach ensures that 100% of the training voxels contain informative structural targets, mitigating the prevalence of ‘empty-scene’ noise and providing the SNN with consistent temporal evidence to drive membrane potential accumulation prior to the target timestamp.

In this project, the time window  $\Delta T = 100\text{ms}$  was chosen to align with the temporal persistency of the human visual system of roughly  $100\text{ms}$  [6]; just as the human eye integrates luminance changes over approximately one frame of perception, the SNN also integrates the asynchronous event stream over the same duration before producing a detection output.

The practical consequence of this choice is two-fold. First, a  $100\text{ms}$  window is long enough to guarantee a minimum event density sufficient for the LIF neurons in the stem layer to accumulate membrane potential and produce meaningful spike trains across all  $T = 10$  temporal bins. Second, specific to the context of vehicle detection, it is short enough that a vehicle travelling at highway speed (e.g.,  $120\text{ km/h} \approx 33\text{ ms}^{-1}$ ) displaces by only  $3.3\text{m}$  within the window, ensuring that the bounding box annotation assigned to the window remains spatially valid and does not introduce label noise through excessive object displacement.

In combination with the Box-Central sampling approach, the pipeline applies a density filter prior to voxelisation: for each  $100\text{ms}$  window the raw event count is checked, and windows containing fewer than `min_events = 500` events are discarded. This threshold is a structural necessity for SNN training stability. Near-empty windows collapse the learning signal on two fronts: the forward pass produces few or no spikes, yielding a zero surrogate gradient [29] which causes the optimiser to suppress firing globally [8], while the FRR regulariser, which penalises deviation from a 20% target firing rate [5, 43], dominates the total loss because the target firing rate is unattainable when the input carries no events. A 500-event floor also filters out the baseline Background Activity noise that HD event cameras emit from transistor mismatch and

thermal effects [14], ensuring training attention is directed only at windows where meaningful scene change has occurred.

### 3.3.2 Signal Conditioning

Following windowing, the BOLT pipeline applies a series of signal-conditioning operations, lifted from the original ETraM study since they were tuned to the characteristics of the EVK4 sensor [42]. Normalising the voxel tensor to  $[0, 1]$  prevents LIF neuron saturation at the stem layer. Without it, high-magnitude or noisy voxel values drive every neuron to fire continuously and flood the network with uninformative spikes, whereas the constrained range ensures only neurons encoding genuine structural features (vehicle edges, motion boundaries) exceed threshold [8]. The Background Activity Filtering (BAF) denoising step then rejects the low-activity clusters that HD event cameras emit from flickering illumination and sensor noise, freeing representational capacity for real vehicle geometry [8, 33]. Finally, every background event suppressed at preprocessing removes a spike emission at inference, tying preprocessing quality directly to the SOP count on neuromorphic hardware [33].

### 3.3.3 Spatiotemporal voxelisation

To make the asynchronous event data compatible with neural network layers, a voxelisation process is performed to transform the continuous stream into a structured tensor. This process consists of three core operations:

- **Temporal Binning:** The total duration of an event sample (e.g., a 100ms window) is divided into 10 discrete time bins ( $T = 10$ ).
- **Polarity Splitting:** Events are separated into two distinct channels based on their polarity ( $C = 2$ ), representing positive and negative brightness changes.
- **Event Accumulation (Histogramming):** Within each discrete temporal bin and polarity channel, the individual asynchronous events are aggregated at their respective spatial coordinates  $(x, y)$ . Specifically, the value of each “pixel” in the resulting spatial grid becomes the total count of events (of that specific polarity) that occurred at that exact location during that fraction of time. This process effectively converts the sparse, continuous point-cloud of events into a sequence of dense 2D frames—or event histograms—which standard convolutional and spiking layers can natively process.

This accumulation process preserves the temporal distribution of the events while generating a dense 4D spatiotemporal tensor of shape  $[10, 2, 720, 1280]$  (representing Time  $\times$  Channels  $\times$  Height  $\times$  Width).

### 3.3.4 Downsampling and Normalisation

Following spatiotemporal voxelisation, the voxelised data undergoes a dual-phase refinement consisting of spatial downsampling and amplitude normalisation. This is the final polishing step of the pipeline, transforming the high-resolution event counts into a stabilised, low-memory format that the neural backbones can process efficiently.

#### Spatial Downsampling (16-fold Data Reduction)

The generated high-definition voxel grid ( $1280 \times 720$ ) is subjected to a  $4 \times 4$  Max-Pooling operation, reducing the spatial resolution to  $320 \times 180$ . This 16-fold reduction in data volume is critical for managing the high-dimensional state-space of the SNN backbone. Because SNNs must store neuron membrane potentials across time, an HD model would require massive amounts of GPU memory. By aggressively reducing the spatial dimensions, we ensure the backbone scalability, making it computationally feasible to train the deep 4-block architecture over extended epochs.

Furthermore, utilising Max-Pooling (as opposed to Average-Pooling) acts as a feature pre-server: it retains the strongest event signal within each  $4 \times 4$  area, helping the network maintain the crisp, high-contrast edges of the vehicles even at a lower resolution.

It should be noted that the downsampling factor  $ds = 4$  was driven by the High Performance Computing (HPC) cluster’s GPU memory constraints; without using  $ds = 4$ , the data would not fit into the GPU memory, or would require a batch size of 1, making training infeasible within the time constraints of this project.

#### Max-Normalisation (Input Calibration)

Subsequently, max-normalisation is performed to scale the raw event accumulation counts into a continuous  $[0, 1]$  interval, by dividing every pixel in the temporal volume by the single highest event count found within that 10-bin voxel.

This normalisation serves as a fundamental safeguard against neural saturation. LIF nodes have fixed firing thresholds (in this architecture,  $V_{th} = 0.2$ ) - if a raw, unscaled event count of 50 were fed into the network, the first-layer neurons would fire instantaneously and repeatedly, effectively “screaming” and drowning out all other spatiotemporal information. By enforcing input calibration, we ensure that the input magnitudes are calibrated relative to the LIF thresholds, facilitating stable surrogate gradient propagation during the 150-epoch training cycle.

### 3.3.5 Multi-Object Label Alignment

The final stage of the pipeline implements a Multi-Object Label Alignment strategy, which is fundamental to the architecture’s multi-object detection capabilities. This stage transforms

individual, asynchronous bounding box annotations into structured, scale-independent target tensors suitable for the network’s multi-box regression head.

### **Timestamp-Based Grouping (Unified Scene Representation)**

Instead of creating isolated training samples for every individual vehicle, the pipeline parses the ground truth annotations and groups all vehicles that exist at the exact same microsecond ( $t$ ). This process merges individual detections into a Unified Scene Representation, which is crucial because it eliminates “label noise”; presenting the network with identical visual input but different single-car targets across multiple samples would force the model to average its predictions, resulting in blurry, low-confidence bounding boxes with poor IoU scores.

### **Fixed-Capacity Padding**

Deep learning models inherently require consistent tensor dimensions for batched training. To accommodate scenes with varying numbers of vehicles, we enforce a Fixed-Capacity Interface of exactly 5 bounding boxes per frame. If a scene contains 2 vehicles, the remaining 3 slots are populated with “empty” zero-padded boxes. If a frame contains no vehicles, 5 empty boxes are provided. This ensures every target label is a consistent  $[5, 5]$  tensor. The limit of 5 boxes was empirically selected based on an analysis of the ETram validation set, where the vast majority of frames contain 3 or fewer vehicles, making 5 a safe upper bound for this specific traffic monitoring environment.

### **Binary Masking**

To enable the network to distinguish between genuine vehicle targets and zero-padded instances, the tensor utilises a Binary Masking dimension. For every box, the target tensor stores five values:  $[Center_X, Center_Y, Width, Height, Mask]$ .

- For a real object, the mask value is set to 1.0.
- For a padded or empty box, the mask value is set to 0.0.

This mask acts as the objectness confidence target. It is utilised by the Multi-Box Loss function to selectively apply coordinate gradients only to valid targets, while simultaneously regularising the model’s confidence head to output 0.0 for empty regions.

### **Normalisation to Sensor Space**

Finally, all spatial parameters (centre coordinates, width, and height) are originally provided in raw pixel values (e.g.,  $x = 842$ ). These are normalised by dividing by the raw sensor width (1280) and height (720), scaling all coordinates into the continuous range  $[0.0, 1.0]$ . This provides a Scale-Independent Signal, ensuring the regression head can learn the underlying geometry robustly, regardless of the spatial downsampling factors applied earlier in the pipeline.

### 3.3.6 Chunking for High-Throughput I/O

Training the model for 150 epochs is only feasible if the BOLT pipeline is executed once rather than on every epoch. A naive dataloader would stream the raw h5 and npy files in whatever segment fits in memory and re-run the full pipeline (signal conditioning, voxelisation, downsampling, label alignment) on each pass, multiplying the preprocessing cost 150-fold and starving the GPU while the CPU works through every voxelisation and denoising call.

The chunking strategy moves the preprocessing out of the training loop entirely. The BOLT pipeline is executed once, offline, and the resulting spatiotemporal voxels and aligned label tensors are serialised to disk as contiguous 250-sample chunks (compressed .pt), indexed by a global `manifest.json` byte-offset map. During training, the dataloader reads these precomputed chunks directly into VRAM, so the 150 epochs execute against structured tensors rather than raw events.

#### LRU Caching in `VoxelDiskDataset`

To further amortise disk I/O, the custom `VoxelDiskDataset` wraps the chunk layout with a Least Recently Used (LRU) memory cache sized to hold the full 40-chunk dataset in the node’s 120 GB RAM. During Epoch 1, chunks are read from NFS into memory as the dataloader encounters them, populating the cache in full. Every subsequent epoch is then served entirely from RAM, so the training loop never touches disk after Epoch 1. The LRU eviction policy is a safety net for cache-smaller-than-dataset configurations: the least-recently-used chunk is evicted to make room for the next load, ensuring the hot data is always ready for the GPU.

The combined effect of offline precomputation, chunked serialisation, and in-memory caching reduces the 150-epoch wall time from a projected multi-day run to approximately six hours at  $\sim 160$  ms/iter.

### 3.3.7 BOLT Pipeline: Visualised

Figure 3.2 details the order of which the aforementioned preprocessing steps are applied to the raw event data.

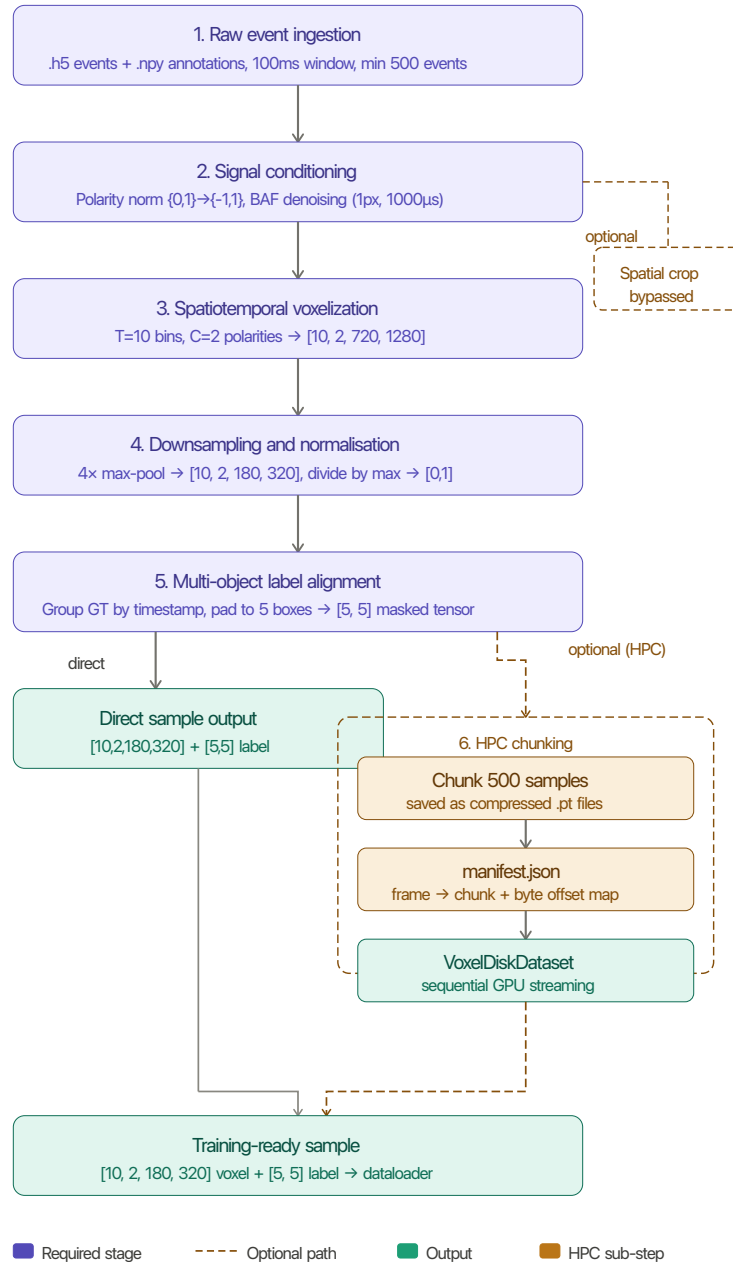


Figure 3.2: Overview of the spatiotemporal preprocessing pipeline applied to the ETrAM dataset. Raw events are voxelised into a 4D tensor, spatially downsampled, and aligned with multi-object ground truth labels to produce a final sample of shape  $[10, 2, 180, 320]$  with a corresponding label tensor of shape  $[5, 5]$ .

### 3.4 Spiking Neural Network (SNN) Architecture

The core focus, effort and innovation of this project is the development and optimisation of the `DetectorNX-G3-SNN` architecture. This model is explicitly designed to process high-definition neuromorphic event data while adhering to the computational paradigms of SNNs. By prioritising event-driven, binary communication, the architecture aims to drastically reduce both the dynamic power consumption and the computational complexity compared to traditional deep learning models, making it superior for power and time constrained environments.

#### 3.4.1 Spatiotemporal Input and Multi-Step Execution

The raw ETraM event data is initially preprocessed to a spatial resolution of  $320 \times 180$  with  $T = 10$  discrete temporal bins. Consequently, the network ingests a massive, high-dimensional spatiotemporal tensor of shape  $[B, 10, 2, 180, 320]$  (Batch, Time, Polarity, Height, Width).

Given the high spatial dimensionality and the iterative nature of spiking neurons, native Python loop execution over the time dimension proved computationally restrictive. To overcome this bottleneck, the entire architecture was refactored to utilise the SpikingJelly framework's optimised Multi-Step (`step_mode='m'`) execution paradigm [13]; instead of iterating through the 10 time bins sequentially, the entire spatiotemporal tensor is passed directly into custom-compiled Compute Unified Device Architecture (CUDA) kernels. This low-level optimisation enables high-speed, parallelised execution of the temporal dynamics, making the extensive benchmarking of this HD dataset feasible on local GPU hardware, and reducing overall training time from approximately 16 hours to 7.

#### 3.4.2 Neuron Dynamics: LIF

At the foundation of the architecture lies the LIF neuron model as defined in Section 2.3.1, which governs the temporal dynamics and binary spike generation of the network.

##### The Forward Pass (Spike Generation)

Both the membrane potential dynamics from Equation 2.2 and Heaviside spike function from Equation 2.3 are configured with the following project-specific parameters:

$$\begin{aligned} V_{th} &= 0.2 \\ V_{reset} &= 0.0 \\ \tau &= 4.0 \end{aligned}$$

The voltage threshold  $V_{th} = 0.2$  was tuned to balance information flow against noise. Earlier revisions used  $V_{th} = 0.3$ , which silenced the network, whereas decreasing  $V_{th}$  further submitted erroneous background noise as spikes.



The time constant  $\tau = 4.0$  creates a leaky effect, allowing the neuron to retain input evidence across roughly four to five timesteps – low enough to prevent over-saturation, but high enough for the signal to be remembered until the regression head before decay.

Following each spike, the neurons apply a hard reset to  $V_{reset} = 0.0$ , enforcing temporal sparsity by requiring the neuron to integrate new spatiotemporal information before re-activating, and providing the zero-centred baseline required for stable surrogate gradient descent [12].

### The Backward Pass (Surrogate Gradients)

As described in Section 2.3.2, the Heaviside function’s non-differentiable property poses a challenge in spiking neural networks. To get around the zeroed gradients and the consequent “dead neuron” problem, the SNN model employs the ATan function described in Section 2.3.2, where the backward pass approximates the step function using Equation 2.4; this mathematical substitution allows the network to learn complex spatiotemporal representations from scratch while maintaining a purely binary forward execution. In this implementation, SpikingJelly’s default value of  $\alpha = 2.0$  was used, which is the standard value used for high-performance SNN training.

### 3.4.3 Backbone: SEW-ResNet and Attention

To extract high-level semantic features and manage the computational complexity of the high-definition input, the architecture utilises a deep residual backbone divided into five sequential stages, as outlined in Table 3.2.

| Stage | Name              | Resolution      | Channels | Primary Operation                                       |
|-------|-------------------|-----------------|----------|---------------------------------------------------------|
| 1     | Initial Stem      | $90 \times 160$ | 64       | Stride-2 Conv + LIF (Downsamples HD input)              |
| 2     | Backbone Alpha    | $22 \times 40$  | 256      | Blocks 1 & 2 (SEWResBlocks with Stride-2)               |
| 3     | Attention Bridge  | $22 \times 40$  | 256      | Squeeze-and-Excitation (SE) Global Firing Recalibration |
| 4     | High-Res Backbone | $22 \times 40$  | 512      | Blocks 3 & 4 (SEWResBlocks with Stride-1)               |
| 5     | Temporal Fusion   | $22 \times 40$  | 512      | Summation of spikes across all $T=10$ timesteps         |

Table 3.2: Detailed breakdown of the five sequential stages in the `DetectorNX-G3-SNN` backbone.

### Spatial Downsampling

As illustrated, the architecture implements an aggressive  $8 \times$  total spatial downsampling strategy to compress the initial  $320 \times 180$  input. This is achieved through strided convolutions applied

within the Initial Stem and Backbone Alpha, progressively reducing the spatial dimensions to a highly compressed  $22 \times 40$  grid while expanding the channel width up to 512.

### Spike-Element-Wise (SEW) Residual Blocks

Building deep SNNs presents significant challenges due to the “vanishing spike” problem inherent in standard residual connections. In a standard Residual Neural Network (ResNet), the residual addition occurs *before* the activation function. However, if we follow this standard and add two binary spike trains, the result is a non-binary, continuous value (e.g.,  $1 + 1 = 2$ ), which destroys the energy-efficient, event-driven nature of the network [12].

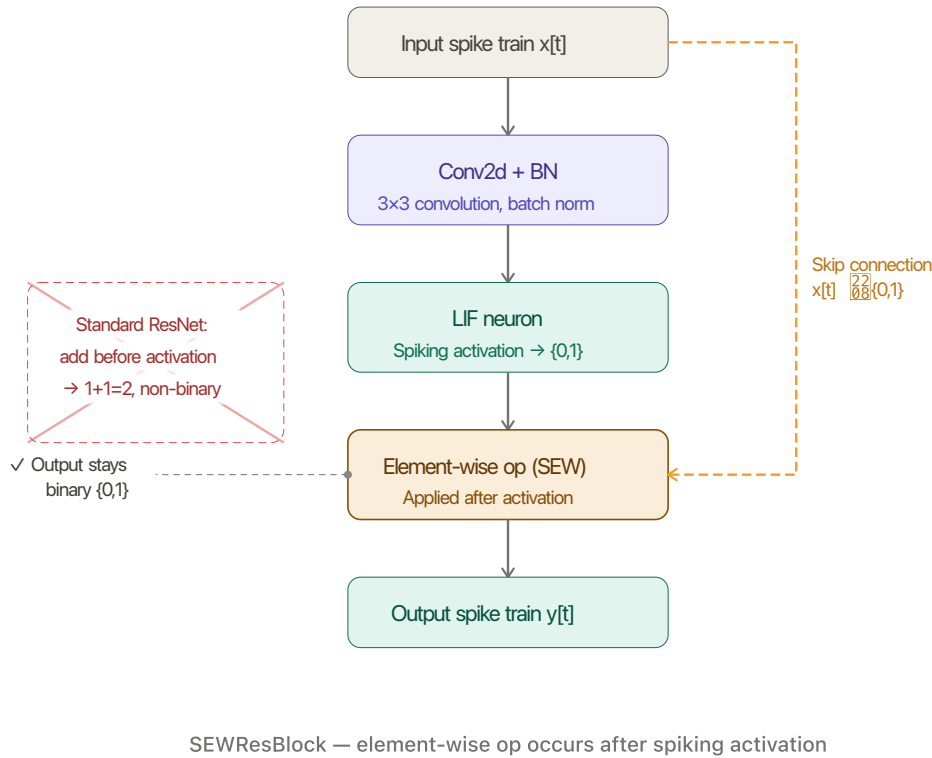


Figure 3.3: The SEW residual block (SEWResBlock) as proposed by Fang et al. [12].

To overcome this, the DetectorNX-G3-SNN architecture adopts the SEW residual paradigm [12], instantiated as SEWResBlock units throughout the backbone. Unlike standard Spiking ResNets, SEW ResBlocks apply the element-wise residual operation *after* the spiking activation function, guaranteeing that the output remains strictly binary. Fang et al. [12] prove that this reordering is both necessary and sufficient to achieve identity mapping in a purely spike-based network, resolving the degradation problem without sacrificing neuromorphic compatibility. In this architecture, this property is exploited to stack 4 SEWResBlocks across the backbone stages (Stages 2 and 4 in Table 3.2), enabling representational depth that would otherwise cause spike degradation under a standard residual formulation.

### Structural Attention: SE-Block

In deep learning, a bottleneck refers to a point in the neural network where the information flow is forced through its most compressed and abstract representation. In this `DetectorNX-G3-SNN` architecture, the bottleneck exists at the transition point between stages 2 and 3 of the backbone, and represents two things:

1. **A Spatial Constraint (Resolution Limit):** At this stage, the original  $320 \times 180$  spatiotemporal input has been downsampled by a factor of 8x via the strided convolutions in the stem and early residual blocks, resulting in a highly compressed  $22 \times 40$  feature map. This spatial bottleneck is critical because each remaining pixel must now represent a large  $8 \times 8$  receptive field of the original HD sensor data, forcing the network to discard fine-grained geometric redundancy while retaining the core structural boundaries of multiple detected objects.
2. **A Semantic Shift (High-Level Abstraction):** This juncture marks the definitive pivot where the network transitions from extracting low-level geometric primitives, such as spike-edge orientation and motion direction, to synthesizing complex semantic concepts. Without an explicit selection mechanism, the network at this depth is susceptible to noise and artefacts from the HD event stream, which can blur the distinction between legitimate targets and background interference.

To both mitigate and solve these bottleneck constraints, a Squeeze-and-Excitation (SE) mechanism is introduced in Stage 3 of the backbone, sitting directly at the transition point [20]. The SE block operates by performing a global average pooling of the 256 feature channels (the “Squeeze” phase), followed by two fully connected layers that calculate a weighting factor for each channel (the “Excitation” phase). This effectively mitigates the spatial constraint by allowing the network to incorporate global context into its local representations, compensating for the loss of resolution.

When applied in the context of an SNN, this acts as a dynamic firing modulator: the backbone’s activations are re-calibrated by looking at the global context of the entire frame, magnifying the most informative channels whilst suppressing those containing artefacts. By applying this recalibration, the SE block ensures that only the most relevant “objectness”, where objectness is the probability that a specific region contains a genuine object of interest rather than background noise, are passed into the final high-capacity blocks, effectively mitigating the semantic ambiguity before the final bounding box regression.

#### 3.4.4 Firing Rate Regularisation (FRR)

A common failure mode in deep SNN training is network silencing (zero spikes emitted) or over-saturation (firing continuously at every timestep), both of which halt gradient flow and

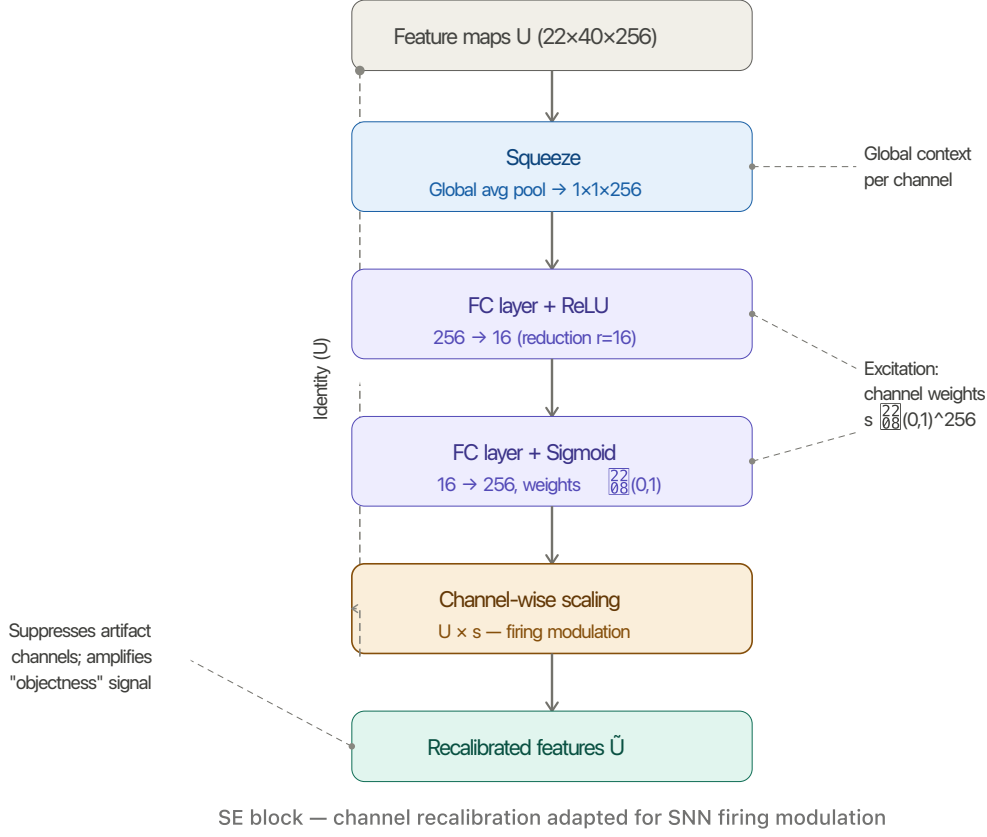


Figure 3.4: The Squeeze-and-Excitation (SE) block [20] as applied in Stage 3 of the DetectorNX-G3-SNN backbone ( $22 \times 40 \times 256$ ).

destroy energy efficiency. To constrain the network, a custom Firing Rate Regularisation (FRR) Metabolic Penalty was integrated into the training pipeline.

The FRR applies a squared error loss to the network’s average firing rate:

$$L_{frr} = \lambda \cdot (Rate_{avg} - Target)^2 \quad (3.1)$$

This biologically inspired metabolic constraint [33] forces the 11.7M parameter backbone to maintain a selective but informative firing rate (e.g.,  $Target = 20\%$ ) across its hidden layers. Penalising deviation from a target firing rate is an established technique for preserving activation sparsity in directly trained SNNs [5, 43]. Adding this penalty to the model’s loss ensures high information flow while strictly preserving the activation sparsity required for efficient neuromorphic deployment.

### 3.4.5 Multi-Box Regression Head

Following the final backbone stage, the fused spatiotemporal feature map is routed into a unified detection head structured as a three-stage pipeline: a channel projection, a dual adaptive pool, and a final regression mapping.

A  $1 \times 1$  convolution first projects the 512-channel fused tensor down to 128 channels, compressing feature dimensionality ahead of the spatial reduction and lowering the parameter count of the downstream head.

The projected map is then passed through two parallel  $8 \times 8$  adaptive pools, an Adaptive Average Pool and an Adaptive Max Pool, whose outputs are summed element-wise. Average pooling captures the global scene statistics but tends to smooth out sparse, informative responses; max pooling preserves the strongest localised activations but discards distributional context. Summing the two retains both signals, mitigating the spatial ambiguities observed during the iterative design process (detailed in Section 3.6).

The resulting  $[B, 128, 8, 8]$  tensor is then mapped via final convolutional and linear layers to an output tensor of shape  $[B, 5, 5]$ , where  $B$  represents the batch size. The final two dimensions encode the top 5 predicted bounding boxes globally per image, with each box defined by a feature tuple  $\mathcal{F} = \{x, y, w, h, c\}$ . Here,  $(x, y, w, h)$  represent the normalised spatial coordinates and dimensions of the bounding box, and  $c$  denotes the object confidence logit. This formulation allows the network to predict multiple vehicles simultaneously, which are subsequently evaluated using a vectorised Multi-Box Loss function.

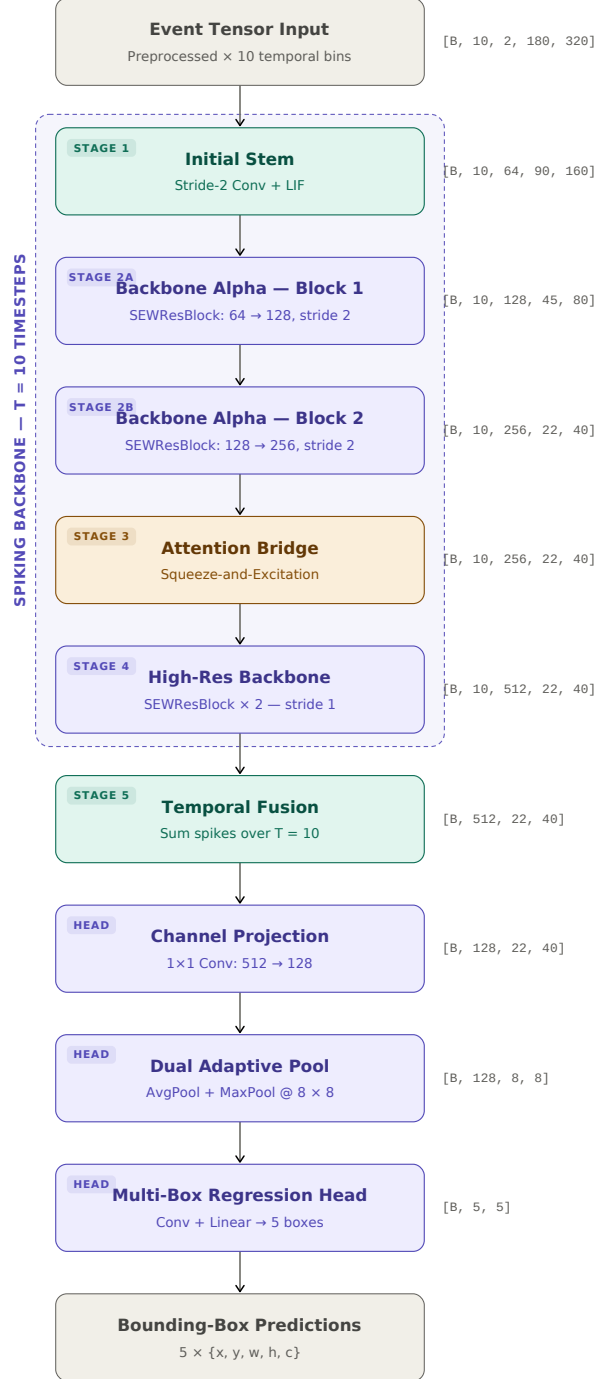


Figure 3.5: End-to-end data flow through the DetectorNX-G3-SNN architecture. The spiking backbone (Stages 1–4) processes the full spatiotemporal tensor in parallel across  $T = 10$  timesteps, after which Temporal Fusion collapses the time dimension prior to the multi-box regression head. A full list of architectural and training hyperparameters is provided in Table A.1 of the appendix.

## 3.5 Convolutional Neural Network (CNN) Baseline

The CNN architecture `DetectorNX-G3-CNN` was engineered as the primary baseline to evaluate the SNN against. It is a 11.73M parameter CNN specifically designed to mirror the SNN architecture, satisfying the ‘‘Architectural Parity’’ functional requirement specified in 3.1.1. This ensures that the only variable being tested in this comparative study is the neural activation mechanism (event-driven binary spikes versus continuous-valued floats), making the comparison immune to common criticisms regarding unmatched or unoptimised baselines.

### 3.5.1 Structural Configuration and Layer-Wise Mapping

The CNN mirrors the SNN’s configuration strictly 1:1, replacing every temporal and spiking component with a standard continuous equivalent. Table 3.3 outlines this direct mapping.

| SNN Component                                  | CNN Equivalent                          | Technical Purpose                                                                |
|------------------------------------------------|-----------------------------------------|----------------------------------------------------------------------------------|
| Spatiotemporal Voxel $[B, 10, 2, H, W]$        | Flattened Channel Input $[B, 20, H, W]$ | Translates 10 time bins into a format the respective network can ingest.         |
| LIF Neuron (Leaky Integrate-and-Fire)          | ReLU Activation                         | Provides the nonlinearity; SNN uses discrete pulses, CNN uses continuous values. |
| Binary Spikes $\{0, 1\}$                       | Continuous Floats $[0, \infty)$         | Represents the information ‘‘currency’’ being passed between layers.             |
| Surrogate Gradient                             | Exact Derivative (Standard)             | Enables backpropagation through the non-differentiable spiking threshold.        |
| SEWResBlock (Spiking Shortcut)                 | Standard Residual Block                 | Implements identity mapping to prevent vanishing gradients during training.      |
| Multi-Step Mode ( <code>step_mode='m'</code> ) | Single spatial pass                     | Processes the temporal window efficiently on the GPU.                            |
| Squeeze-and-Excitation (Spike-based)           | Squeeze-and-Excitation (Float-based)    | Recalibrates channel importance based on global frame context.                   |
| Temporal Summation ( $\sum_{t=1}^T S_t$ )      | N/A (Direct Feature Pass)               | Aggregates 100ms of evidence into a single map for the regression head.          |
| Multi-Box Head (5 Boxes)                       | Multi-Box Head (5 Boxes)                | Predicts coordinates and objectness scores for 5 targets simultaneously.         |
| Synaptic Operations (SOPs)                     | Floating Point Ops (FLOPs)              | The metric used to quantify and compare hardware energy efficiency.              |

Table 3.3: 1:1 Architectural Mapping (SNN vs. CNN) demonstrating strict structural parity. Both models utilise an identical parameter budget (11.7M) and channel progression, differing only in the temporal dimension and activation functions (LIF vs. ReLU).

### 3.5.2 Input Strategy: Channel-Wise Temporal Integration

Unlike the SNN, which processes event data sequentially over discrete time steps  $T$ , the CNN baseline utilises a flattened temporal input strategy. Instead of processing 10 timesteps sequentially, it concatenates the  $T = 10$  bins (each containing 2 polarity channels) into a single input tensor of 20 channels, changing the input shape from  $[B, 10, 2, H, W]$  to  $[B, 20, H, W]$ .

This approach allows the CNN to correlate events across the entire 100ms window in a single spatial pass. This channel-wise stacking of event bins is a standard and effective methodology for feeding asynchronous event-camera data into traditional, non-spiking CNN architectures [15].

### 3.5.3 Backbone: Identity Mapping and Residual Connections

The CNN backbone employs deep residual learning (Section 2.2), utilising four Standard Residual Blocks that expand to 512 channels in the final stage. To satisfy Architectural Parity, the SEWResBlocks of the SNN were replaced with Standard Residual Blocks of the structure Conv  $\rightarrow$  BN  $\rightarrow$  ReLU  $\rightarrow$  Conv  $\rightarrow$  BN + Shortcut  $\rightarrow$  ReLU; these shortcut paths mitigate vanishing gradients and enable stable training at this depth [18].

### 3.5.4 SEWResBlock vs Standard ResBlock

The core mathematical difference between the two architectures lies in their summation logic and the resulting information flow across the shortcut connection:

- **Standard ResBlock (CNN):** Performs an accumulative floating-point addition ( $y = f(x) + x$ ). The continuous intensity of the residual path is directly added to the identity path before the final ReLU activation. Here, the shortcut carries the *raw input features* forward. This mechanism allows the model to learn the “difference” (the residual) needed to improve the representation, focusing on information preservation [18].
- **SEWResBlock (SNN):** Performs a relational element-wise spiking operation *after* the activation function. Because adding two binary spikes (e.g.,  $1 + 1 = 2$ ) produces a non-binary value, the SNN uses a spike-merging function (such as ADD or AND) to preserve the discrete  $\{0, 1\}$  state. Crucially, the SNN shortcut carries the *membrane history* forward, prioritising temporal integrity.

Table 3.4 summarises these foundational differences in identity mapping.

| Feature           | Standard ResBlock (CNN)         | SEWResBlock (SNN)                 |
|-------------------|---------------------------------|-----------------------------------|
| Summation         | Accumulative ( $y = f(x) + x$ ) | Relational (Spiking Element-Wise) |
| Output Type       | Continuous Float $[0, \infty)$  | Binary Spikes $\{0, 1\}$          |
| Primary Benefit   | Prevents vanishing gradients    | Prevents spike vanishing          |
| Mathematical Goal | Information preservation        | Temporal integrity                |

Table 3.4: Comparison of identity mapping between Standard ResBlocks and SEWResBlocks.

This distinction is particularly relevant for high-speed neuromorphic data: while the Standard ResBlock accumulates continuous values that effectively blur fast movements into a single averaged float over the 100ms window, the SNN equivalent preserves exact spike timing, maintaining the temporal precision required for the regression head.



### 3.5.5 Attention Mechanism: SE-Block

The CNN encounters the same bottleneck as its SNN counterpart (Section 3.4.3) and adopts the same SE mechanism [20]. The two differ only at the signal level: the squeeze operates over continuous activations via standard global average pooling, and the excitation produces a float-valued per-channel scale rather than modulating spike rates. While this block offers no hardware-level sparsity advantages here, it is retained solely to maintain architectural parity with the SNN (Section 3.1.1).

### 3.5.6 The “Gold Standard” Upper Bound

By constructing this custom CNN mirror, the baseline serves as the upper theoretical bound of this specific architecture’s capability on the ETraM dataset. For example, if the CNN baseline achieves a Mean IoU of 48%, this represents the maximum performance extractable by this parameter configuration. If the corresponding SNN achieves 43% IoU, the 5% gap explicitly represents the “quantisation cost” of switching from 32-bit continuous floats to 1-bit binary spikes [3]. Adhering to this methodology ensures that the evaluation of the neuromorphic paradigm is both fair and scientifically sound.

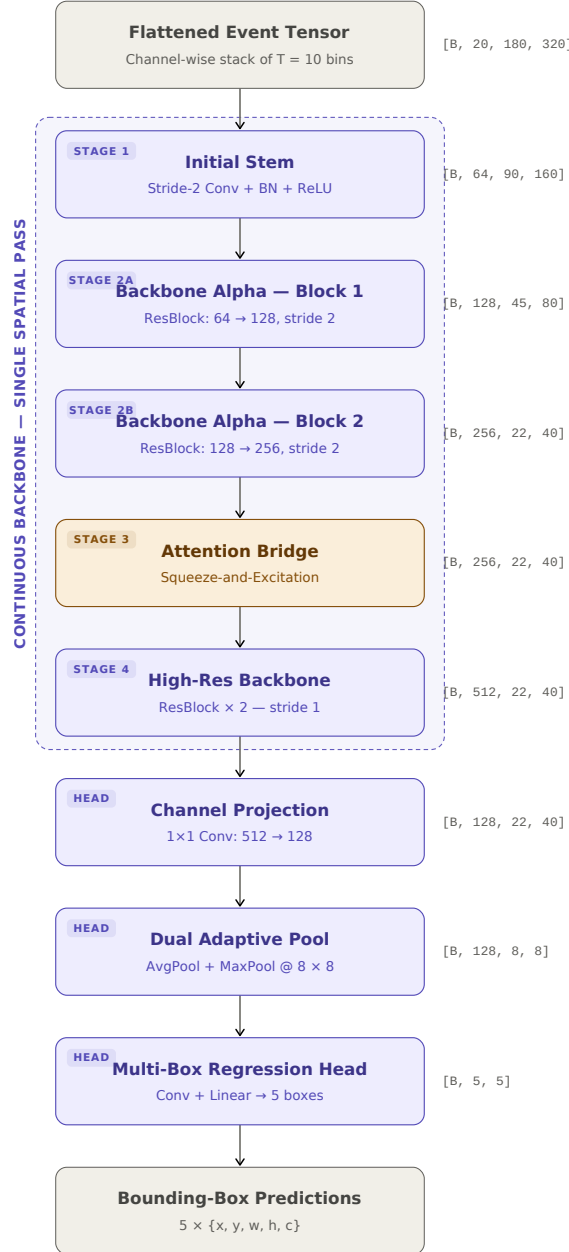


Figure 3.6: End-to-end data flow through the DetectorNX-G3-CNN baseline. The four backbone stages mirror those of the SNN (Figure 3.5) 1:1, differing only in activation (ReLU instead of LIF), residual formulation (Standard ResBlock instead of SEWResBlock), and temporal handling: the  $T = 10$  bins are channel-stacked into a 20-channel input and processed in a single spatial pass, removing the need for a Temporal Fusion stage. A full list of architectural and training hyperparameters is provided in Table A.2 of the appendix.

## 3.6 Architectural Evolution

The final G3 iteration of the `DetectorNX` architecture was not the result of a single design phase, but rather the culmination of an iterative engineering process across four distinct generations. According to the project’s success criteria, the model required both high temporal precision and robust spatial localisation. This section documents the empirical obstacles encountered during development, and the actionable insights that led to the final optimised configuration.

### 3.6.1 Generation 0: The Spatial Cropping Pitfall (Early Prototyping)

#### 1. Architectural Intent

Prior to the implementation of global downsampling, early prototyping explored the use of aggressive spatial cropping to bypass the computational costs of high-definition event processing.

The hypothesis was based on improving the Signal-to-Noise Ratio (SNR) at the network’s input: by cropping the raw  $1280 \times 720$  ETraM sensor data into deterministic  $224 \times 224$  patches anchored to the target vehicle, the initial stem layers would receive a significantly higher frequency of spikes per pixel. This *localised foveation* aimed to preserve fine geometric detail without incurring the memory overhead of processing the full HD frame.

#### 2. Empirical Result

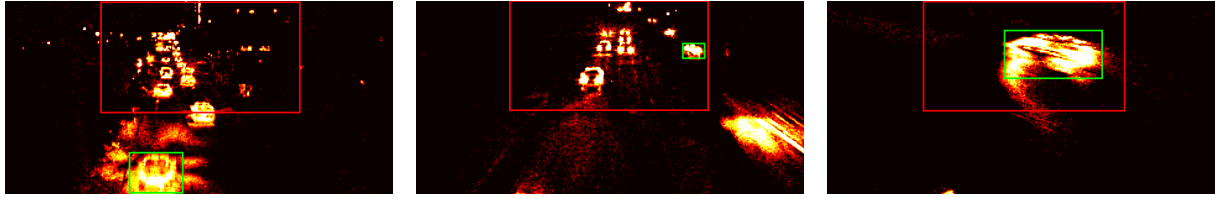
During training on the cropped datasets, the model appeared highly successful, easily converging to an apparent validation mIoU of  $> 70\%$ . However, when this trained model was evaluated against raw, uncropped full frames, performance collapsed to an effective mIoU of near zero.

#### 3. Diagnostic Analysis: Constant Target Bias

Analysing the output frames revealed that the network had suffered from *Constant Target Bias*. Because the pipeline used deterministic, centroid-anchored cropping, the target vehicle occupied the exact geometric centre of every training sample. As a result, the regression head ignored the sparse event stream and learned to output a constant coordinate vector instead. This bias was invisible during validation because the validation split inherited the same cropping method; it only appeared during full-frame inference, where the predicted box persistently anchored to the image centre regardless of the vehicle’s true dynamic position, as visually shown in Figure 3.7.

#### 4. Actionable Insight and Iteration

The *Constant Target Bias* failure proved that deterministic cropping eliminates the spatial variance required to train a regression head. To force the network to actively locate targets, the localised foveation had to be abandoned entirely. Consequently, Generation 1 (G1) was



(a) Train sample, IoU= 0.000.

(b) Train sample, IoU= 0.015.

(c) Test sample, IoU= 0.214.

Figure 3.7: Qualitative evidence of *Constant Target Bias* in G0. Across three raw full-frame samples drawn from the training and test splits, the SNN regression head (red) emits a near-identical large central bounding box while the ground-truth vehicle (green) varies in position and scale between scenes. The marginally higher overlap in (c) arises only because the target vehicle incidentally sits near the image centre where the predicted box anchors, confirming the model has learnt a constant coordinate shortcut rather than extracting vehicle features from the event stream.

designed to process the entire global frame, relying on aggressive spatial downsampling rather than cropping.

### 3.6.2 Generation 1 (G1): The Multi-Box Paradigm Shift

#### 1. Architectural Intent

After removing spatial cropping, the architecture progressed to full-frame global downsampling. The G1 prototype attempted to process this global frame using the standard single-box regression head (outputting a single  $[x, y, w, h]$  vector per frame).

The hypothesis underpinning the choice of a single-box regression head was driven by computational cost: for simple automotive perception using a focused, forward-facing camera, a single bounding box could theoretically encompass vehicles ahead to draw a simplified “critical safety region”. If successful, this would bypass the heavy computational overhead and complexity required by a multi-box detection head. In hindsight, this assumption was structurally flawed for the specific domain of autonomous driving and ADAS, which requires precise tracking of every vehicle independently for safe path planning.

#### 2. Empirical Result

The single-box model proved incapable of handling the multi-vehicle traffic scenes from the ETraM dataset, and unable to accurately draw the “critical region” so that all vehicles present were covered by one box. The model failed to converge on distinct targets, instead outputting a meaningless “super-box” that hovered in the empty space between vehicles or encompassed the entire roadway.

### 3. Diagnostic Analysis: Super-Box

This failure was determined to be a case of *Single-Target Ambiguity and Label Noise*. The deep backbone naturally extracted features for all visible vehicles in the global frame. However, because the regression head could only output one bounding box, the single-target label penalised the network if it correctly detected a secondary vehicle. To minimise the loss, the network averaged the spatial coordinates of all active spike clusters, producing the floating “super-box” artefact: a single bounding box that encompasses all vehicles and roadways rather than individual targets, as visually evidenced in Figure 3.8.

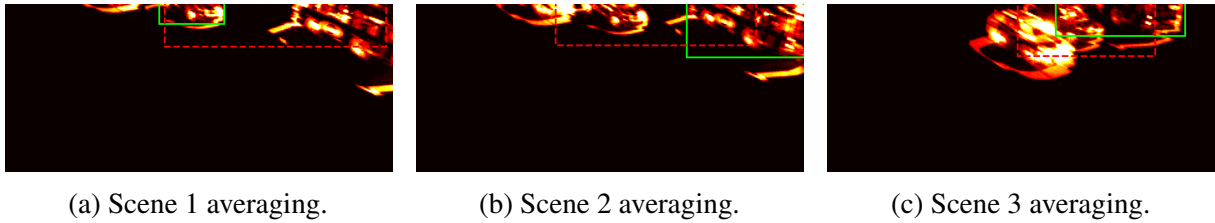


Figure 3.8: Qualitative evidence of Single-Target Ambiguity in Generation 1. Forced to output a single bounding box for multi-vehicle scenes, the network averages the spatial locations of all detected spike clusters. This results in a single, oversized bounding box (red) that hovers between vehicles or encompasses the entire lane, failing to localise the true individual targets (green).

## 4. Actionable Insight and Iteration

Generation 1 confirmed that a single global regression head is incompatible with uncropped automotive perception, which requires precise and isolated detection of all obstacles present in the scene. To resolve this, the network’s output topology shifted to a multi-box regression head; the output was expanded to a fixed-capacity  $[B, 5, 5]$  tensor, predicting 5 boxes per inference, each augmented with a fifth “Objectness” confidence score  $(x, y, w, h, c)$ . To support this, Generation 2 (G2) introduced a Vectorised Matching Loss to match predictions to their respective ground-truth annotations.

### 3.6.3 Generation 2 (G2): The Expanded Spatial Head (7.00M Parameters)

#### 1. Architectural Intent

As determined by the results and analysis of Generation 1, Generation 2 (G2) introduced the multi-box regression head. This replaced the global output vector with a fixed-capacity  $[B, 5, 5]$  tensor, predicting 5 boxes per inference alongside the aforementioned “Objectness” score  $(x, y, w, h, c)$ .

To train this new multi-box paradigm, G2 implemented the *Vectorised Matching Loss*. This algorithm computed a 3D matrix of IoU overlaps between the 5 predictions and  $M$  ground-truth

targets, using greedy matching to assign coordinate gradients. Unmatched predicted boxes were assigned a target confidence of 0.0 to define a penalty for false positives. To feed this new regression head, the deep backbone features were compressed using an Adaptive Average Pooling layer of  $4 \times 4$ .

## 2. Empirical Result

G2 successfully resolved the “super-box” error and learned to predict distinct, separate bounding boxes, however validation revealed a new failure: high false-positive rates and overlapping predictions. The network’s Average Detections per Frame (ADP) rose to 2.50, significantly higher than the true validation-set average of 2.02.

## 3. Diagnostic Analysis: Information Congestion

The hallucination of extra vehicles was determined to be *Information Congestion* within the regression head. Using the  $4 \times 4$  Adaptive Pooling layer compressed the field of view into just 16 spatial blocks, creating a severe spatial bottleneck. When vehicles were in close proximity, or near clusters of sensor noise, their respective spiking features were averaged into the same  $4 \times 4$  block; the regression head lacked the spatial memory required to distinguish legitimate target spike trains from adjacent background noise, leading it to output multiple overlapping boxes for a single stimulus.

## 4. Actionable Insight and Iteration

G2 proved that a multi-box head requires a finer structural map of backbone activations to perform Object-Noise-Discrimination, which isolates vehicle spike trains from surrounding sensor noise. To resolve this, the pooling resolution was quadrupled from  $4 \times 4$  to  $8 \times 8$ .

Initial prototyping of the wider pool size revealed a new limitation in the model’s architecture, with the Mean IoU plateauing at 41.1%. This confirmed that the backbone itself was downsampling the raw sensor data too aggressively, leading to the development of Generation 3 (G3), which expanded both the head’s pooling capacity and the backbone’s semantic depth.

### 3.6.4 Generation 3 (G3): The High-Resolution Backbone (11.72M Parameters)

#### 1. Architectural Intent

The objective of this final generation was to break the 41.1% mIoU precision ceiling established at the end of Generation 2. Analysis of the network’s internal spiking dynamics revealed two distinct flaws that constrained its geometric precision:

1. **Spatial Compression:** The G2 backbone downsampled the feature map to a resolution of  $11 \times 20$ . At this scale, a single pixel in the final feature map represented a massive  $16 \times 16$  area of the original HD sensor. As a result, the network was blind to fine-grained pixel shifts, since it lacked the geometric detail required to draw tight, accurate bounding boxes.
2. **Neural Silencing:** The G2 backbone collapsed to an average firing rate of  $< 1.5\%$ . This extreme sparsity starved the regression head of the temporal evidence needed for high-confidence predictions, preventing the network from fully utilising its 10-timestep observation window.

## 2. Actionable Implementation: Semantic Expansion and Awakening

To solve these bottlenecks, the architecture was overhauled to create the final G3 configuration:

- **Resolution Boost and Semantic Depth:** To provide the necessary geometric detail, the convolution stride in Block 3 was reduced to  $s = 1$ . This effectively halted the downsampling earlier in the network, doubling the final feature map resolution to  $22 \times 40$  (quadrupling the available spatial cells to 880). To handle the increased complexity and high-frequency sensor artefacts exposed by this higher resolution, the backbone’s semantic capacity was expanded via the addition of a 4th SEWResBlock (512 channels). This allowed the model to reason more deeply about the structural composition of the vehicle.
- **Metabolic Regularisation:** The LIF firing threshold was lowered ( $V_{th} = 0.5 \rightarrow 0.2$ ) to wake up the silent backbone, making it significantly easier for neurons to activate. Concurrently, the FRR penalty multiplier ( $\lambda$ ) was also increased ( $1.0 \rightarrow 5.0$ ) with a target rate of 20%. This forced the network to generate a dense, high-frequency stream of geometric data, providing the regression head with the robust temporal evidence needed to sense the exact boundaries of the vehicles.
- **Hardware Optimisation:** Expanding the parameter count to 11.7M and quadrupling the spatial resolution substantially increased the computational load. To prevent latency penalties, the architecture was refactored to utilise SpikingJelly’s `step_mode='m'` (*Fused-Kernel optimisation*) [13]. Rather than using a slow, sequential Python loop to evaluate the network across  $T = 10$  timesteps, this method passes the entire spatiotemporal tensor directly to custom CUDA kernels, allowing the temporal dynamics to be executed in parallel on the GPU. This was coupled with AMP utilising 16-bit tensor cores, resulting in a  $\sim 1.8\times$  training speedup and making the G3 configuration computationally feasible to develop.

## 3. Final Empirical Result

The G3 overhaul successfully broke the precision ceiling. By preserving spatial resolution and enforcing metabolic activity, the network achieved a peak validation mIoU of 46.1% and

mAP<sub>50</sub> of 59.8% (detailed fully in Chapter 4). This architecture was consequently locked as the definitive `DetectorNX-G3-SNN` model used for all final hardware extrapolations and baseline comparisons.

### 3.6.5 Generations: Summarised

To summarise, Table 3.5 maps the complete empirical evolution of the `DetectorNX` architecture, tracing the resolution of each major failure mode across the four generations.

| Gen.      | Primary Architectural Shift                                                | Params | mIoU               | Empirical Breakthrough                                                                      | Diagnosed Limitation                                                           |
|-----------|----------------------------------------------------------------------------|--------|--------------------|---------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <b>G0</b> | Deterministic $224 \times 224$ Spatial Cropping                            | N/A    | 1.37% <sup>‡</sup> | Preserved high SNR via localised foveation.                                                 | <i>Constant Target Bias</i> : Complete collapse of spatial variance.           |
| <b>G1</b> | Full Frame Downsampling with Single-Box Head                               | 5.43M  | 20.4%              | Restored global spatial variance, breaking the target bias.                                 | <i>Single-Target Ambiguity</i> : Averaged targets into floating “Super-Boxes”. |
| <b>G2</b> | Multi-Box $[B, 5, 5]$ Topology with $8 \times 8$ Spatial Pool              | 7.00M  | 41.1%              | Resolved Information Congestion, dropping ADP to 2.06.                                      | <i>Geometric Blindness</i> : Heavy downsampling enforced a 41.1% mIoU ceiling. |
| <b>G3</b> | $22 \times 40$ Backbone Resolution, 4th SEWResBlock and FRR Regularisation | 11.72M | 46.1%              | Broke the precision ceiling of G2, achieving the highest mIoU and mAP <sub>50</sub> scores. | <i>Trade-off</i> : Significant increase in parameter and SOP count vs. G2.     |

Table 3.5: Empirical evolution of the `DetectorNX` architecture. Each generation was designed to resolve the diagnosed limitation of its predecessor. <sup>‡</sup> This result was collected using a separately-created test set, whereas all others use the stratified and sampled validation set.

## 3.7 Training Structure

This section specifies how the `DetectorNX-G3-SNN` architecture established in Section 3.4 is trained on the precomputed voxel chunks produced by the BOLT pipeline (Section 3.3). Each hyperparameter choice addresses a specific failure mode encountered during architectural



evolution (Section 3.6), and the configuration summarised here is the one used to train the model benchmarked in Section 3.8.

### 3.7.1 Objective Function

The total training loss combines a vectorised multi-box loss with the firing rate regularisation introduced in Section 3.4.4:

$$L_{total} = L_{coord} + L_{conf} + L_{frr} \quad (3.2)$$

The multi-box loss operates on the  $[B, 5, 5]$  head output. A broadcast IoU matrix of shape  $[B, K, M]$  is computed between the  $K = 5$  predicted boxes and the  $M = 5$  padded ground-truth slots, and each ground-truth slot is independently paired with its highest-IoU prediction via `iou.argmax(dim=1)`. Each matched pair contributes a combined coordinate term

$$L_{coord} = \text{SmoothL1}(\hat{\mathbf{b}}, \mathbf{b}) + \lambda_{giou} \cdot (1 - \text{GIOU}(\hat{\mathbf{b}}, \mathbf{b})) \quad (3.3)$$

with  $\lambda_{giou} = 2.0$ . SmoothL1 supplies the per-coordinate gradient; the Generalised Intersection over Union (GIOU) term contributes a scale-invariant geometric signal that remains non-zero on disjoint boxes, avoiding the vanishing-gradient problem of raw IoU [32]. The confidence term applies `BCEWithLogitsLoss` to the raw logits, targeting 1.0 for the slot matched to each real object and 0.0 elsewhere. The fused sigmoid/BCE formulation is numerically stable under the float16 autocast described in Section 3.7.2.

The FRR term is a symmetric squared penalty across all nine spike tensors emitted by the backbone (the stem LIF output plus two per `SEWResBlock`):

$$L_{frr} = \frac{\lambda_{frr}}{N} \sum_{i=1}^N (r_i - r^*)^2 \quad (3.4)$$

with target rate  $r^* = 0.20$  and  $\lambda_{frr} = 5.0$ . The spike tensors remain in the autograd graph, so FRR gradients propagate backward through the `ATan` surrogate gradient and adjust backbone convolution weights directly, making regularisation an active training signal rather than a post-hoc diagnostic.

### 3.7.2 Optimisation

Parameters are updated with Adam [24] at  $\eta = 2 \times 10^{-4}$  and weight decay  $10^{-4}$ . The learning rate is controlled by `ReduceLROnPlateau` (factor 0.5, patience 15 epochs) operating on an exponentially smoothed validation IoU:

$$\text{EMA}_t = \beta \cdot \text{EMA}_{t-1} + (1 - \beta) \cdot \text{IoU}_t, \quad \beta = 0.9 \quad (3.5)$$

Scheduler steps are gated by  $\text{EMA} \geq 0.10$ , which prevents premature decay while the model is still below baseline localisation competence.

Training runs under `torch.amp.autocast('cuda')` with a `GradScaler`, yielding a  $\sim 1.8\times$  throughput gain and halved activation memory on the NVIDIA A100 tensor cores. After loss scaling but before the optimiser step, gradients are unscaled and clipped to  $\ell_2$  norm 1.0 to contain the heavy gradient tail produced by the eight-deep surrogate path through the four `SEWResBlock` pairs.

### 3.7.3 Validation and Checkpointing

After each training epoch the model is evaluated across the full validation split. Predictions are sigmoid-activated, real objects are identified by  $\text{mask} > 0.5$ , and a detection is counted as active when its confidence logit clears the same threshold. The per-epoch metric is the mean of the best IoU achieved against each real object across the active predictions. A best-model checkpoint is written whenever this raw IoU surpasses the current maximum.

### 3.7.4 Hyperparameter Summary

Table 3.6 consolidates the trained configuration.

Table 3.6: Training configuration for `DetectorNX-G3-SNN`.

| Group        | Parameter            | Value              |
|--------------|----------------------|--------------------|
| Input        | $T$ (timesteps)      | 10                 |
|              | Batch size           | 8                  |
| Neuron (LIF) | $\tau$               | 4.0                |
|              | $V_{th}$             | 0.2                |
|              | Surrogate            | ATan               |
| Head         | Adaptive pool        | $8 \times 8$       |
|              | Head channels        | 128                |
| Loss         | $\lambda_{giou}$     | 2.0                |
|              | $\lambda_{frr}$      | 5.0                |
|              | $r^*$ (target rate)  | 0.20               |
| Optimiser    | Adam $\eta$          | $2 \times 10^{-4}$ |
|              | Weight decay         | $10^{-4}$          |
|              | LR factor            | 0.5                |
|              | LR patience (epochs) | 15                 |
|              | EMA $\beta$          | 0.9                |
|              | IoU gate             | 0.10               |
| Stability    | Grad clip $\ g\ _2$  | 1.0                |
|              | AMP                  | CUDA float16       |
| Training     | Epochs               | 150                |

## 3.8 Experimental Setup and Benchmarking

Having established the theoretical design and optimisation of the `DetectorNX` architecture, both the SNN and its CNN counterpart, the final step is to formulate a rigorous evaluation strategy to empirically validate its performance against the continuous CNN baseline. In accordance with the project’s primary objectives, the evaluation cannot be limited to a singular metric of bounding box accuracy; rather, it must holistically assess the neuromorphic paradigm. To achieve this, a comprehensive benchmarking suite was designed (as mentioned in Figure 3.1) covering seven distinct experimental protocols:

- Dynamic Power Estimation
- Quantisation Error Analysis (Accuracy vs. Object Size)
- Confidence Calibration
- Temporal Evidence Accumulation
- Real-time Inference latency
- Theoretical Energy Consumption
- Global Evaluator: Mean/Median IoU, mAP<sub>50</sub>, Recall, and Average Detections Per Frame

### 3.8.1 Dynamic Power Estimation

A fundamental objective of this research is to quantify the energy efficiency gains achieved by transitioning from a continuous CNN to a bio-plausible SNN. Traditional CNN architectures are characterised by static power consumption; they perform a dense, fixed number of MAC operations for every forward pass, regardless of the information density within the input scene. In contrast, SNNs leverage event-driven dynamics, where metabolic energy is consumed only when a neuron reaches its firing threshold and generates a discrete spike.

The Dynamic Power experiment investigates the reactive scalability of the architecture by mapping instantaneous energy consumption against the temporal density of the input stream. This analysis serves to validate the event-driven nature of the SNN, demonstrating that its metabolic cost is a dynamic function of activation sparsity—mimicking biological neural efficiency by consuming energy only in response to informative stimuli.

#### CNN Power Estimation

In a standard CNN, the total energy per frame ( $E_{CNN}$ ) is calculated by multiplying the total number of Floating-Point Operations (FLOPs) by the cost of a 32-bit MAC ( $E_{MAC}$ ):

$$E_{CNN} = \sum_{l=1}^L (FLOPs_l \times E_{MAC}) \quad (3.6)$$

Where  $E_{MAC} = 4.6$  pJ per 32-bit floating point operation, as referenced in Table 3.7 [19].

The number of FLOPs is the total number of MACs operations required to produce the output feature map; in a CNN, this number is static, regardless of what the image contains. The FLOPs of a CNN can be calculated as follows:

$$FLOPs_l = (H_{out} \times W_{out}) \times (K \times K) \times C_{in} \times C_{out} \quad (3.7)$$

Where  $(H_{out} \times W_{out})$  is the number of pixels in the output frame,  $(K \times K)$  is the size of the convolutional kernel,  $C_{in}$  is the number of input channels, and  $C_{out}$  is the number of output channels.

### SNN Power Estimation

For the spiking architecture, the calculation is scaled by the activation sparsity ( $\zeta$ ) of each layer. The metabolic cost of the spiking backbone can be quantified via the SOP metric; unlike the static complexity of the CNN baseline, the SOP count for a given layer is a dynamic function, which can be calculated as follows:

$$SOPs_l = FLOPs_l \times \zeta \times T \quad (3.8)$$

where  $\zeta_l$  represents the average firing rate of layer  $l$ ,  $T$  is the number of temporal simulation steps ( $T = 10$ ).

In order to obtain  $\zeta_l$ , a specialised `SpikeRateMonitor` tool was developed. The tool attaches a hook to every LIF inside the SNN backbone, which listens to the layer during the forward pass and intercepts the binary spike tensor before it moves to the next layer. For every layer  $l$ , the SNN produces a spike tensor  $S_l$  with the shape  $[T, B, C, H, W]$ , where  $T = 10$  is the number of timesteps,  $B$  is the batch size, and  $C, H, W$  are the spatial dimensions. Since a spike is binary (1 for fire, 0 for rest),  $\zeta$  can be calculated by taking the global mean of the entire tensor:

$$\zeta_l = \frac{1}{N} \sum_{i=1}^N S_{l,i} \quad (3.9)$$

To calculate Dynamic Power,  $\zeta$  was calculated by running inference over all validation samples and taking the mean firing rate per layer across all samples in the dataset.

Because SNNs utilise binary activations, the expensive multiplication part of the MAC operation is omitted, leaving only a AC of the weight to the membrane potential. The total energy per frame ( $E_{SNN}$ ) is defined as:

$$E_{SNN} = \sum_{l=1}^L (SOP_{s_l} \times E_{AC}) \quad (3.10)$$

where  $E_{AC} = 0.9$  pJ is the energy cost of a single floating-point addition, as referenced in Table 3.7 [19].

### 3.8.2 Quantisation Error Analysis (Accuracy vs. Object Size)

The Quantisation Error Analysis experiment empirically tests the  $1/T$  quantisation ceiling introduced in Section 2.3.3: it measures the extent to which the SNN’s finite representational states limit regression precision on objects of differing spatial scales, compared against the continuous 32-bit CNN baseline which is not subject to this ceiling.

#### Core Metrics and Scale Partitioning

The primary performance metric for this experiment is the mIoU, which quantifies the overlap between the predicted bounding box ( $P$ ) and the ground truth ( $GT$ ). To analyze the impact of scale on regression precision, the validation set is partitioned into three distinct categories based on the absolute pixel area ( $A_{px}$ ) of the ground truth objects:

- **Small:** Objects in the bottom 33rd percentile of area (typically distant vehicles or small artefacts).
- **Medium:** Objects in the middle 33rd percentile.
- **Large:** Objects in the top 33rd percentile (vehicles in close proximity to the sensor).

#### Mathematical Framework

The categorisation and evaluation are governed by two fundamental geometric formulas. First, the pixel area is calculated by denormalising the ground truth dimensions relative to the raw sensor resolution:

$$A_{px} = (W_{norm} \cdot W_{sensor}) \times (H_{norm} \cdot H_{sensor}) \quad (3.11)$$

where  $W_{sensor} = 1280$  and  $H_{sensor} = 720$ . Subsequently, the precision of the model’s spatial regression is quantified via the IoU formula:

$$IoU = \frac{\text{Area}(P \cap GT)}{\text{Area}(P \cup GT)} \quad (3.12)$$

#### Hypothesis: The Scale-Dependent Accuracy Gap

We hypothesise a significant scale-dependent accuracy gap between the two architectures. For large-scale objects, the SNN is expected to achieve parity with the CNN baseline: because the

coordinate values are high, the minor fluctuations inherent in binary spike-trains represent a negligible percentage of the total object area, allowing for stable regression.

Conversely, for small-scale objects, a substantial performance divergence is anticipated. For the  $T = 10$  configuration used here, this ceiling resolves to eleven discrete states ( $0/10, 1/10, \dots, 10/10$ ). As documented by [3], this rounding error, or quantisation penalty, is massive relative to the dimensions of small vehicles. If a target requires a precise coordinate gradient (e.g., 0.154) for accurate localization, the SNN is physically forced to discretise this value, establishing a fundamental “resolution floor” that limits mIoU on small features. While the use of SEWResBlock architectures improves signal flow [12], it does not inherently resolve this quantisation of the final regression output, which is further exacerbated by the low event counts typical of small objects in event-based vision [14].

### 3.8.3 Confidence Calibration

The Confidence Calibration evaluates predictive reliability by investigating the calibration gap between spiking and continuous architectures. Modern CNNs are prone to systematic overconfidence, frequently assigning high objectness scores to inaccurate detections [17]. Conversely, the temporal dynamics of SNNs may offer superior uncertainty quantification, providing a more robust reliability profile for safety-critical automotive tasks [36].

#### Methodology: Reliability Diagrams

To quantify calibration, we generate a Reliability Diagram mapping predicted confidence ( $C$ ) against actual regression accuracy ( $IoU$ ) for every validation detection. In a perfectly calibrated system, these values align along the identity line ( $C = IoU$ ). Clumping in the high-confidence, low-accuracy quadrant indicates an overconfidence bias, where the model fails to signal its own predictive errors.

#### Hypothesis: Spikes as Uncertainty Sensors

The CNN baseline is hypothesised to exhibit significant miscalibration due to its continuous activation functions and exact gradient descent, which tend to overfit the confidence head. The SNN is expected to demonstrate a more linear, calibrated trend. In the spiking domain, low-quality features generate fewer spikes, leading to lower membrane potentials in the detection head. This mechanism allows the SNN to function as an implicit uncertainty sensor, where discrete spike rates effectively gate the output confidence logit relative to the strength of the temporal evidence.

### 3.8.4 Temporal Evidence Accumulation

The Temporal Evidence Accumulation experiment investigates the sequential decision-making dynamics inherent to the SNN architecture. Unlike the CNN baseline, which performs a single spatial pass over the fully integrated 100ms voxel, the SNN processes the event stream iteratively across  $T = 10$  timesteps. This property makes SNNs intrinsically suited to low-latency spatiotemporal integration — the capacity to produce meaningful predictions before the full observation window has elapsed [30]. The experiment therefore examines whether this theoretical advantage manifests empirically: specifically, whether the model can produce a valid bounding box regression before the 100ms window is complete.

In the context of vehicle object detection, this is a highly significant experiment — for example, if the SNN can accurately show a bounding box at  $t = 5$  (50ms) instead of the full  $t = 10$  (100ms) as the CNN would, this would lead to a 50% reduction in detection time. In a time-sensitive domain such as vehicle and pedestrian detection, such advantages could be the difference between a crash and a safe stop.

#### Methodology: Temporal Snapshots

To evaluate the evolution of predictions, we extract intermediate outputs at three specific checkpoints:  $t = 2$  (20ms),  $t = 5$  (50ms), and  $t = 10$  (100ms). For each snapshot, the candidate bounding box coordinates and instantaneous confidence scores are recorded to observe the *hunting behaviour* of the regression head: where the network’s predicted bounding box moves, vibrates or searches for the object’s true edges before finally settling into a stable, high-precision detection. This process is driven by the discrete LIF dynamics:

$$v_i[t] = v_i[t-1] + \frac{1}{\tau} \left( \sum_j w_{ij} s_j[t] - (v_i[t-1] - V_{reset}) \right) \quad (3.13)$$

where  $v_i[t]$  represents the accumulated evidence (membrane potential) and  $\sum w_{ij} s_j[t]$  represents the incoming observations (spikes). This recursive filtering allows the network to update its belief about vehicle localization with every incoming pulse of data.

#### Hypothesis: Bounding Box Sharpening

We hypothesise a phenomenon of *Bounding Box Sharpening* over time, consistent with the rapid categorisation principle established in Section 2.3.3. At  $t = 20$ ms, we anticipate high coordinate variance, as the majority of LIF neurons will not have accumulated sufficient membrane potential to cross the firing threshold ( $V_{th} = 0.2$ ) within the first two of ten timesteps. As  $t$  progresses toward 100ms, continued spike train integration allows stochastic membrane noise to average out, consolidating responses to the vehicle’s geometric edges into a stable coordinate estimate.

### 3.8.5 Real-Time Latency Benchmark

This experiment quantifies the forward-pass inference speed of both architectures on conventional GPU hardware at batch size  $B = 1$ , simulating a live single-stream event camera deployment. Latency (ms/f) and throughput (FPS = 1000/ms/f) are measured using `torch.cuda.Event` timing to capture true GPU execution time, bypassing Central Processing Unit (CPU)-to-GPU communication overhead.

#### Hypothesis

The CNN baseline is expected to exhibit lower latency, as GPUs are optimised for the dense matrix multiplications that underpin its single flattened spatial pass. The SNN must instead simulate LIF membrane dynamics iteratively:

$$v[t] = v[t - 1] + x[t] - \beta \cdot v[t - 1] \quad (3.14)$$

This constitutes a hardware-mismatch penalty: the latency advantage of SNNs only fully materialises on asynchronous neuromorphic hardware, not on synchronous GPU accelerators [4, 1]. The experiment aims to quantify this penalty and assess whether `step_mode='m'` parallelisation [13] brings the SNN within a  $2\times$ – $3\times$  latency margin of the CNN.

#### Limitations

The measured GPU latency figures cannot be extrapolated to neuromorphic silicon. The A100’s 312 TFLOPS (FP16) characterises dense matrix throughput, which cannot be judged against asynchronous spike-based execution. While Loihi benchmarks report up to  $10\times$  lower latency over embedded GPUs for batch-size-1 inference [4], these figures are task-specific; direct deployment on neuromorphic hardware is identified as future work.

### 3.8.6 Theoretical Energy Consumption

The Theoretical Energy Consumption is very similar in nature to the Dynamic Power experiment discussed in Section 3.8.1. It uses the same principles and formulae for SOPs and FLOPs, but differs in the final calculation: as opposed to calculating the dynamic power consumption per frame, this experiment aims to find the average power consumption per frame across the entire dataset, and extrapolates for multiple hardware platforms:

$$\bar{E}_{SNN} = \sum_{l=1}^L (SOPs_l \times E_{chip}) \quad (3.15)$$

$$\bar{E}_{CNN} = \sum_{l=1}^L (FLOPs_l \times E_{MAC}) \quad (3.16)$$



The calculation mirrors Equation 3.10, however  $E_{SNN}$  replaces  $E_{AC}$  for  $E_{chip}$ , which is the power consumed per SOP for different chipsets as depicted in Table 3.7.

Subsequently, the total percentage energy saving is formulated using the following equation:

$$\text{Energy Savings} = \left(1 - \frac{\bar{E}_{SNN}}{\bar{E}_{CNN}}\right) \times 100\% \quad (3.17)$$

### Energy Constants Per Operation ( $E_{chip}$ ) Across Multiple Platforms

| Architecture / Platform   | Operation Type | Energy ( $E_{chip}$ ) | Primary Citation |
|---------------------------|----------------|-----------------------|------------------|
| CNN Baseline              | MAC            | 4.6 pJ                | [19]             |
| Theoretical Silicon Limit | AC             | 0.9 pJ                | [19]             |
| Intel Loihi 2             | SOP            | 16.1 pJ               | [16]             |
| Intel Loihi 1             | SOP            | 23.6 pJ               | [4]              |
| IBM TrueNorth             | SOP            | 26.0 pJ               | [28]             |
| NVIDIA DGX A100 (FP32)    | FLOP           | 19.6 pJ               | [41]             |

Table 3.7: Energy constants per operation ( $E_{chip}$ ) for each architecture and platform.

### Hypothesis

We hypothesise that the SNN will demonstrate a metabolic advantage of at least one order of magnitude ( $> 90\%$ ) over the CNN baseline, predicated on two complementary properties of the  $\bar{E}_{SNN}$  formulation:

1. **Activation sparsity:** By exploiting the sparsity of the ETraM event stream, the backbone is expected to maintain  $\zeta < 5\%$ , bypassing the majority of computations that the CNN performs densely regardless of scene activity.
2. **Mathematical simplification:** Replacing MAC operations with binary AC operations yields a  $5\times$  reduction in per-operation energy cost (0.9 pJ vs 4.6 pJ), which is expected to provide a substantial efficiency dividend even when accounting for the  $T = 10$  temporal overhead [19].

When extrapolated to physical neuromorphic platforms, the SNN’s event-driven resting state is expected to yield an exponential power advantage over synchronous GPU accelerators. With Intel Loihi 2, the neuromorphic paradigm benefits from both extreme activation sparsity and a lower per-operation cost (16.1 pJ per SOP) compared to the per-FLOP cost of the benchmark NVIDIA A100 (19.6 pJ) documented in Table 3.7 [16, 28, 41].

### 3.8.7 Global Performance Evaluator

The Global Performance Evaluator serves as the primary comparative evaluation between the SNN and CNN architectures, aggregating performance across the full 200-sample validation set. Unlike the preceding targeted experiments, this benchmark establishes the *Performance-Efficiency Frontier*: quantifying the accuracy penalty incurred by transitioning to discrete spiking dynamics, whilst validating whether the multi-box refinements and others in the G3 revision of `DetectorNX` mentioned in Section 3.6 are sufficient to resolve scene-level detection ambiguity.

#### Metrics

In this experiment, four complementary metrics are evaluated:

- **mIoU**: measures geometric precision between predicted and ground truth bounding boxes.
- **mAP<sub>50</sub>**: the COCO-convention detection-quality metric, measuring average precision at a fixed 0.5 IoU threshold; enables external comparison against published detectors and validates that mIoU parity is not an artefact of the overlap-only evaluation.
- **Recall**: measures the proportion of ground truth objects successfully localised at a confidence threshold of 50%, penalising architectures that fail to respond to sparse event regions.
- **Average Detections Per Frame (ADP)**: quantifies detector selectivity relative to the ground truth mean of 2.02 objects per frame; significant positive deviation indicates spurious “ghost” detections arising from background noise misclassification.
- **Mean Prediction Confidence**: captures network self-certainty by averaging sigmoid-activated confidence scores across all positive detections, probing whether spiking dynamics produce decisive or hesitant class activations.

#### Hypothesis

We hypothesise that whilst the CNN establishes the upper mIoU bound, the SNN will achieve *detection parity* in recall and ADP, yielding an average detection count consistent with the ground truth distribution. This would demonstrate that spiking temporal dynamics are sufficient for objectness discrimination without continuous-valued feature maps [33, 3]. A measurable confidence gap is also anticipated: the SNN’s binary spike outputs are expected to produce lower but temporally stable confidence distributions across the  $T = 10$  timestep window, consistent with prior observations that surrogate-trained networks distribute representational energy across timesteps rather than single forward passes [12].

**Significance**

This experiment provides three contributions. It delivers the *parity proof*, framing the accuracy-efficiency tradeoff quantitatively. It offers *logical validation* that the Phase 1 ambiguity problem has been resolved through the architectural refinements described in Section 3.6. Finally, evaluation across the full validation set ensures statistical robustness, confirming that results are representative of the broader ETraM environment rather than artefacts of favourable individual frames.

# Chapter 4

## Results and Evaluation

Having established a rigorous, multi-faceted benchmarking methodology in Section 3.8, this final section presents the empirical evaluation of the G3 generation of the `DetectorNX` architecture against its CNN counterpart. The following results were conducted on the pre-processed `BOLT-10K-v2` dataset snapshot, using a validation split to ensure strict comparative integrity.

Rather than relying on a singular metric of spatial accuracy, the following analysis systematically deconstructs the performance-efficiency frontier of the neuromorphic paradigm. By sequentially examining global detection parity, energy consumption, quantisation-induced scaling errors, and predictive calibration, this evaluation aims to quantify the precise trade-offs inherent in transitioning from 32-bit floating-point convolutions to 1-bit, event-driven spiking dynamics for high-definition automotive perception.

### 4.1 Runtime Environment

The following experiments were run using the following model and data configuration:

- **Model:** The G3 generation of the `DetectorNX` model architecture was used for both SNN and CNN experiments, as described in Section 3.6.
- **Dataset:** A stratified-sampling approach was used to sample the full `ETraM` dataset, in order to ensure that all scenes (day and night) were represented using the same proportions. For training and validation, 10,000 samples were selected.
- **Precomputing:** The `BOLT-10K-v2` dataset snapshot was formed by applying the `BOLT` (Section 3.3) pipeline to the sampled 10,000-sample dataset.
- **Training:** The `DetectorNX` model was trained on the `BOLT-10K-v2` dataset using an 80% training data split.
- **Evaluation:** The results of the following experiments were collected using a 200-sample subset from the validation split of the `BOLT-10K-v2` dataset, using the pre-trained `DetectorNX`

model.

- **Compute:** The University of Manchester’s HPC was used for preprocessing, training and evaluation. The environment was set up using 12 CPU cores, 120 GB RAM, and one NVIDIA A100 (80 GB) GPU.

## 4.2 Architectural Progression: Validation Evidence

The `DetectorNX-G3` configuration was selected as the primary architecture for all subsequent experiments in this chapter. To justify this choice, Table 4.1 and Figure 4.1 present the validation mIoU improvements across the four design generations documented in Section 3.6.

| Generation | Primary Architectural Shift                                 | Params | Validation mIoU    |
|------------|-------------------------------------------------------------|--------|--------------------|
| G0         | Deterministic $224 \times 224$ cropping                     | N/A    | 1.37% <sup>‡</sup> |
| G1         | Full-frame, single-box head                                 | 5.43M  | 20.4%              |
| G2         | Multi-box $[B, 5, 5]$ , $8 \times 8$ pool                   | 7.00M  | 41.1%              |
| G3         | $22 \times 40$ backbone, 4th <code>SEWResBlock</code> , FRR | 11.72M | 46.1%              |

Table 4.1: Validation mIoU achieved by each design generation of `DetectorNX`. Each generation was developed to resolve the diagnosed limitation of its predecessor, as documented in Section 3.6. <sup>‡</sup> G0 was measured on a separately-created test set, since the validation split inherited the same cropped-input distribution as training; all other generations report metrics on the stratified validation subset of `BOLT-10K-v2`.

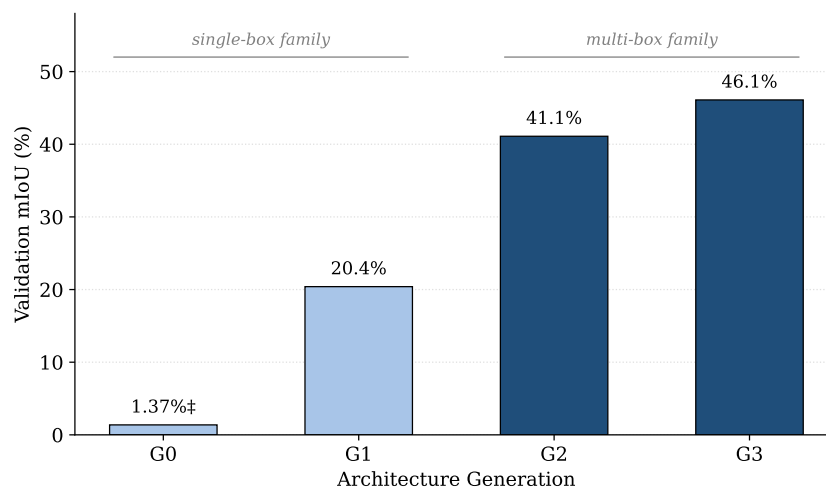


Figure 4.1: Validation mIoU trajectory across the four design generations of `DetectorNX`. The single-box-to-multi-box transition ( $G1 \rightarrow G2$ ) delivered the largest gain of +20.7 pp, while the high-resolution-backbone iteration ( $G2 \rightarrow G3$ ) resulted in a further +5.0 pp gain, leading to the final 46.1% score. G0 is shown for completeness; its measurement strategy differs as documented in Table 4.1.

The progression confirms that each architectural transition delivered a measurable improvement over its predecessor, with no plateau or regression at any step. The largest single-step improvement (+20.7 pp, G1  $\rightarrow$  G2) comes from the introduction of the multi-box regression head, and the final gain (+5.0 pp, G2  $\rightarrow$  G3) reflects breaking the geometric-precision ceiling identified at the end of G2.

Validation mIoU serves as the primary cross-generation metric, since it is the only quantitative measure collected across all four generations. Additional generation-specific metrics corroborate the progression at finer granularity: the mean firing rate across all layers (G2:  $< 1.5\%$ , G3:  $\sim 7.9\%$  mean post-FRR), and the documented failure modes (Constant Target Bias in G0; Super-Box artefact in G1) are presented in Section 3.6.

These results justify the selection of G3 as the canonical `DetectorNX-G3-SNN` architecture used for all subsequent baseline comparisons, energy benchmarking, and confidence-calibration analyses presented in the remainder of this chapter.

### 4.3 Global Evaluator

This global overview defines the baseline from which all subsequent power and precision trade-offs are derived.

#### Quantitative Results

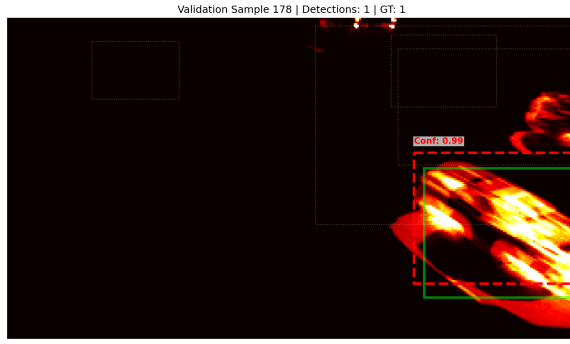
Table 4.2 summarises the headline results for the G3 generation of the `DetectorNX` model architecture against its CNN counterpart.

| Performance Metric               | <code>DetectorNX-G3-SNN</code> | <code>DetectorNX-G3-CNN</code> |
|----------------------------------|--------------------------------|--------------------------------|
| Mean IoU                         | 0.4613                         | 0.4861                         |
| mAP <sub>50</sub>                | 0.5977                         | 0.6550                         |
| Recall (Detections $> 0.5$ )     | 100.0%                         | 100.0%                         |
| Avg. Detections per Frame (GT)   | 2.02                           | 2.02                           |
| Avg. Detections per Frame (Pred) | 1.73                           | 1.75                           |
| Max Confidence                   | 0.9930                         | 0.9918                         |
| Min Confidence                   | 0.0011                         | 0.0132                         |

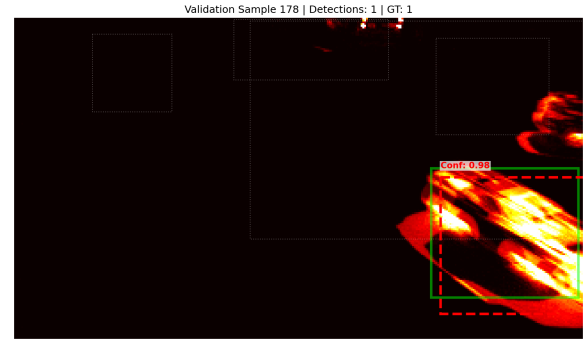
Table 4.2: Global Performance Benchmark results for `DetectorNX-G3-SNN` and `DetectorNX-G3-CNN`, evaluated across the full 200-sample ETraM validation set [42]. Ground truth average detections per frame is 2.02 for both models by definition.

## **Qualitative Results**

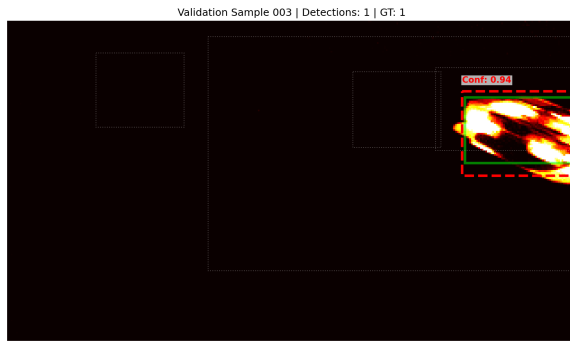
A qualitative audit was also conducted to visualise the models’ spatial reasoning across the validation subset. Figure 4.2 illustrates the architectures’ ability to resolve vehicle geometries in varied traffic conditions, providing the empirical context for the subsequent critical analysis of SNN precision. An expanded set of 20 paired samples is provided in Appendix A.3, with the full 200-sample audit available via the link.



(a) CNN Baseline - Sample 178



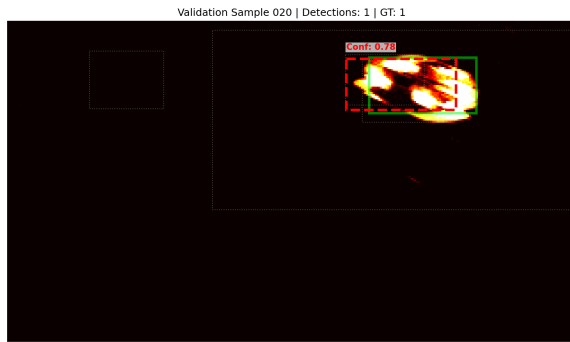
(b) SNN (G3) - Sample 178



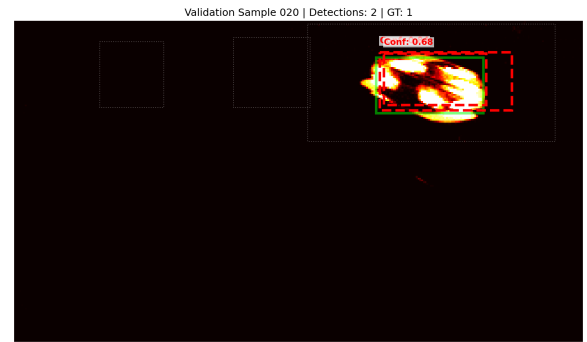
(c) CNN Baseline - Sample 003



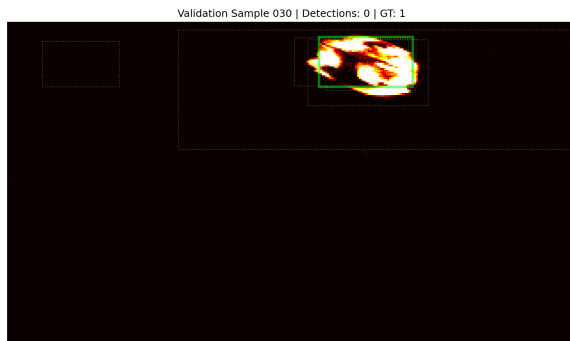
(d) SNN (G3) - Sample 003



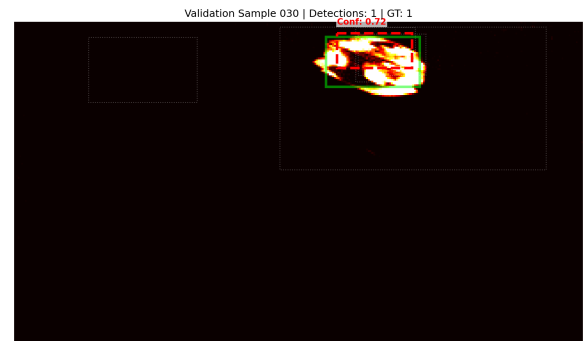
(e) CNN Baseline - Sample 020



(f) SNN (G3) - Sample 020



(g) CNN Baseline - Sample 030



(h) SNN (G3) - Sample 030

Figure 4.2: Qualitative comparison of bounding box regression between the CNN baseline and SNN (G3) across representative validation samples.



## Critical Analysis

The results from Table 4.2 demonstrate a remarkable degree of parity between the spiking and continuous paradigms. Achieving a mean IoU of 0.4613, which corresponds to 94.9% of the CNN baseline, proves that the G3 backbone is robust enough to sustain high-resolution spatial reasoning despite the loss of precision inherent in 1-bit quantisation. This geometric-precision parity is further corroborated by the detection-quality metric: the SNN achieves an mAP<sub>50</sub> of 0.5977 against the CNN’s 0.6550, a 91.3% parity under the stricter COCO-convention protocol that confirms the parity finding is not an artefact of the IoU-only evaluation. This parity indicates that the architectural refinements detailed in Section 3.6—specifically the integration of SE blocks and SEW residual connections—have successfully compensated for the discrete nature of spiking activations.

This convergence in predictive capacity is further reflected in the detection metrics. Both models achieve a perfect recall of 100.0% for objects with an IoU > 0.5, demonstrating that the transition to spiking dynamics has not induced any catastrophic forgetting of vehicle features. Furthermore, both models exhibit a similar detection profile compared to the ground truth (1.73 and 1.75 predicted vs. 2.02 ground truth average detections), which is typical for object detection on noisy, asynchronous event data. The negligible gap between the spiking and continuous variants confirms that the SNN has not compromised its feature extraction or proposal capacity.

Furthermore, the confidence distribution analysis provides the first indication of representational divergence. The SNN exhibits a lower minimum confidence (0.0011 vs. 0.0132) compared to the baseline, while maintaining a competitive maximum confidence (0.9930 vs. 0.9918). This suggests that the spiking model utilises a wider dynamic range of membrane potentials to differentiate between high-evidence targets and background noise. The ability of the SNN to suppress low-evidence detections more effectively than the continuous baseline points toward a more honest internal representation, a hypothesis further investigated in the calibration analysis (Section 4.7).

Qualitatively, Figure 4.2 confirms that the SNN maintains precise spatial alignment across diverse traffic densities. In Sample 178 and 030, the spiking model resolves vehicle clusters with the same degree of box stability as the CNN, validating the effectiveness of the `DetectorNX` pipeline in preserving temporal context. In summary, the `DetectorNX-G3-SNN` has successfully bridged the precision gap, establishing a foundation for the subsequent evaluation of its power efficiency and temporal reliability.

## 4.4 Dynamic Power

### Spike Activation Per Layer

Table 4.3 shows the mean activation rates ( $\zeta$ ) of each spiking layer inside the SNN backbone, aggregated across the 200-sample validation subset. These layer-wise activation values quantify

the metabolic sparsity of the network and were subsequently used to calculate the dynamic power consumption (SOPs) for each frame in the dataset.

| Layer Name  | Mean Firing Rate ( $\zeta$ ) |
|-------------|------------------------------|
| stem_lif    | 20.19%                       |
| block1.lif1 | 17.93%                       |
| block1.lif2 | 19.22%                       |
| block2.lif1 | 0.19%                        |
| block2.lif2 | 11.11%                       |
| block3.lif1 | 0.19%                        |
| block3.lif2 | 0.47%                        |
| block4.lif1 | 0.58%                        |
| block4.lif2 | 1.32%                        |

Table 4.3: Mean spike activation rates ( $\zeta$ ) per layer for the `DetectorNX-G3-SNN` backbone over the 200-sample experimental set.

## Mean Energy Savings

By substituting the empirical layer-wise firing rates ( $\zeta$ ) recorded in Table 4.3 into the dynamic power formulation established in Equation 3.10 (Section 3.8.1), the network’s computational operations can be directly translated into theoretical energy consumption in microjoules ( $\mu J$ ). This mathematical translation reveals a substantial metabolic advantage for the neuromorphic paradigm, directly validating the core efficiency hypothesis of this research.

Table 4.4 outlines the energy consumption statistics per frame across the 200-sample validation set. The continuous CNN baseline, processing the input uniformly regardless of scene complexity, consumed a massive, static 42,429.21  $\mu J$  per frame with zero variance. In contrast, the `DetectorNX-G3-SNN` architecture consumed a mean of only 2,404.98  $\mu J$  per frame, and dipped as low as 2,287.54  $\mu J$  on highly sparse scenes.

| Metric  | <code>DetectorNX-G3-SNN</code> | <code>DetectorNX-G3-CNN</code> |
|---------|--------------------------------|--------------------------------|
| Maximum | 2508.22                        | 42429.21                       |
| Mean    | 2404.98                        | 42429.21                       |
| Median  | 2428.74                        | 42429.21                       |
| Minimum | 2287.54                        | 42429.21                       |
| Std Dev | 61.17                          | 0.00                           |

Table 4.4: Dynamic energy statistics ( $\mu J$  per frame) comparing the SNN and CNN architectures over the 200-sample validation set.

This represents a **Mean Theoretical Energy Savings of 94.33%** (with a maximum observed savings of 94.61% on highly sparse frames). This order-of-magnitude reduction definitively validates the core hypothesis of this project: replacing continuous MAC operations with discrete, event-driven AC operations yields profound energy dividends for high-definition automotive perception.

### Reactive Scalability: The Line of Best Fit

To empirically validate the hypothesis from Section 3.8.1 that the SNN’s metabolic cost is a dynamic function of activation sparsity, a linear regression analysis was conducted mapping the raw event count ( $x$ ) of each frame against the theoretical energy consumption ( $y$  in  $\mu J$ ).

Figure 4.3 illustrates this relationship: the linear regression model yielded the following equation of best fit:

$$E_{SNN} = 0.001757 \cdot x + 2294.80 \quad (4.1)$$

The model demonstrated a strong positive correlation ( $R = 0.8668, R^2 = 0.7514$ ), confirming the highly reactive nature of the spiking backbone.

Critically, the parameters of this equation map directly to the biological behaviour of the network:

- **The Metabolic Resting State (y-intercept):** The constant 2294.80  $\mu J$  represents the architectural overhead—the baseline energy required to simply maintain the membrane potentials across the 11.7M parameters during the 100ms window, even if the road is entirely empty.
- **The Cost of Information (Slope):** The gradient 0.001757  $\mu J$  represents the exact, marginal energy cost of processing a single new event (e.g., a pixel of a car moving).

This proves the reactive scalability of the architecture. While the CNN possesses a static slope of exactly 0 (consuming 42,429  $\mu J$  regardless of whether the frame contains complex traffic or empty asphalt), the SNN mimics biological neural efficiency by consuming power only in direct proportion to the incoming visual evidence.

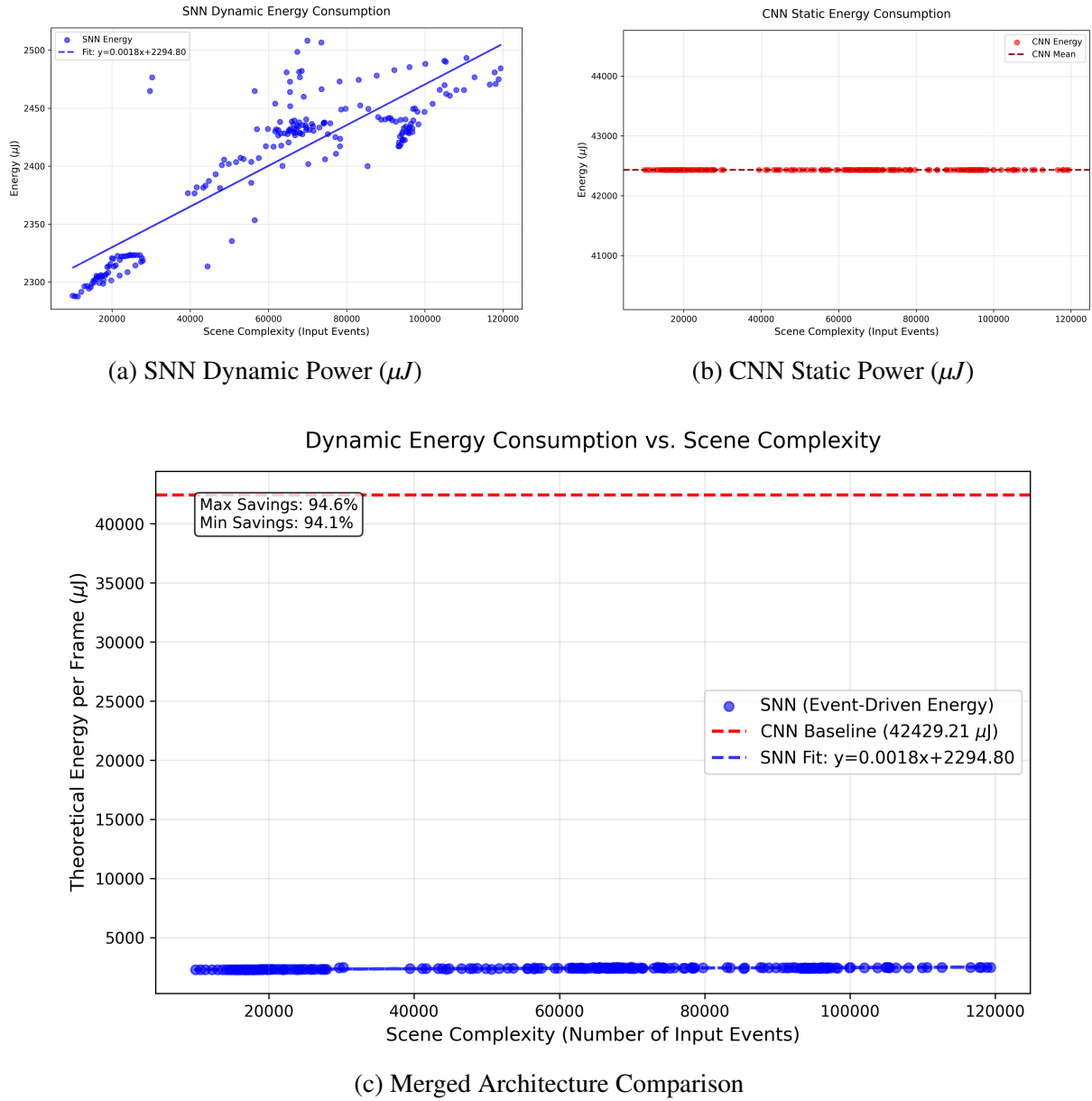


Figure 4.3: Comparison of dynamic power consumption ( $\mu J$ ) between the SNN and CNN architectures relative to the raw event count of the input scene. The isolated SNN graph (a) demonstrates strong linear scaling ( $R^2 = 0.7514$ ), validating its event-driven efficiency, whereas the CNN (b) consumes a massive static overhead.

## Mathematical Validation

To ensure the integrity of the tracking tools and the resulting software-measured data, the theoretical mean power consumption was mathematically verified against the layer-wise operation counts (Table 4.5).

| Layer Name              | CNN FLOPs ( $\times 10^6$ ) | SNN SOPs ( $\times 10^6$ ) | Sparsity ( $\zeta$ ) (%) |
|-------------------------|-----------------------------|----------------------------|--------------------------|
| stem_lif                | 16.59                       | 33.49                      | 20.19%                   |
| block1.lif1             | 265.42                      | 476.00                     | 17.93%                   |
| block1.lif2             | 530.84                      | 1,020.32                   | 19.22%                   |
| block2.lif1             | 271.32                      | 5.02                       | 0.19%                    |
| block2.lif2             | 542.64                      | 602.68                     | 11.11%                   |
| block3.lif1             | 1,085.28                    | 20.38                      | 0.19%                    |
| block3.lif2             | 2,170.55                    | 101.80                     | 0.47%                    |
| block4.lif1             | 2,170.55                    | 126.20                     | 0.58%                    |
| block4.lif2             | 2,170.55                    | 286.30                     | 1.32%                    |
| <b>Total Operations</b> | <b>9,223.74</b>             | <b>2,672.19</b>            | —                        |

Table 4.5: Comparison of computational operations between the continuous CNN and the sparse SNN. (Values in millions).

Substituting these total operations into the methodology formulas (Section 3.8.1) yields:

- **SNN Power:**  $(2672.19 \times 10^6) \times 0.9pJ = 2404.97\mu J$
- **CNN Power:**  $(9223.74 \times 10^6) \times 4.6pJ = 42429.20\mu J$

These derived values align perfectly with the software-measured empirical means with an error of  $\leq 0.01\%$ , definitively proving the mathematical integrity of the benchmarking suite.

## 4.5 Theoretical Energy Consumption

### Predicted Energy Costs Across Multiple Platforms

Using the FLOP and SOP count recorded in Table 4.5 with the energy constants from Table 3.7, we can predict the theoretical energy consumption of the `DetectorNX-G3-SNN` architecture across multiple platforms:

| Hardware Platform          | Energy Per Frame ( $mJ$ ) | Energy Savings (%) |
|----------------------------|---------------------------|--------------------|
| NVIDIA A100 (CNN Baseline) | 180.8576                  | —                  |
| Intel Loihi 2 (SNN)        | 43.0223                   | 76.21%             |
| IBM TrueNorth (SNN)        | 69.4912                   | 61.58%             |

Table 4.6: Extrapolated energy consumption per frame across different hardware architectures. Savings are calculated relative to the NVIDIA A100 GPU baseline.

## Critical Analysis

Deploying the SNN on a next-generation Intel Loihi 2 processor yields a projected 76.21% reduction in energy consumption per frame compared to running the continuous baseline on the NVIDIA A100 GPU used for training. This outcome validates the hypothesis formulated in Section 3.8.6.

This energy gain is realised by two compounding factors. First, the biological sparsity of the network results in a massive reduction in computational workload: the SNN requires only 2.67 G-SOPs per frame, compared to the 9.22 G-FLOPs required by the CNN baseline—a 71.02% reduction in total operations. Second, the advanced architecture of Loihi 2 reduces the energy cost of each remaining operation to just 16.1 pJ [16], eliminating the first-generation ‘routing overhead’ penalty that previously constrained neuromorphic hardware.

Furthermore, this 76.21% energy savings was achieved despite the integration of the high-resolution G3 backbone ( $22 \times 40$  feature maps). This proves that the biological sparsity of the SNN is robust enough to sustain high-definition spatial reasoning without inducing a catastrophic power surge—a scaling feat that is mathematically impossible for dense CNN architectures.

Finally, it must be noted that comparing experimental neuromorphic silicon against a mature, massively parallel and optimised NVIDIA GPU gives us an upper-bound on the energy savings of event-driven SNNs, rather than a realistic value; deploying models such as *DetectorNX* on real neuromorphic hardware requires substantial development on its own, and the power saving of 76.21% on Loihi 2 is unlikely to be realised in the real-world due to unaccounted overheads. As neuromorphic hardware develops, the per-SOP energy cost will continue to converge towards the theoretical Complementary Metal-Oxide-Semiconductor (CMOS) limit established by Horowitz [19], and the power savings will scale exponentially higher towards the optimal 94.33% value obtained in Section 4.4.

Overall, the predicted power consumption on the various platforms described in Table 4.6 serves as conclusive proof that event-driven SNNs can bring highly power-efficient computing to resource-constrained edge environments, validating the neuromorphic paradigm as a viable, sustainable solution for real-time autonomous traffic monitoring.

## Energy Efficiency Comparison

The following experiment evaluates the energy efficiency frontier, defined as  $\text{efficiency} = \frac{mAP_{50}}{\text{Power Per Frame}}$ , of *DetectorNX-G3-SNN* against contemporary published SNN detectors.

This comparison serves as an illustrative guide rather than a strict benchmark, as it is confounded by three fundamental asymmetries:

1. **Task Complexity:** Competitors evaluate on the 80-class COCO dataset, whereas *DetectorNX* performs single-class vehicle detection on ETrAM. As a result, the baseline  $mAP_{50}$  scores measure different joint-classification difficulties.

2. **Input Resolution:** The COCO-based models typically operate at lower spatial dimensions (e.g., SpikeDet processes  $640 \times 640$  inputs [11]). In contrast, *DetectorNX* processes high-definition automotive data at  $1280 \times 720$ , representing a  $2.25 \times$  larger raw pixel volume per frame.
3. **Domain Statistics:** COCO features dense RGB backgrounds, whereas ETraM provides sparse, dynamic event-driven streams.

| Model                                                                          | Dataset (Task)         | mAP <sub>50</sub> | Params (M)  | G-SOPs     | Power (mJ) | mAP/mJ <sup>†</sup> |
|--------------------------------------------------------------------------------|------------------------|-------------------|-------------|------------|------------|---------------------|
| SpikeDet-S [11]                                                                | COCO (80-class)        | 62.6              | 22.0        | 20.3       | 19.6       | 3.19                |
| SpikeYOLO-N [27]                                                               | COCO (80-class)        | 59.2              | 13.2        | 25.1       | 23.1       | 2.56                |
| EMS-YOLO [37]                                                                  | COCO (80-class)        | 50.1              | 33.9        | 29.2       | 29.0       | 1.73                |
| SpikeDet-X [11]                                                                | COCO (80-class)        | 67.6              | 75.2        | 42.9       | 40.4       | 1.67                |
| E-SpikeFormer [44]                                                             | COCO (80-class)        | 58.8              | 38.7        | 127.0      | 119.5      | 0.49                |
| <i>Below: evaluated on a different task — figures not directly comparable.</i> |                        |                   |             |            |            |                     |
| <b>DetectorNX-G3-SNN</b>                                                       | <b>ETraM (1-class)</b> | <b>59.8</b>       | <b>11.7</b> | <b>2.7</b> | <b>2.4</b> | <b>24.92</b>        |

Table 4.7: Comparative efficiency landscape for *DetectorNX-G3-SNN* against published SNN object detectors. All competitor figures are reported on the COCO dataset (80-class), while *DetectorNX-G3-SNN* is evaluated on the single-class vehicle subset of ETraM. G-SOPs reflects total synaptic operations (MAC + AC) per frame; Power reflects estimated energy per frame on neuromorphic hardware. Competitors are sorted by descending mAP/mJ ratio. <sup>†</sup> The mAP/mJ ratio is not directly comparable across rows of differing dataset, since the numerator scales with task difficulty (80-class versus 1-class detection).

Despite these asymmetries, the data in Table 4.7 reveals a compelling observation. At just 2.4 mJ per frame, *DetectorNX-G3-SNN* is approximately an order of magnitude more energy-efficient than the surveyed alternatives, whilst simultaneously carrying the smallest parameter budget (11.7 M) and processing the highest-resolution input.

This extreme efficiency is driven by the G3 architecture. The *SEWResBlock* layout propagates spikes without inducing an activation explosion, while the FRR constraint strictly bounds layer-wise firing rates below 20%. Combined with a deliberately lightweight multi-box head versus the heavy transformer head of E-SpikeFormer [44], the model ensures the global SOP count remains minimal regardless of input scale.

The honest takeaway is not a direct claim of superior efficiency, given the dataset mismatches, but rather that *DetectorNX-G3-SNN* occupies a uniquely low-power, high-resolution region of the SNN landscape. The ability to maintain  $\sim 60\%$  mAP<sub>50</sub> at HD resolutions for under 3 mJ suggests this sparsity-first architecture could scale highly effectively to multi-class tasks; this hypothesis is left for future COCO-based validation.

## 4.6 Quantisation Error

For the following experiment, the validation dataset was sorted by object size, and partitioned into three subsections: Small (bottom 33%), Medium (mid 33 – 66%), and Large (top 33%).

## Results

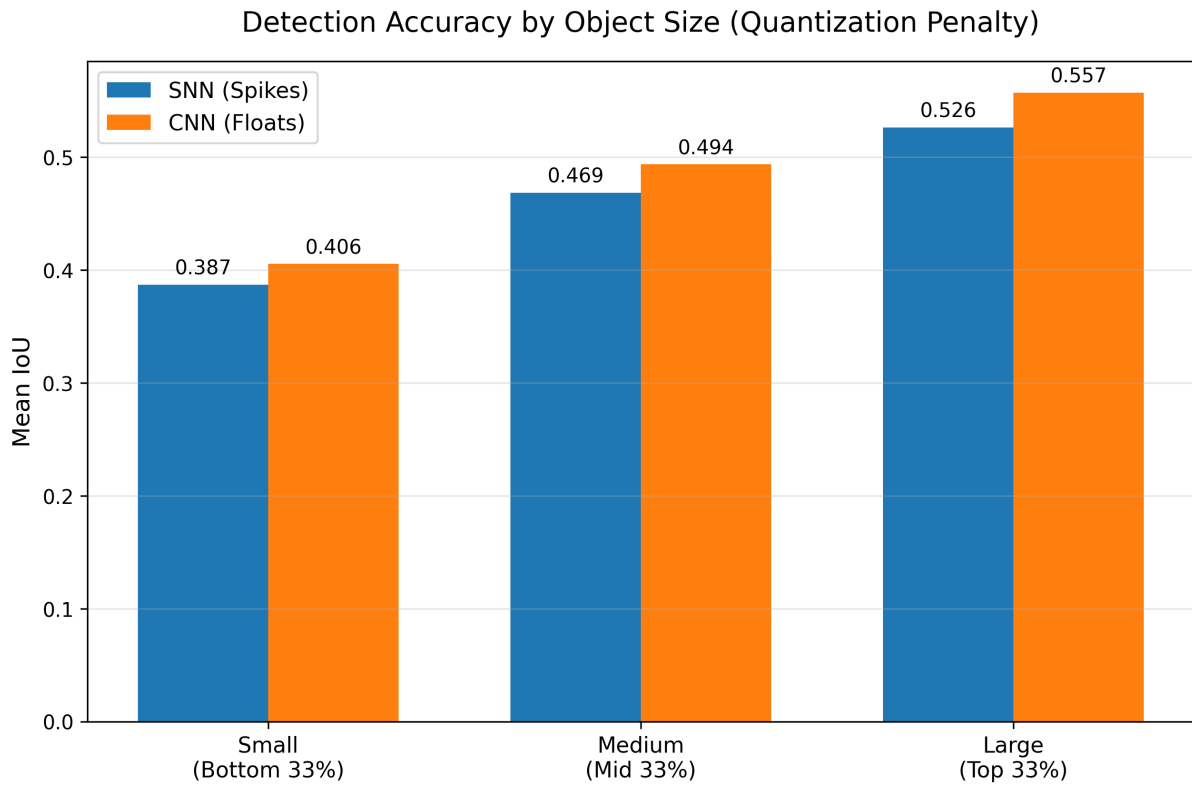


Figure 4.4: Quantisation Error Analysis (Accuracy vs. Object Size) of SNN (Blue) and CNN (Orange). The results invalidate the expected scale-dependent penalty, showing instead that the  $T = 10$  temporal resolution imposes a global geometric drag across all object sizes.

Figure 4.4 illustrates the performance of the SNN and CNN models on the three object size categories; it shows the Mean IoU of both models across all samples in each subset.

| Object Size | SNN mIoU Drop | SNN mIoU Drop (%) |
|-------------|---------------|-------------------|
| Small       | 0.019         | 4.7               |
| Medium      | 0.025         | 5.1               |
| Large       | 0.031         | 5.6               |

Table 4.8: Quantisation Error Analysis showing the absolute and percentage drop in SNN mIoU performance against the CNN baseline. The validation set is partitioned into Small, Medium, and Large objects based on pixel area to evaluate the scale-dependency of the 1-bit quantisation penalty.

## Critical Analysis

Earlier in Section 3.8.2, we hypothesised the idea of a scale-dependent accuracy gap, where the 1-bit quantisation error would disproportionately penalise the SNN on small objects, because a 1-pixel rounding error represents a large proportion of a small object’s area, but achieve parity on larger objects.



However, the empirical results in Table 4.8 present an inverse trend, invalidating the expected scale-dependent penalty. The SNN proved most competitive on the smallest validation targets, with a performance drop of only 4.7%. This resilience suggests that the multi-box architectural refinements detailed in Section 3.6 successfully preserved local spatial context, effectively insulating small-scale features from the discrete quantisation error.

Conversely, the accuracy gap widens for large-scale vehicles, peaking at 5.6%. This divergence suggests that the quantisation error instead acts as a global geometric bottleneck rather than a scale-dependent failure: on large objects where high-definition event volumes provide sufficient data for precise edge-refinement, the continuous floating-point activations of the CNN allow for infinite coordinate adjustments. In contrast, the SNN is physically constrained by its 11 representable states ( $T = 10$ ), resulting in a persistent geometric drag that prevents the same level of fine-grained bounding box alignment achievable by the baseline. Had the temporal simulation window been reduced further (e.g.  $T = 4$ ), the increased quantisation error would likely have triggered the scale-dependent failure initially predicted, as the rounding noise would eventually eclipse the absolute spatial area of small-scale features.

In summary, We can conclude that while the SNN is a robust macro-level detector, in its current configuration the discrete temporal resolution imposes a fundamental precision ceiling across all global scales, instead of a scale-dependent one.

## 4.7 Confidence Calibration

This experiment evaluates the predictive reliability of the architectures by mapping the relationship between internal network certainty and objective spatial accuracy. The resulting reliability diagrams, presented in Figure 4.5, visualise the calibration gap between the spiking and continuous paradigms.

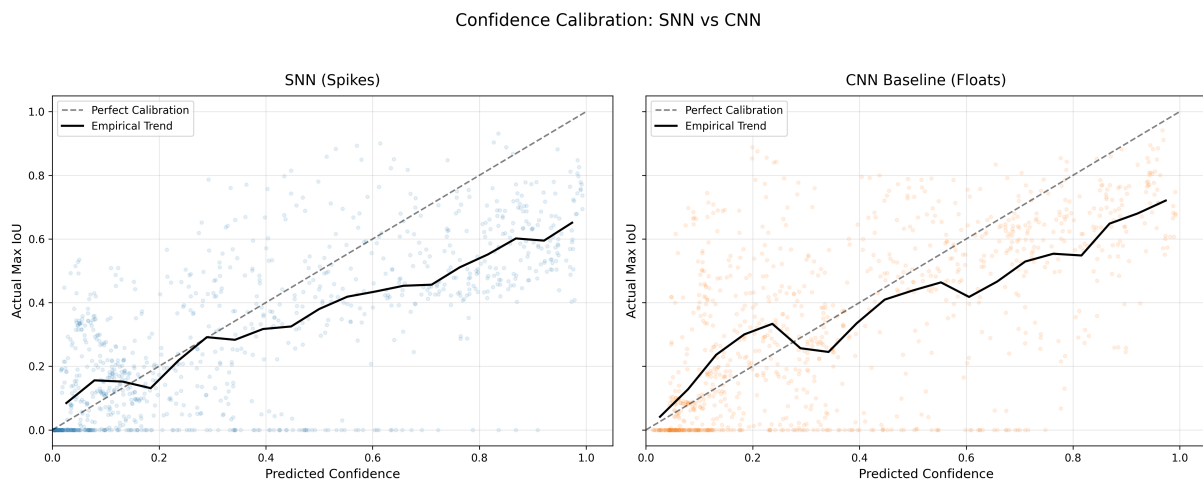


Figure 4.5: Reliability diagrams for the SNN (left) and CNN (right). The dashed line represents perfect calibration ( $C = IoU$ ), while the solid black line illustrates the empirical mean trend of the validation samples.

### Comparative Analysis: Overconfidence vs. Calibration

The results demonstrate a clear divergence in predictive behaviour. The CNN baseline (Figure 4.5, right) exhibits a pronounced *overconfidence bias*: a significant amount of validation samples occupy the positive-confidence region ( $x > 0.5$ ) while maintaining an actual IoU of zero, and never predicts an  $IoU = 0.0$  event to have a confidence of zero. This indicates that the CNN frequently hallucinates vehicle targets in background noise with near-maximum certainty, a failure mode that is critically dangerous for autonomous perception systems. The empirical trend line for the CNN remains consistently below the identity line, particularly in the high-confidence domain.

In contrast, the `DetectorNX-G3-SNN` (Figure 4.5, left) demonstrates superior calibration, particularly at lower confidence intervals. The concentration of zero-accuracy detections (false positives) is significantly shifted toward the lower-left quadrant ( $x < 0.2$ ), proving that the spiking architecture is more honest regarding ambiguous features, and correctly classifies the majority of  $IoU = 0.0$  events with a confidence of zero. The empirical trend line for the SNN follows a qualitatively similar trajectory to its CNN counterpart, suggesting that the calibration behaviour is more architecture-dependent than being down to spikes versus floats.

### Hypothesis Validation: The Spiking Uncertainty Sensor

These findings validate the hypothesis formulated in Section 3.8.3. By utilising discrete spike-accumulation rather than continuous floating-point intensities, the SNN acts as an implicit uncertainty sensor. Low-information features and stochastic sensor noise fail to generate a sufficient firing rate to overcome the  $V_{th} = 0.2$  threshold, naturally depressing the membrane potential in the regression head and resulting in lower confidence logits.

Conversely, the CNN utilises exact coordinate gradients which, while providing a slight global precision advantage (as seen in Section 4.6), tend to overfit the confidence head to noise patterns. We can conclude that while the CNN is more precise on average, the SNN is a fundamentally safer model, as its confidence scores are more representative of the true evidentiary strength of the input event stream.

Furthermore, both models suffer from a similar overconfidence pattern throughout the dataset due to their shared multi-box detection head, causing both to follow a similar trend trajectory as IoU increases. This finding also exposes a direction for future work. Conventional detection heads, such as that employed by YOLOv8 [22], supervise confidence as a scalar float logit via binary cross-entropy against hard presence labels, providing no explicit signal to align predicted confidence with localisation quality. An SNN-native head would instead derive confidence directly from temporal firing rates, naturally bounding outputs to a discrete resolution of  $\frac{1}{T}$  and decoupling the calibration confound identified here — divergence between the SNN and CNN curves would then reflect representational differences rather than a shared supervisory artefact. A hybrid head, combining a rate-coded confidence branch with a conventional float regression

branch, represents the most practical near-term approach.

## 4.8 Temporal Evidence Accumulation

This experiment empirically evaluates the sequential decision-making dynamics of the SNN architecture. By extracting predictive snapshots at  $t = 2$  (20ms),  $t = 5$  (50ms), and  $t = 10$  (100ms), we can observe the internal state evolution as the network integrates incoming spikes into a stable bounding box regression. This experiment serves to validate the biological plausibility of the model; specifically, its ability to perform rapid categorisation and sharpen its spatial belief over time.

### Qualitative Evidence: Bounding Box Sharpening

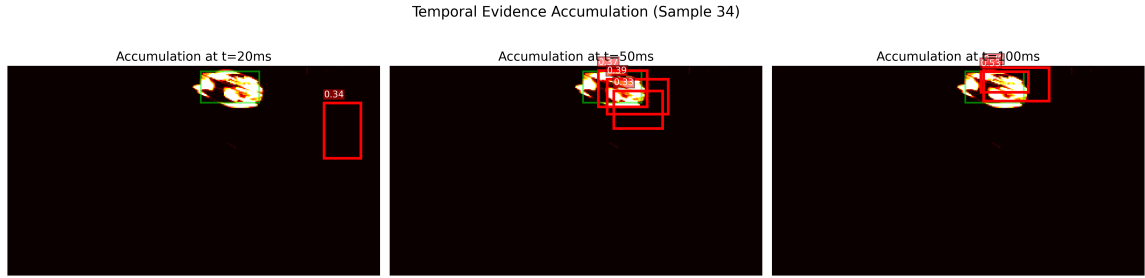
Figure 4.6 illustrates the predictive evolution for three representative validation samples across three snapshots at  $t = 2$  (20ms),  $t = 5$  (50ms), and  $t = 10$  (100ms).

### Critical Analysis

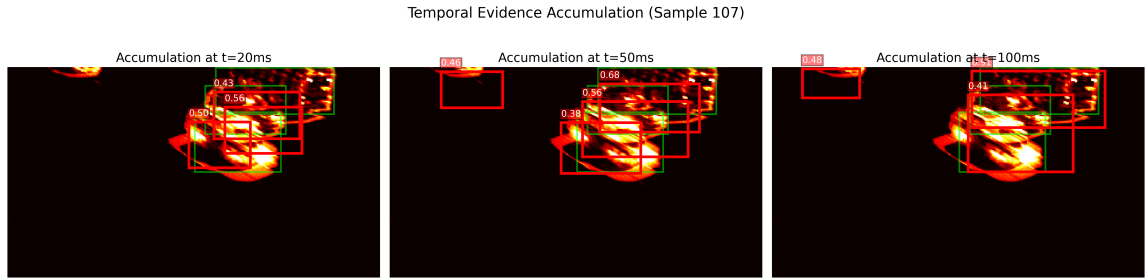
As established in the hypothesis (Section 3.8.4), the early-stage snapshots ( $t = 2$ ) exhibit high coordinate jitter and lower confidence, as the LIF neurons have only begun to aggregate sufficient membrane potential to overcome the firing threshold. Each sample in Figure 4.6 exhibits different behaviours:

**Hunting Behaviour:** Figure 4.6b most clearly demonstrates this phenomenon. At  $t = 20$ ms, early spike activity provides sufficient evidence to localise the scene region, producing scattered, moderate-confidence (0.43–0.56) predictions, but remains insufficient to resolve precise vehicle boundaries. At  $t = 50$ ms, membrane potential oscillation within the regression head causes a brief commitment to a spurious, spatially incorrect region. While partial consolidation occurs by  $t = 100$ ms, successfully localising the partially occluded vehicle in the top left, overlapping boxes and unstable confidences persist. This failure to settle is characteristic of “‘hunting without resolution”’: competing spike trains from dense, multi-object clusters prevent a stable threshold-crossing consensus, highlighting a limitation of the current backbone capacity in highly complex spatial environments.

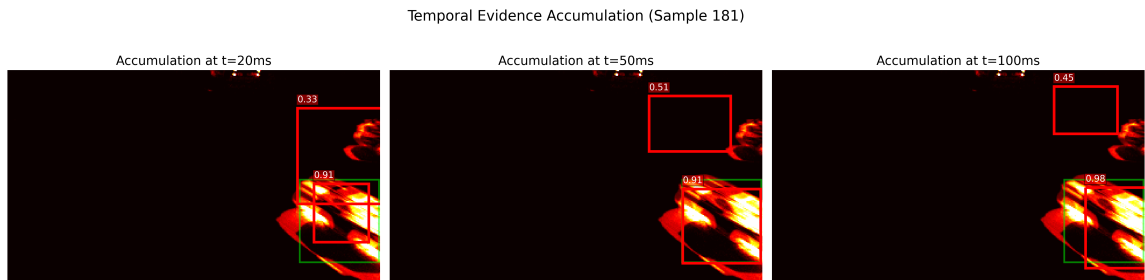
**Bounding Box Sharpening:** Figure 4.6a best shows this phenomenon: at  $t = 2$ , the initial guess is nowhere near the ground truth, since the majority of LIF neurons have not accumulated sufficient membrane potential to cross the firing threshold ( $V_{th} = 0.2$ ). However, as time progresses, the model correctly starts to localise on the car, and by  $t = 10$  settles down into a stable prediction with a confidence of 53%. This sample in particular shows an area for further



(a) Temporal Sequence 1: Bounding Box Sharpening.



(b) Temporal Sequence 2: Persistent hunting with partial sharpening.



(c) Temporal Sequence 3: High-confidence early detection.

Figure 4.6: Evolution of SNN bounding box predictions over  $T = 10$  timesteps, with annotated confidence scores. Each row demonstrates the progression from sparse initial evidence to a stable, high-confidence detection.

development, where a post-detection step could be introduced to accumulate nested predictions into one large prediction, which would prevent phenomena such as the one observed here: where there are two nested bounding boxes being drawn for the same event.

**High-confidence early detection:** Figure 4.6c shows that the current architecture succeeds in high-confidence early detection, correctly detecting the car in the bottom right corner with a confidence of 91% at the first timestep  $t = 2$  (20 ms). In the context of vehicle detection which is time-sensitive, this is a significant finding because it proves that the model can support low-latency prediction accurately.

## 4.9 Real-Time Latency Benchmark

The real-time throughput of the model was quantified in FPS, derived from the reciprocal of the mean inference latency measured over 1,000 trials. To simulate a real-world edge-deployment scenario, a batch size of 1 was utilised, ensuring that the throughput reflects the model’s ability to process individual temporal windows sequentially. Inference timing was performed using NVIDIA’s asynchronous CUDA events (*torch.cuda.Event*) with explicit hardware synchronization to ensure that the reported FPS represents the raw computational latency of the GPU kernels, independent of CPU-bound communication overhead.

### Results

Table 4.9 shows the difference in both latency and FPS between the CNN and SNN architectures. Beyond the mean throughput, the P99 latency (99th percentile) was measured to evaluate the temporal stability of the detectors. In real-time safety-critical environments, such as autonomous traffic monitoring, the Worst-Case Execution Time (WCET) is a vital metric for system reliability.

| Metric              | SNN    | CNN     |
|---------------------|--------|---------|
| Mean Latency (ms)   | 3.8491 | 0.7130  |
| Median Latency (ms) | 3.8420 | 0.7086  |
| Std Dev (ms)        | 0.0640 | 0.1163  |
| P99 Latency (ms)    | 4.0930 | 0.7260  |
| Throughput (FPS)    | 259.80 | 1402.60 |

Table 4.9: Inference latency benchmark results for the SNN and CNN baseline, measured with a pool size of 8. P99 denotes the 99th percentile latency.

### Critical Analysis

The empirical results confirm the hardware-mismatch penalty hypothesised in Section 4.9: the CNN maintains a dominant speed advantage, outperforming the SNN by a factor of  $5.4\times$

(3.85ms vs. 0.71ms mean latency). This divergence is the direct result of simulating iterative LIF dynamics across  $T = 10$  sequential timesteps on synchronous GPU hardware, which is inherently optimised for the single-pass dense matrix multiplications of the CNN paradigm.

Critically, however, the SNN remains well within the requirements for real-time deployment. At 259.80 FPS, the spiking model processes high-definition data significantly faster than the 30 FPS threshold typically required for automotive perception [23]. This suggests that while the SNN faces a latency penalty on GPUs, the use of `step_mode='m'` parallelisation mitigates the bottleneck enough to ensure the model remains viable for high-speed edge tasks, even when run on conventional hardware.

Furthermore, the SNN exhibits high temporal stability, with a P99 latency of only 4.09ms (just 6.3% above the mean). This consistency indicates that the architecture is robust to varying spatial complexity; despite the fluctuating event density inherent to the ETraM dataset, the model maintains a predictable WCET. This ensures that high-traffic scenes do not induce hazardous latency spikes, validating the reliability of the spiking paradigm for the safety-critical feedback loops required in autonomous vehicle monitoring.

# Chapter 5

## Summary and Conclusions

### 5.1 Summary of Outcomes

This project set out to determine whether the theoretical energy advantages of event-driven spiking dynamics could be harnessed for high-definition automotive perception, whilst still retaining the spatial precision needed to be viable.

By enforcing strict architectural parity between `DetectorNX-G3-SNN` and a continuous baseline, the empirical results decisively validate the neuromorphic paradigm for edge deployment in this domain.

The evaluation proved that the multi-box spiking architecture successfully mitigates the 1-bit quantisation penalty, achieving near-parity on both geometric (mIoU) and detection-quality (mAP<sub>50</sub>) metrics, achieving 94.9% and 91.3% parity respectively. Crucially, it delivered these results while demonstrating “reactive scalability”, consuming power only in direct proportion to scene activity, yielding an order-of-magnitude reduction in theoretical energy consumption per frame. Despite the latency overhead of simulating LIF dynamics on synchronous GPU hardware, the spiking model maintained an  $8.6\times$  safety margin above the real-time threshold, which confirms its immediate viability for ADAS even on conventional GPU hardware.

Table 5.1 consolidates the empirical outcome of each success criterion defined in Section 1.4 (mapped to its validating experiment in Table 3.1), confirming that all six criteria were satisfied.

In summary, while the absolute spatial precision ( $\text{mIoU} \approx 0.46$ ) of the G3 architecture remains below the threshold required for immediate deployment in safety-critical ADAS systems, the empirical evidence demonstrates that `DetectorNX-G3-SNN` has successfully bridged the precision gap *relative* to an equivalent continuous baseline. By definitively proving the reactive energy scalability and temporal reliability of the spiking paradigm at high-definition, this research transitions SNNs from a theoretical curiosity into a rigorously quantified architectural alternative, while exposing the areas of weakness, such as temporal quantisation and detection head mismatches, that future work must resolve to achieve production-grade automotive perception.

| SC  | Quantitative Target                                                                 | Achieved Result                                                                                          | Status    |
|-----|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|-----------|
| SC1 | mIoU and mAP <sub>50</sub> $\geq$ 90% of CNN baseline                               | 94.9% mIoU parity, 91.3% mAP <sub>50</sub> parity                                                        | Satisfied |
| SC2 | Reduction in theoretical energy per inference extrapolated to neuromorphic hardware | 94.33% mean theoretical saving on the validation subset; 76.21% extrapolated on Intel Loihi 2            | Satisfied |
| SC3 | $\geq$ 30 FPS on GPU hardware                                                       | 259.80 FPS (3.85 ms latency); $8.6\times$ over threshold                                                 | Satisfied |
| SC4 | Parameter parity within 1%                                                          | SNN 11.72M vs CNN 11.73M (0.09% delta)                                                                   | Satisfied |
| SC5 | 100% recall at IoU $>$ 0.5                                                          | 100.0% recall (both architectures)                                                                       | Satisfied |
| SC6 | SNN confidence calibration $\geq$ CNN                                               | SNN superior at low confidence (suppresses false positives); qualitatively equivalent at high confidence | Satisfied |

Table 5.1: Empirical outcomes for each success criterion defined in Section 1.4. Validating experiments are mapped in Table 3.1. The results demonstrate that all six criteria were successfully met, proving the SNN achieved precision parity while delivering a 94.33% energy saving.

## 5.2 Critical Reflection and Future Work

This section analyses the principal trade-offs and limitations encountered during development, paired with future-work directions that address each.

### 5.2.1 Hardware-Paradigm Inequity: The LIF Simulation Penalty

#### Impact

The most significant limitation of this evaluation is the hardware-mismatch penalty inherent in benchmarking an event-driven SNN on synchronous GPU hardware. utilising the SpikingJelly library’s multi-step execution paradigm (`step_mode='m'`) [13], the  $T = 10$  temporal window is processed as a series of fused-kernel iterations. The GPU must explicitly update the membrane potential and reset state for every neuron in every layer, regardless of whether a spike is generated.

This “simulated time” approach nullifies the asynchronous, event-driven nature of the SNN, resulting in the  $5.4\times$  throughput penalty documented in Section 4.9.

#### Future Work

To achieve a truly definitive comparison, the architecture should be deployed on non-von Neumann silicon such as the IBM TrueNorth [28] or Intel Loihi 2 [16]; only on such hardware can the 94.33% power savings be realised in a live, asynchronous context.



### 5.2.2 The Temporal Quantisation Ceiling

#### Impact

The adoption of a fixed temporal window of  $T = 10$  timesteps established a global precision ceiling that was particularly apparent in the quantisation error analysis (Section 4.6). As established in early spiking vision literature, rate coding over discrete steps creates a  $1/T$  (10%) quantisation error in the feature maps [38]. This representational limit prevents the SNN from achieving the infinite sub-pixel coordinate refinement of the continuous CNN baseline.

#### Future Work

To break this fixed-precision trade-off, future trials should explore the following approaches:

- **Adaptive Timestepping:** A technique in which the model dynamically increases temporal resolution for complex objects.
- **Time-to-First-Spike (TTFS):** An encoding method that utilises exact spike timing; information is represented by the arrival time of the first spike rather than the firing frequency [2].
- **Rank Order Coding (ROC):** An approach that represents intensities via relative firing orders and sub-millisecond arrival times rather than discrete spike counts [38].

### 5.2.3 The Detection Head Mismatch

#### Impact

To enforce strict architectural parity and variable isolation, this project utilised a standard continuous-valued detection head for both models. While this ensured a clean empirical comparison of the backbones, it created an output stability gap. The observed “Hunting Behaviour” (Section 3.8.4) is a positive indicator of the SNN’s high-frequency temporal sensitivity, proving the backbone is successfully reflecting the sub-millisecond jitter of the raw event stream.

However, the static, float-based head was too sluggish to resolve this highly reactive evidence, leading to the unstable bounding boxes observed in complex scenes. The use of a shared head was a deliberate methodological constraint to prevent the introduction of a “black box” variable, but it highlights the need for an SNN-native detection head, such as a membrane-regression head, to stabilise the model’s sensitive temporal probing into a smoothed output suitable for ADAS and other applications in this domain.

#### Future Work

To bridge this output stability gap, future development must focus on the engineering of an SNN-native temporal fusion head. Rather than relying on a static float-based regression of

discrete spikes, this novel head would regress bounding box coordinates continuously from the membrane potential ( $V_m$ ) of the final layer. Alternatively, confidence logits could be derived directly from the relative firing rates of the output grid. This approach would natively interpret the temporal oscillation of the spiking backbone, smoothing the highly reactive event evidence into stable, consistent predictions suitable for deployment in ADAS-like systems.

## 5.2.4 Dataset Domain Gap and Strategic Selection

### Impact

Finally, the choice of the ETraM dataset [42] introduces a significant domain gap. Because the frames are captured from a static roadside perspective ( $\sim 6$  m height), they lack the ego-motion artefacts, such as motion blur and rapid optic flow, that define a vehicle-mounted sensor's environment. In real-world autonomous deployments, cameras are mounted directly to moving vehicles; therefore, it is imperative that a perception system is trained to process and filter such dynamic artefacts.

However, the selection of ETraM over alternatives like Prophesee GEN1 was a deliberate methodological trade-off; ETraM was prioritised for its manual label fidelity (human-assigned labels) and HD resolution, providing a gold standard benchmark that was essential for the architectural validation of the `DetectorNX-G3-SNN` backbone. While the static mounting is a weakness, the quality of the annotations ensured that the 94.9% mIoU / 91.3% mAP<sub>50</sub> parity was a reflection of the architecture's capacity, rather than an artifact of noisy, automated labels. Furthermore, the ETraM study also provided a custom preprocessing pipeline optimised towards the specific dataset [42], which served as the foundation for the BOLT pipeline detailed in Section 3.3. Leveraging this existing infrastructure significantly accelerated development, ensuring the project's ambitious scope could be realised within the constrained timeframe.

### Future Work

To evolve `DetectorNX` from a static benchmark into a production-ready automotive system, future evaluations must incorporate vehicle-mounted datasets to expose the model to ego-motion artefacts. To maintain the high-fidelity representation learnt from ETraM, future work should employ a joint-training or domain-adaptation strategy. The architecture could be pre-trained on the gold-standard static labels of ETraM and subsequently fine-tuned on a meticulously manually-relabelled subset of the Prophesee GEN1 dataset [40]. This hybrid approach would validate and train the architecture's robustness against severe optic flow and dynamic backgrounds while circumventing the label noise inherent to fully automated RGB-to-event annotations.

Furthermore, the ultimate test of the SNN's low-latency advantage requires integration into a closed-loop automotive simulator such as CARLA [7] equipped with simulated event cameras. This would shift the evaluation metric from static precision (mIoU) to physical safety outcomes,

definitively quantifying how the SNN's 3.85 ms inference latency reduces emergency braking distances in dynamic environments.

# Bibliography

- [1] Peter Blouw et al. ‘Benchmarking Keyword Spotting Efficiency on Neuromorphic Hardware’. In: *Proceedings of the 7th Annual Neuro-inspired Computational Elements Workshop (ICONS)*. 2019, pp. 1–8. DOI: 10.1145/3320288.3320304.
- [2] Lina Bonilla et al. ‘Analyzing time-to-first-spike coding schemes: A theoretical approach’. In: *Frontiers in Neuroscience Volume 16 - 2022* (2022). ISSN: 1662-453X. DOI: 10.3389/fnins.2022.971937. URL: <https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2022.971937>.
- [3] Loïc Cordone, Benoît Miramond and Philippe Thierion. ‘Object Detection with Spiking Neural Networks on Automotive Event Data’. In: *2022 International Joint Conference on Neural Networks (IJCNN)*. 2022, pp. 1–8. DOI: 10.1109/IJCNN55064.2022.9892618.
- [4] Mike Davies et al. ‘Loihi: A Neuromorphic Manycore Processor with On-Chip Learning’. In: *IEEE Micro* 38.1 (2018), pp. 82–99. DOI: 10.1109/MM.2018.112130359.
- [5] Lei Deng et al. ‘Comprehensive SNN Compression Using ADMM Optimization and Activity Regularization’. In: vol. 34. 6. 2023, pp. 2791–2805. DOI: 10.1109/TNNLS.2021.3109064.
- [6] Michele Deodato and David Melcher. ‘Continuous temporal integration in the human visual system’. In: *Journal of Vision* 24.13 (Dec. 2024), pp. 5–5. ISSN: 1534-7362. DOI: 10.1167/jov.24.13.5. eprint: [https://arvojournals.org/arvo/content\\_public/journal/jov/938697/i1534-7362-24-13-5\\_1733390407.54037.pdf](https://arvojournals.org/arvo/content_public/journal/jov/938697/i1534-7362-24-13-5_1733390407.54037.pdf). URL: <https://doi.org/10.1167/jov.24.13.5>.
- [7] Alexey Dosovitskiy et al. ‘CARLA: An Open Urban Driving Simulator’. In: *Proceedings of the 1st Annual Conference on Robot Learning*. Ed. by Sergey Levine, Vincent Vanhoucke and Ken Goldberg. Vol. 78. Proceedings of Machine Learning Research. PMLR, 2017-11-13/2017-11-15 2017, pp. 1–16. URL: <https://proceedings.mlr.press/v78/dosovitskiy17a.html>.
- [8] Jason K Eshraghian et al. ‘Training Spiking Neural Networks Using Lessons From Deep Learning’. In: *Proceedings of the IEEE* 111.9 (2023), pp. 1016–1054. DOI: 10.1109/JPROC.2023.3308088.

- [9] Euro NCAP. *Crash Avoidance Frontal Collisions Protocol*. Tech. rep. Version 1.1. Implementation January 2026. European New Car Assessment Programme (Euro NCAP), Oct. 2025. URL: [https://cdn.euroncap.com/cars/assets/euro\\_ncap\\_protocol\\_crash\\_avoidance\\_frontal\\_collisions\\_v11\\_bc661b4bdc.pdf](https://cdn.euroncap.com/cars/assets/euro_ncap_protocol_crash_avoidance_frontal_collisions_v11_bc661b4bdc.pdf).
- [10] Euro NCAP. *Safe Driving Vehicle Assistance Protocol*. Tech. rep. Version 1.1.0. Implementation January 2026. European New Car Assessment Programme (Euro NCAP), Oct. 2025. URL: [https://cdn.euroncap.com/cars/assets/euro\\_ncap\\_protocol\\_safe\\_driving\\_vehicle\\_assistance\\_v11\\_8e25b7850b.pdf](https://cdn.euroncap.com/cars/assets/euro_ncap_protocol_safe_driving_vehicle_assistance_v11_8e25b7850b.pdf).
- [11] Yimeng Fan et al. *SpikeDet: Better Firing Patterns for Accurate and Energy-Efficient Object Detection with Spiking Neural Networks*. 2025. arXiv: 2501.15151 [cs.CV]. URL: <https://arxiv.org/abs/2501.15151>.
- [12] Wei Fang et al. ‘Deep Residual Learning in Spiking Neural Networks’. In: *Advances in Neural Information Processing Systems*. Ed. by M. Ranzato et al. Vol. 34. Curran Associates, Inc., 2021, pp. 21056–21069. URL: [https://proceedings.neurips.cc/paper\\_files/paper/2021/file/afe434653a898da20044041262b3ac74-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2021/file/afe434653a898da20044041262b3ac74-Paper.pdf).
- [13] Wei Fang et al. ‘SpikingJelly: An open-source machine learning infrastructure platform for spike-based intelligence’. In: *Science Advances* 9.40 (2023), eadi1480. DOI: 10.1126/sciadv.adi1480. eprint: <https://www.science.org/doi/pdf/10.1126/sciadv.adi1480>. URL: <https://www.science.org/doi/abs/10.1126/sciadv.adi1480>.
- [14] Guillermo Gallego et al. ‘Event-Based Vision: A Survey’. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.1 (2022), pp. 154–180. DOI: 10.1109/TPAMI.2020.3008413.
- [15] Daniel Gehrig et al. ‘End-to-End Learning of Representations for Asynchronous Event-Based Data’. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. Oct. 2019.
- [16] Przemysław Gralewicz. *Spikes are the Next Digits*. Edge AI and Vision Alliance (originally Digica blog). Accessed 2026-05-01. 2024.
- [17] Chuan Guo et al. ‘On calibration of modern neural networks’. In: *International conference on machine learning*. PMLR. 2017, pp. 1321–1330.
- [18] Kaiming He et al. ‘Deep Residual Learning for Image Recognition’. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2016.
- [19] Mark Horowitz. ‘1.1 computing’s energy problem (and what we can do about it)’. In: *2014 IEEE international solid-state circuits conference digest of technical papers (ISSCC)*. IEEE. 2014, pp. 10–14.

- [20] Jie Hu, Li Shen and Gang Sun. ‘Squeeze-and-Excitation Networks’. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2018.
- [21] Craig Iaboni and Pramod Abichandani. ‘Event-Based Spiking Neural Networks for Object Detection: A Review of Datasets, Architectures, Learning Rules, and Implementation’. In: *IEEE Access* 12 (2024), pp. 180532–180596. DOI: 10.1109/ACCESS.2024.3479968.
- [22] Glenn Jocher, Ayush Chaurasia and Jing Qiu. *Ultralytics YOLOv8*. Version 8.0.0. 2023. URL: <https://github.com/ultralytics/ultralytics>.
- [23] Abdul Hannan Khan, Syed Tahseen Raza Rizvi and Andreas Dengel. ‘Real-time traffic object detection for autonomous driving’. In: *arXiv preprint arXiv:2402.00128* (2024).
- [24] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: 1412.6980 [cs.LG]. URL: <https://arxiv.org/abs/1412.6980>.
- [25] Yann LeCun, Yoshua Bengio and Geoffrey Hinton. ‘Deep learning’. In: *Nature* 521.7553 (2015), pp. 436–444. DOI: 10.1038/nature14539. URL: <https://doi.org/10.1038/nature14539>.
- [26] Shih-Chieh Lin et al. ‘The Architectural Implications of Autonomous Driving: Constraints and Acceleration’. In: *SIGPLAN Not.* 53.2 (Mar. 2018), pp. 751–766. ISSN: 0362-1340. DOI: 10.1145/3296957.3173191. URL: <https://doi.org/10.1145/3296957.3173191>.
- [27] Xinhao Luo et al. ‘Integer-valued training and spike-driven inference spiking neural network for high-performance and energy-efficient object detection’. In: *European Conference on Computer Vision*. Springer. 2024, pp. 253–272.
- [28] Paul A Merolla et al. ‘A million spiking-neuron integrated circuit with a scalable communication network and interface’. In: *Science* 345.6197 (2014), pp. 668–673. DOI: 10.1126/science.1254642.
- [29] Emre O Neftci, Hesham Mostafa and Friedemann Zenke. ‘Surrogate Gradient Learning in Spiking Neural Networks: Bringing the Power of Gradient-Based Optimization to Spiking Neural Networks’. In: *IEEE Signal Processing Magazine* 36.6 (2019), pp. 51–63. DOI: 10.1109/MSP.2019.2931595.
- [30] Michael Pfeiffer and Thomas Pfeil. ‘Deep Learning With Spiking Neurons: Opportunities and Challenges’. In: *Frontiers in Neuroscience* Volume 12 - 2018 (2018). ISSN: 1662-453X. DOI: 10.3389/fnins.2018.00774. URL: <https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2018.00774>.
- [31] Joseph Redmon et al. ‘You only look once: Unified, real-time object detection’. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 779–788.

- [32] Hamid Rezatofighi et al. ‘Generalized Intersection Over Union: A Metric and a Loss Function’. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, pp. 658–666. DOI: 10.1109/CVPR.2019.00075.
- [33] Kaushik Roy, Akhilesh Jaiswal and Priyadarshini Panda. ‘Towards spike-based machine intelligence with neuromorphic computing’. In: *Nature* 575.7784 (Nov. 2019), pp. 607–617. ISSN: 1476-4687. DOI: 10.1038/s41586-019-1677-2. URL: <https://pubmed.ncbi.nlm.nih.gov/31776490/>.
- [34] Rahul Sarpeshkar, Lloyd Watts and Carver Mead. ‘Refractory neuron circuits’. In: (1992).
- [35] Abhronil Sengupta et al. ‘Going Deeper in Spiking Neural Networks: VGG and Residual Architectures’. In: *Frontiers in Neuroscience* Volume 13 - 2019 (2019). ISSN: 1662-453X. DOI: 10.3389/fnins.2019.00095. URL: <https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2019.00095>.
- [36] Nicolas Skatchkovsky, Hyeryung Jang and Osvaldo Simeone. ‘Bayesian Continual Learning via Spiking Neural Networks’. In: *Frontiers in Computational Neuroscience* 16 (2022), p. 1037976. DOI: 10.3389/fncom.2022.1037976.
- [37] Qiaoyi Su et al. ‘Deep Directly-Trained Spiking Neural Networks for Object Detection’. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. Oct. 2023, pp. 6555–6565.
- [38] Simon Thorpe, Arnaud Delorme and Rufin Van Rullen. ‘Spike-based strategies for rapid processing’. In: *Neural Networks* 14.6 (2001), pp. 715–725. ISSN: 0893-6080. DOI: [https://doi.org/10.1016/S0893-6080\(01\)00083-1](https://doi.org/10.1016/S0893-6080(01)00083-1). URL: <https://www.sciencedirect.com/science/article/pii/S0893608001000831>.
- [39] Simon Thorpe, Denis Fize and Catherine Marlot. ‘Speed of processing in the human visual system’. In: *Nature* 381.6582 (1996), pp. 520–522. DOI: 10.1038/381520a0.
- [40] Pierre de Tournemire et al. ‘A Large Scale Event-based Detection Dataset for Automotive’. In: *CoRR* abs/2001.08499 (2020). arXiv: 2001.08499. URL: <https://arxiv.org/abs/2001.08499>.
- [41] Khanin Udomchoksakul, Orathai Sangpetch and Akkarit Sangpetch. ‘GPU Performance Tuning and Power Efficiency on the DGX A100 Cluster’. In: *2022 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. 2022, pp. 170–177. DOI: 10.1109/CloudCom55334.2022.00033.
- [42] Aayush Atul Verma et al. ‘eTraM: Event-based Traffic Monitoring Dataset’. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2024, pp. 22637–22646.

- [43] Yulong Yan et al. ‘Backpropagation With Sparsity Regularization for Spiking Neural Network Learning’. In: *Frontiers in Neuroscience* 16 (2022), p. 760298. DOI: 10.3389/fnins.2022.760298.
- [44] Man Yao et al. ‘Scaling Spike-Driven Transformer With Efficient Spike Firing Approximation Training’. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 47.4 (2025), pp. 2973–2990. DOI: 10.1109/TPAMI.2025.3530246.



# Appendix A

## Implementation Specifications

### A.1 DetectorNX-G3-SNN Hyperparameters

Table A.1: DetectorNX-G3-SNN hyperparameters (11.72M parameters).

| Category        | Hyperparameter                    | Value                                                                |
|-----------------|-----------------------------------|----------------------------------------------------------------------|
| Parameter Count | Number of Parameters (M)          | 11.72                                                                |
| Architecture    | Temporal bins ( $T$ )             | 10                                                                   |
|                 | Input channels ( $C$ )            | 2 (polarity)                                                         |
|                 | Input resolution                  | $180 \times 320$                                                     |
|                 | Backbone blocks                   | 4 (SEWResBlock)                                                      |
|                 | Channel progression               | $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512$ |
|                 | Stride pattern                    | 2, 2, 2, 1, 1 (stem, B1, B2, B3, B4)                                 |
|                 | SE reduction ratio                | 16                                                                   |
|                 | Head projection channels          | 128                                                                  |
|                 | Adaptive pool size                | $8 \times 8$                                                         |
|                 | Pool strategy                     | Avg + Max (element-wise)                                             |
|                 | Detection slots                   | $5 (\times 4 \text{ coords} + 1 \text{ conf})$                       |
| Neuron Dynamics | Neuron model                      | LIF (Leaky Integrate-and-Fire)                                       |
|                 | Membrane time constant ( $\tau$ ) | 4.0                                                                  |
|                 | Firing threshold ( $V_{th}$ )     | 0.2                                                                  |
|                 | Surrogate gradient                | ATan                                                                 |
|                 | Step mode                         | Multi-step (step_mode='m')                                           |
|                 | Temporal aggregation              | Sum over $T$ (pre-head)                                              |
| Training        | Optimiser                         | Adam                                                                 |
|                 | Learning rate                     | $2 \times 10^{-4}$                                                   |
|                 | Weight decay                      | $1 \times 10^{-4}$                                                   |
|                 | Epochs                            | 150                                                                  |
|                 | Batch size                        | 8                                                                    |

*continued on next page*

Table A.1 – continued from previous page

| Category       | Hyperparameter                         | Value                                           |
|----------------|----------------------------------------|-------------------------------------------------|
|                | Precision                              | AMP (mixed FP16/FP32)                           |
|                | Gradient clipping (max norm)           | 1.0                                             |
| Loss           | Coordinate loss                        | SmoothL1                                        |
|                | Box loss                               | GIoU ( $\lambda_{\text{GIoU}} = 2.0$ )          |
|                | Confidence loss                        | BCEWithLogitsLoss                               |
|                | Matching strategy                      | Greedy (max IoU per GT)                         |
| Regularisation | Firing rate target                     | 20%                                             |
|                | FRR penalty ( $\lambda_{\text{FRR}}$ ) | 5.0                                             |
|                | FRR form                               | $\lambda \cdot (\text{target} - \text{rate})^2$ |
|                | Head dropout                           | 0.0                                             |
| LR Schedule    | Scheduler                              | ReduceLROnPlateau                               |
|                | Mode / metric                          | max (EMA IoU)                                   |
|                | Reduction factor                       | 0.5                                             |
|                | Patience                               | 15 epochs                                       |
|                | EMA smoothing ( $\beta$ )              | 0.9                                             |
|                | IoU gate (scheduler activation)        | 0.10                                            |

## A.2 DetectorNX-G3-CNN Hyperparameters

| Category        | Hyperparameter                  | Value                                                                |
|-----------------|---------------------------------|----------------------------------------------------------------------|
| Parameter Count | Number of Parameters (M)        | 11.73                                                                |
| Architecture    | Temporal bins ( $T$ )           | 10                                                                   |
|                 | Input channels ( $C$ )          | 2 (polarity)                                                         |
|                 | Effective input channels        | $T \times C = 20$ (flattened)                                        |
|                 | Input resolution                | $180 \times 320$                                                     |
|                 | Backbone blocks                 | 4 (ResBlockCNN)                                                      |
|                 | Channel progression             | $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512$ |
|                 | Stride pattern                  | 2, 2, 2, 1, 1 (stem, B1, B2, B3, B4)                                 |
|                 | SE reduction ratio              | 16                                                                   |
|                 | Head projection channels        | 128                                                                  |
|                 | Adaptive pool size              | $8 \times 8$                                                         |
|                 | Pool strategy                   | Avg + Max (element-wise)                                             |
|                 | Activation function             | ReLU                                                                 |
|                 | Detection slots                 | $5 \times 4$ coords + 1 conf)                                        |
|                 | Temporal handling               | Flattened into channel dim (single pass)                             |
| Training        | Optimiser                       | Adam                                                                 |
|                 | Learning rate                   | $2 \times 10^{-4}$                                                   |
|                 | Weight decay                    | $1 \times 10^{-4}$                                                   |
|                 | Epochs                          | 150                                                                  |
|                 | Batch size                      | 8                                                                    |
|                 | Precision                       | AMP (mixed FP16/FP32)                                                |
|                 | Gradient clipping (max norm)    | 1.0                                                                  |
| Loss            | Coordinate loss                 | SmoothL1                                                             |
|                 | Box loss                        | GIoU ( $\lambda_{\text{GIoU}} = 2.0$ )                               |
|                 | Confidence loss                 | BCEWithLogitsLoss                                                    |
|                 | Matching strategy               | Greedy (max IoU per GT)                                              |
| Regularisation  | Firing rate regularisation      | N/A (no spiking neurons)                                             |
|                 | Head dropout                    | 0.0                                                                  |
| LR Schedule     | Scheduler                       | ReduceLROnPlateau                                                    |
|                 | Mode / metric                   | max (EMA IoU)                                                        |
|                 | Reduction factor                | 0.5                                                                  |
|                 | Patience                        | 15 epochs                                                            |
|                 | EMA smoothing ( $\beta$ )       | 0.9                                                                  |
|                 | IoU gate (scheduler activation) | 0.10                                                                 |

Table A.2: DetectorNX-G3-CNN hyperparameters (11.73M parameters).

### A.3 Qualitative Audit Samples

This appendix presents 20 paired qualitative audit samples comparing `DetectorNX-G3-SNN` (left) and `DetectorNX-G3-CNN` (right) predictions on the ETraM validation set. Each row shows the same input scene, allowing direct visual comparison of detection behaviour between the two architectures. Samples are grouped by what they are intended to demonstrate, beginning with baseline-competence cases and progressing through the architectures’ divergent behaviour, calibration evidence, and shared failure modes.

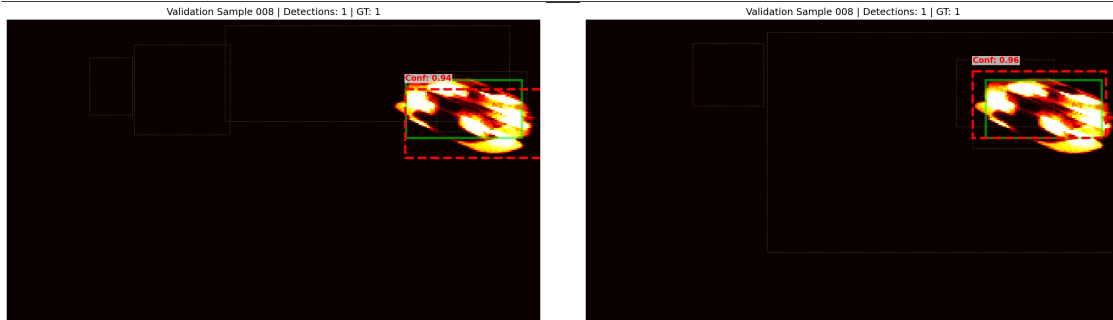
Within each rendered frame, the green solid box denotes the ground-truth annotation, the red dashed box denotes the model’s prediction, and per-detection confidence scores are overlaid in red. The frame header records the validation-set sample index together with the prediction and ground-truth object counts.

#### DetectorNX-G3-SNN

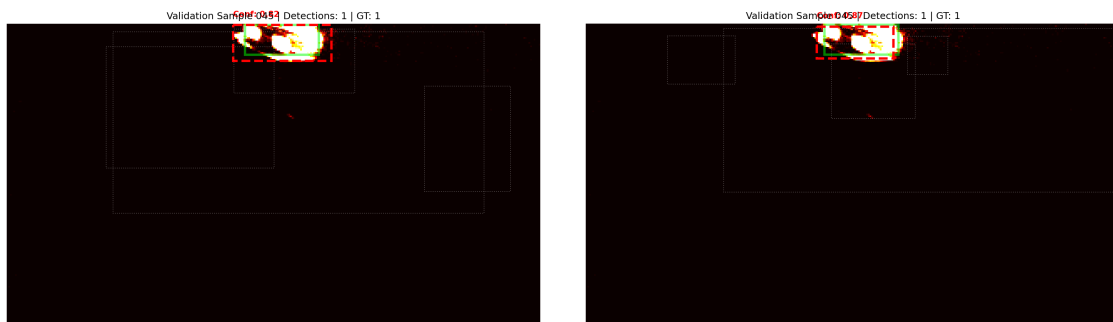
#### DetectorNX-G3-CNN

### 1. Single-Vehicle Successes

*Baseline-competence cases. Both models tightly localise the ground-truth box at high confidence.*



*Sample 008*

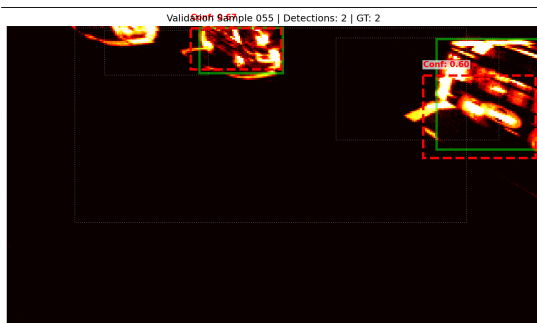
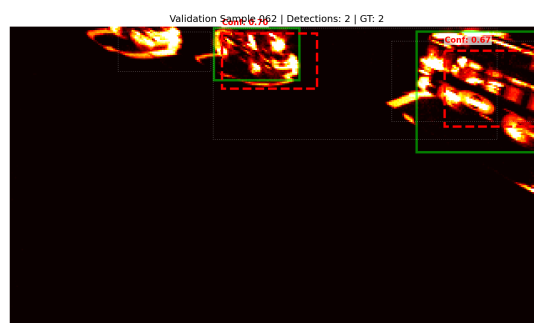
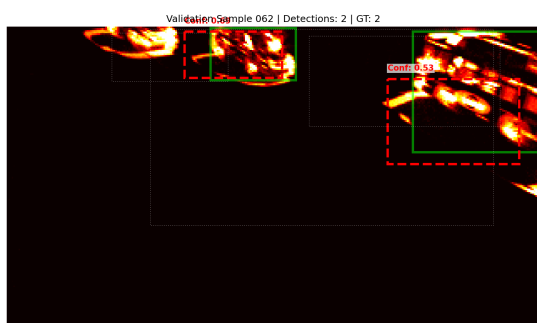
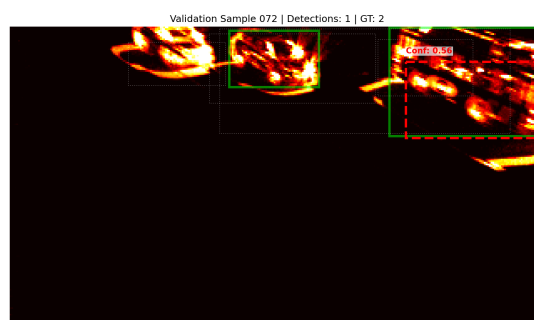
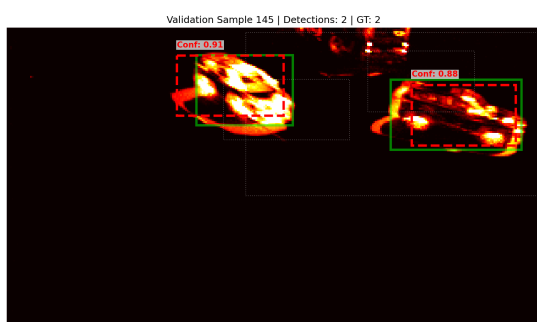


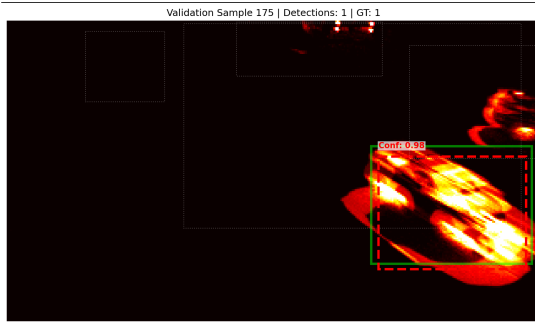
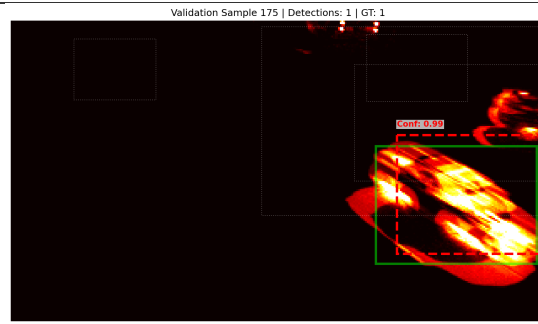
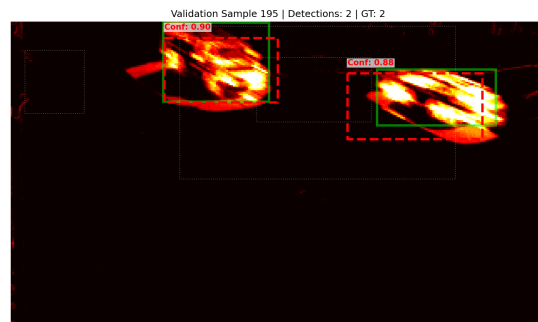
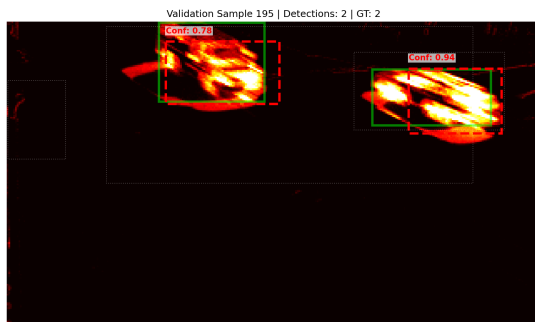
*Sample 045*

### 2. Multi-Vehicle Scenes — Both Models Correct

*Six two-vehicle scenes spanning different spatial layouts and event densities. The repeated competence here is intentional: parity is not a single lucky frame.*

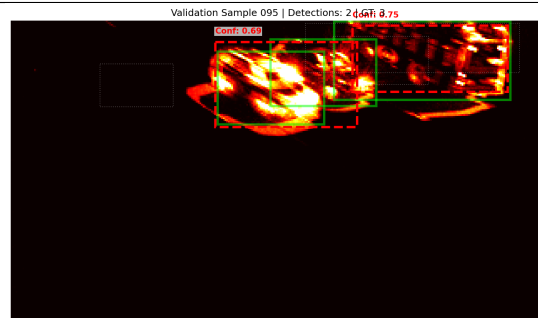
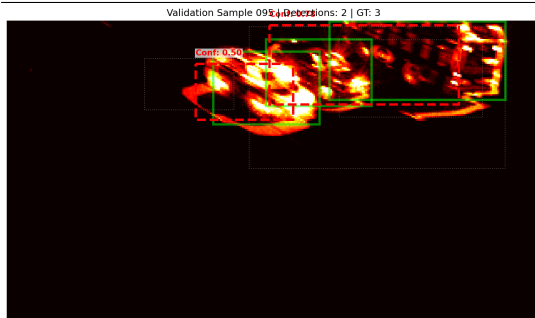
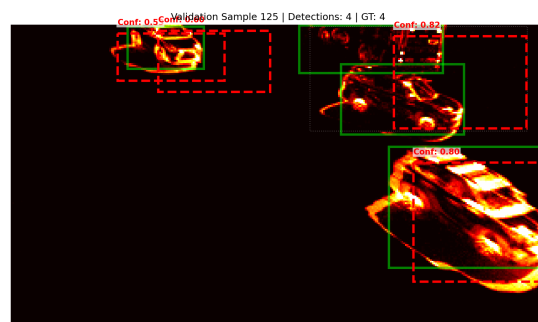
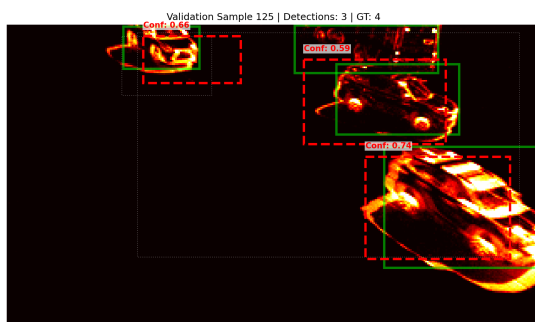
*continued on next page*

**DetectorNX-G3-SNN****DetectorNX-G3-CNN***Sample 055**Sample 062**Sample 072**Sample 145**continued on next page*

**DetectorNX-G3-SNN****DetectorNX-G3-CNN***Sample 175**Sample 195*

### 3. Dense Scenes (Three or More Objects)

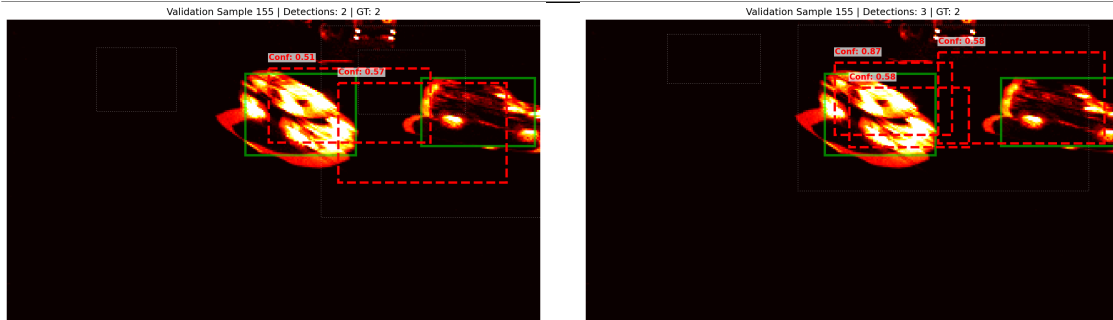
Higher-density frames stress-testing both architectures as object count rises. Sample 125 (four ground-truth vehicles) is the strongest density-handling case in the validation set.

*Sample 095*

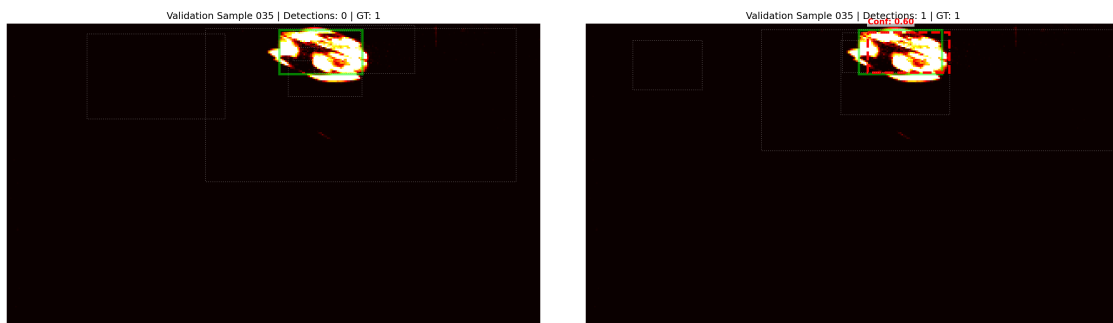
*continued on next page*

**DetectorNX-G3-SNN****DetectorNX-G3-CNN***Sample 125***4. Architectural Divergence — Each Model Stronger on One Scene**

*One scene where the SNN finds a vehicle the CNN misses, and one in the opposite direction. Both error directions exist; included to avoid one-sided framing.*



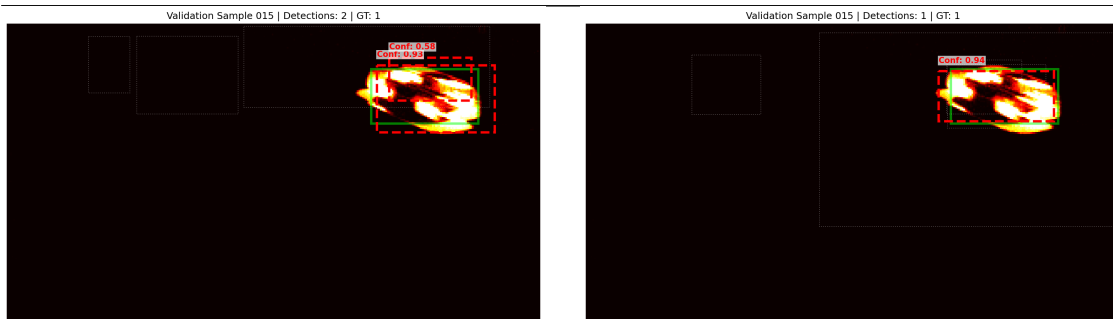
*Sample 155 — SNN detects both objects; CNN misses one.*



*Sample 035 — CNN detects the GT object; SNN misses it entirely.*

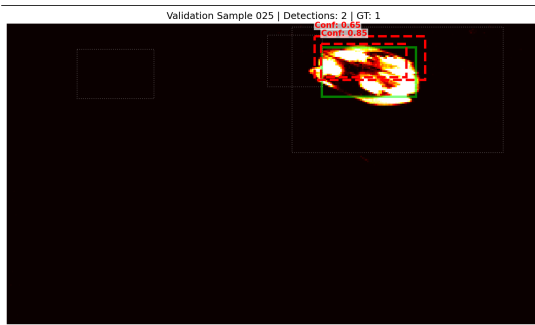
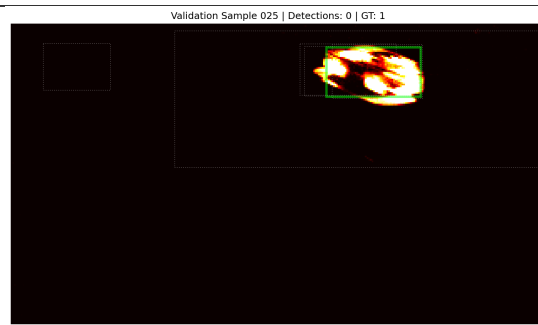
**5. Divergent Counts — Over- and Under-Counting**

*Scenes where the two models disagree on the number of detections. Over-counting is not unique to either architecture.*

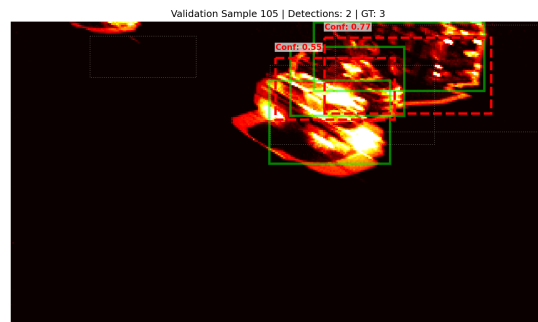
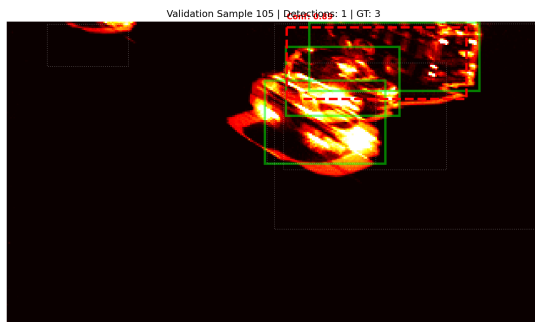


*Sample 015 — SNN reports two detections on a single-object scene; CNN clean.*

*continued on next page*

**DetectorNX-G3-SNN****DetectorNX-G3-CNN**

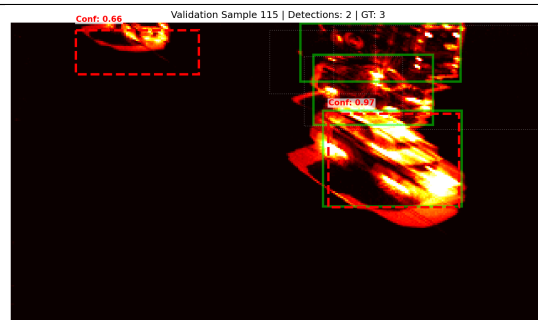
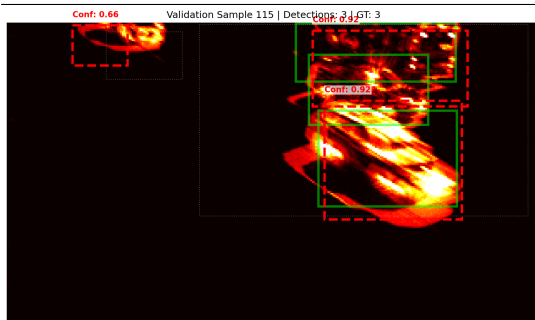
*Sample 025 — SNN over-counts; CNN misses entirely. Most divergent pair in the appendix.*



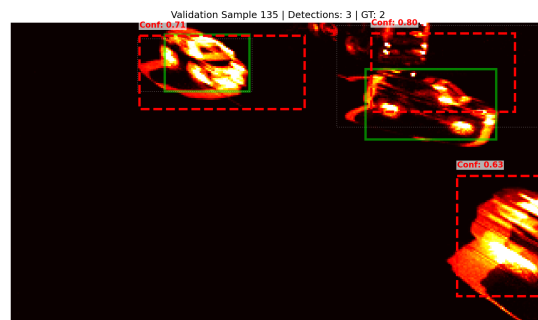
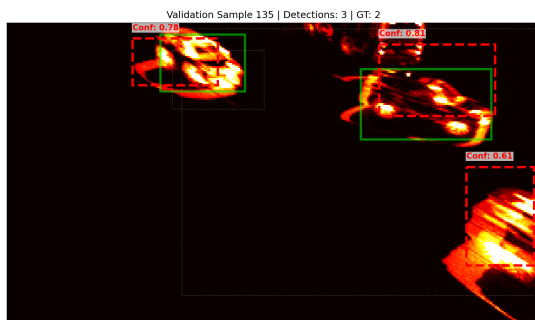
*Sample 105 — Mirror of 015: CNN over-counts; SNN clean.*

## 6. Calibration Evidence

*Cases reinforcing the implicit-uncertainty-sensor pattern characterised in Section 4.5: the SNN's confidence honestly tracks correctness, including dropping toward zero on ambiguous detections.*



*Sample 115 — Confidence-vs-IoU divergence between the two architectures.*

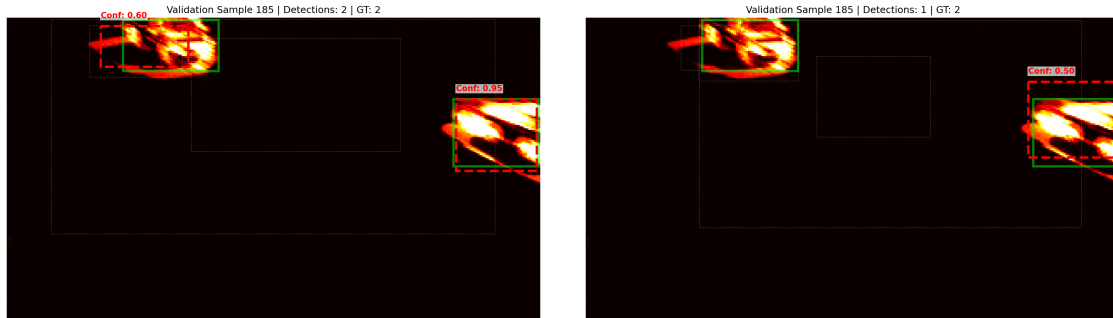


*continued on next page*



**DetectorNX-G3-SNN****DetectorNX-G3-CNN**

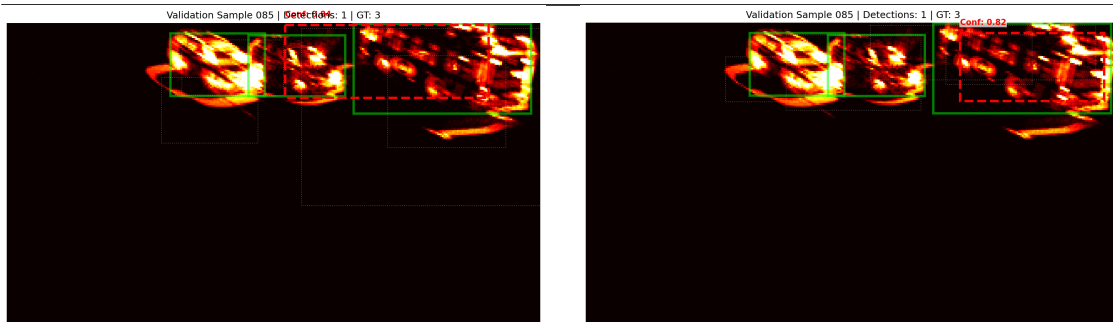
*Sample 135 — SNN reports a low-confidence detection ( $\sim 0.41$ ) on an ambiguous scene.*



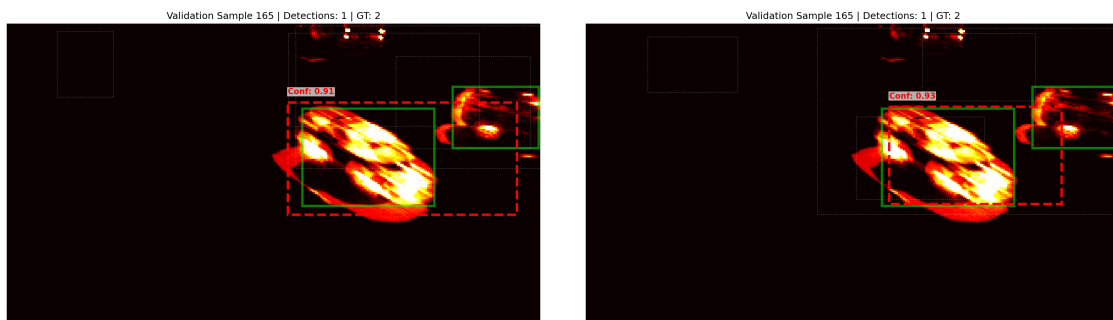
*Sample 185 — Very-low-confidence SNN prediction ( $\sim 0.06$ ) that nonetheless aligns with the GT box.*

## 7. Hard Scenes — Both Models Miss an Object

*Shared-failure scenes where both architectures drop one of the ground-truth vehicles. Honest acknowledgement of the validation set's harder cases.*



*Sample 085 — Both models report one of two ground-truth objects.*



*Sample 165 — Same shared-miss pattern at a different validation-set point.*

**Full audit set.** The complete collection of 200 audit samples per model is available at the following link:

<https://drive.google.com/drive/folders/1h6E90kFM59cgvdx0-J-FcohDKHsyNBEA?usp=sharing>